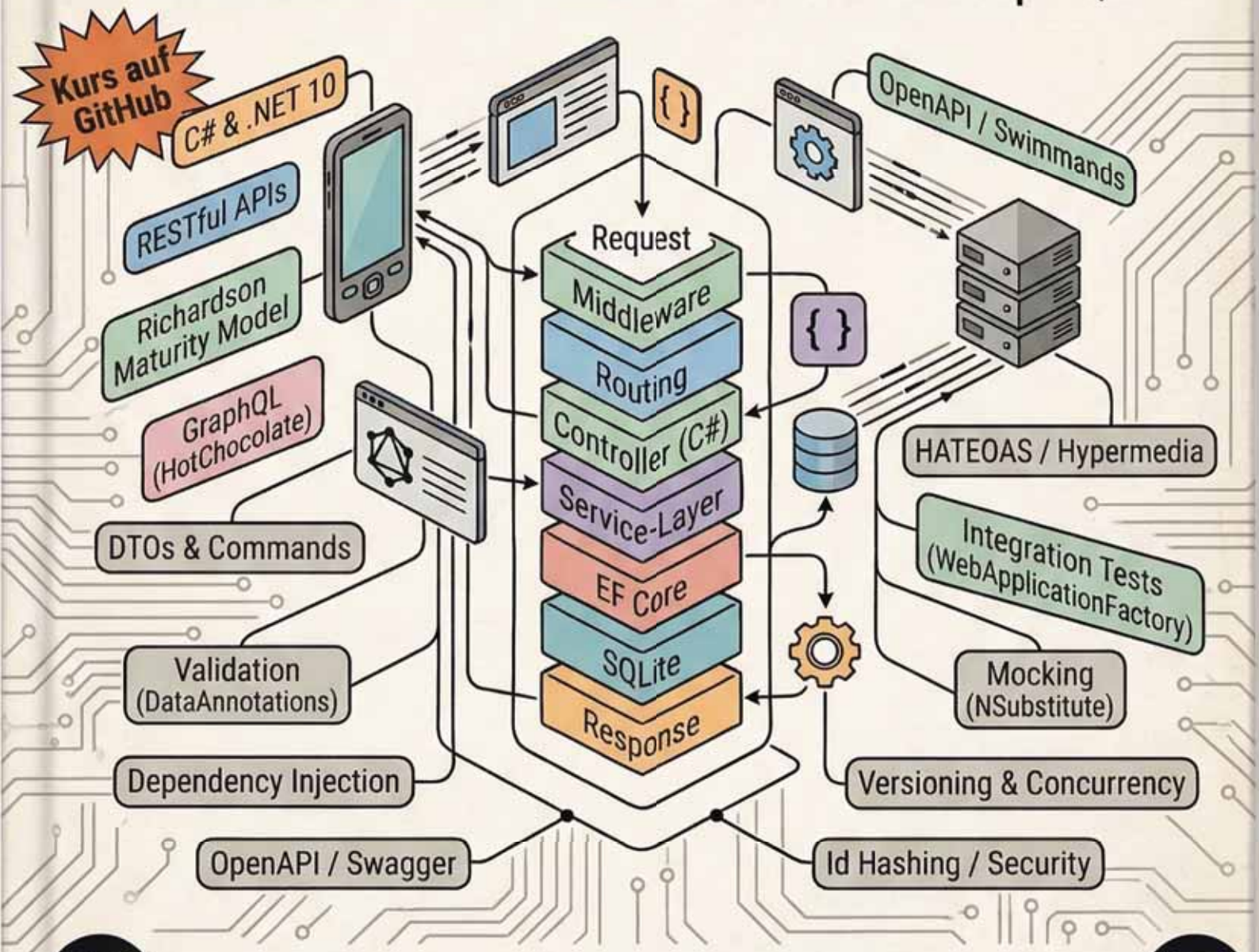
Kurs auf GitHub

Ausgabe vom 20.03.2026

API-Entwicklung mit ASP.NET Core

Das umfassende Praxis-Handbuch: Vom RMM Level 0 bis HATEOAS & GraphQL



Praxisprojekt: Eventmanager

REST Controller & Minimal APIs

GraphQL Integration

Unit- & Integrationstests

Michael Schletz

Spengergasse

Inhaltsverzeichnis

RESTful APIs mit ASP.NET Core . . . 3
- Was ist REST? . . . 3
- Das Richardson Maturity Model . . . 5

RESTful APIs mit ASP.NET Core . . . 8
- Frameworks zur Entwicklung von RESTful APIs . . . 8
- Anlegen unseres ersten ASP.NET Core Projektes . . . 8
- Konfiguration unserer API . . . 10
- Schritt 1: Service-Konfiguration (Builder-Phase) . . . 12
- Schritt 2: Request-Pipeline & Laufzeitverhalten . . . 12
- Schritt 3: Endpunkte und Start . . . 13

Controller in ASP.NET Core: GET Routen . . . 14
- Zusammenspiel zwischen Routing und Controller . . . 14
- Der Controller in ASP.NET Core . . . 14
- Dependency Injection . . . 17
- Erstellen eines HTTP Files zum Testen des Controllers . . . 18

POST Routen, Services und Validierung . . . 20
- Das Event Service . . . 20

PUT, PATCH und DELETE . . . 29
- Der Time Provider . . . 29
- Ressourcen ersetzen mit PUT . . . 29
- Teilweises Aktualisieren mit PATCH . . . 35
- Löschen mit DELETE . . . 36

Servicetests mit der SQLite in-memory DB . . . 39
- Analyse der Dependencies . . . 39
- Aufbau der Testklasse . . . 40
- Erstellen der Musterdaten . . . 41
- Schreiben des ersten Tests . . . 42
- Parametrisierte Tests . . . 43

Integration Tests mit ASP.NET Core . . . 47
- Die Testpyramide . . . 47
- Integration Tests mit ASP.NET Core . . . 47

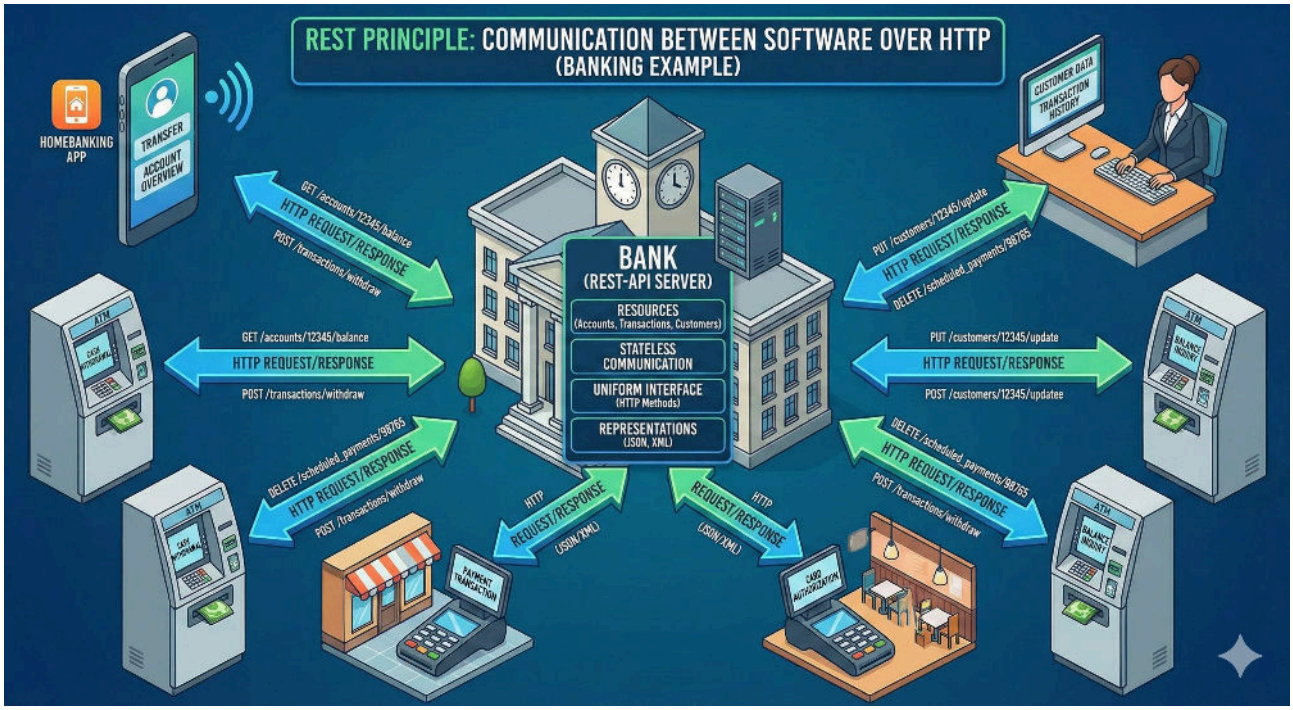
○ Mocking mit NSubstitute . . . 55
- Warum Mocking? . . . 55
- Das Paket NSubstitute . . . 55
- Mocken des Services im Controllertest . . . 56

○ Das Repository Pattern . . . 60
- Das Problem mit dem DbContext . . . 60

Implementierung eines Repositories . . . 61
Verwendung im Service . . . 67
Unittest mit Mock des Repos . . . 68
○ Codierung von ID Werten . . . 73
Das Problem mit auto increment ID Werten . . . 73
Die Id als eigener Typ . . . 73
Verwendung im Controller . . . 76
Gesamtüberblick . . . 77
○ Hypermedia und HATEOAS . . . 79
Das Prinzip von Hypermedia . . . 79
Der HAL-Standard und die Realität . . . 79
Konkrete Implementierung in ASP.NET Core . . . 81
Einsatz in einer SPA (Next.js) . . . 84
○ GraphQL mit ASP.NET Core und HotChocolate . . . 86
Geschichte und Motivation von GraphQL . . . 86
GraphQL im Vergleich zu REST . . . 87
Implementierung in ASP.NET Core mit HotChocolate . . . 88
Konfiguration des ASP Servers . . . 89
Abfragebeispiele in Bezug auf das Programm . . . 91
GraphQL Syntax und Möglichkeiten im Detail . . . 92
Verwendung in einer SPA (Next.js) . . . 96
○ Minimal APIs in ASP.NET Core: Moderne Web-Entwicklung . . . 98
Minimal API vs. Controller . . . 98
Implementierung von Endpunkten . . . 99
Unittests und Integration Tests: Stabilität durch Entkopplung . . . 101
Resümee: Wann sollte man welchen Ansatz wählen? . . . 102

RESTful APIs mit ASP.NET Core

Was ist REST?



Beispiel für Softwarekommunikation. Quelle: Google Gemini KI

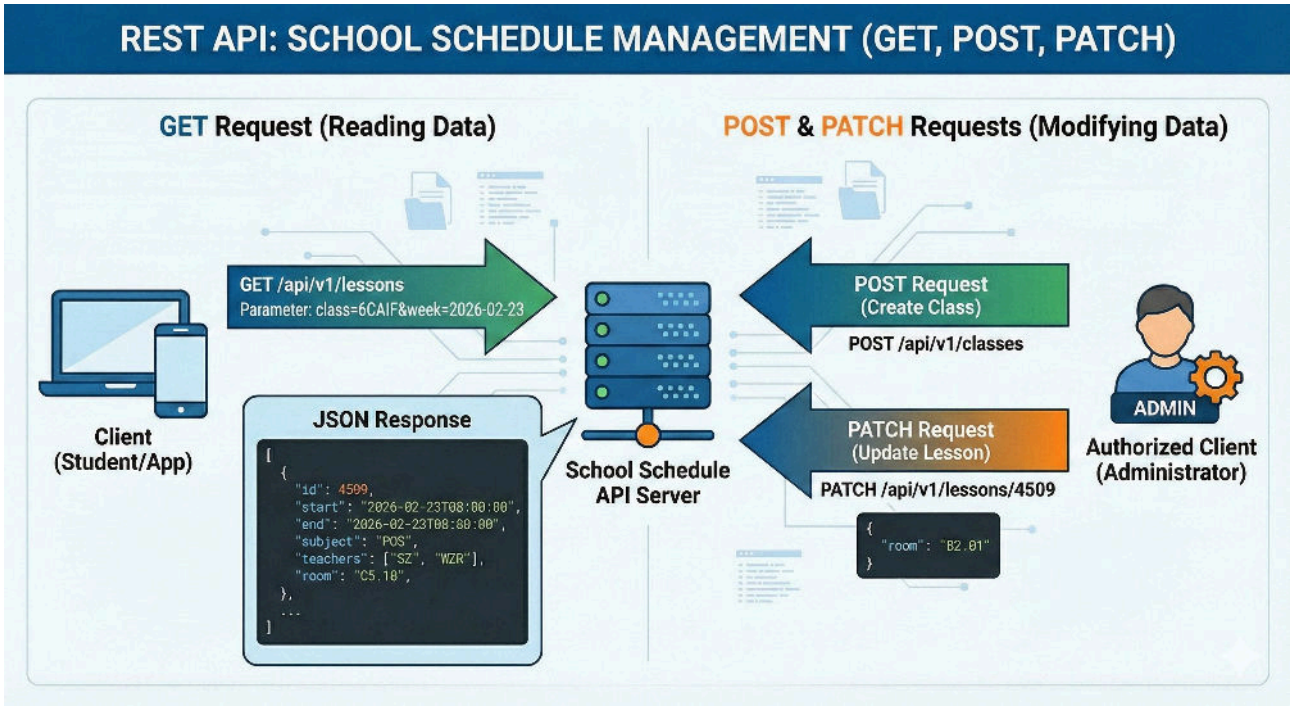
> **Representational State Transfer** (abgekürzt REST) ist ein Paradigma für die Softwarearchitektur von verteilten Systemen, insbesondere für Webservices. REST ist eine Abstraktion der Struktur und des Verhaltens des World Wide Web. REST hat das Ziel, einen Architekturstil zu schaffen, der den Anforderungen des modernen Web besser genügt. Dabei unterscheidet sich REST vor allem in der Forderung nach einer einheitlichen Schnittstelle (siehe Abschnitt Prinzipien) von anderen Architekturstilen.
>
> Quelle: https://de.wikipedia.org/wiki/Representational_State_Transfer

Ein zentrales Merkmal von REST ist die **Zustandslosigkeit**. Das bedeutet, dass jede Anfrage vom Client an den Server alle Informationen enthalten muss, die zum Verständnis und zur Bearbeitung der Anfrage notwendig sind. Der Server speichert keinen Sitzungskontext des Clients; dadurch sind REST-Systeme hervorragend skalierbar und einfach zu warten, da Anfragen unabhängig voneinander bearbeitet werden können.

Für die Interaktion mit Ressourcen verwendet REST standardisierte HTTP-Methoden wie GET (Lesen), POST (Erstellen), PUT (Aktualisieren) und DELETE (Löschen). Die Daten werden dabei meist in einem leichtgewichtigen Format wie JSON oder XML übertragen. Durch diese Trennung von Client und Server sowie die Nutzung einheitlicher Schnittstellen ermöglicht REST eine flexible Entwicklung, bei der Frontend und Backend weitgehend unabhängig voneinander evolvieren können.

Konkretes Beispiel: WebUntis

In einer Stundenplan-App wird der Stundenplan als Ressource behandelt, die über spezifische Parameter (wie die ID einer Klasse) gefiltert wird. Eine typische Anfrage nutzt die URL-Struktur, um genau zu definieren, welcher Datensatz benötigt wird.



HTTP Requests für WebUntis. Quelle: Google Gemini KI

Um den Plan für eine bestimmte Klasse abzurufen, sendet der Client eine GET-Anfrage.

- **Methode:** GET
- **Endpunkt:** /api/v1/lessons
- **Parameter:** class=6CAIF&week=2026-02-23

Antwort:

```
[
    {
        "id": 4509,
        "start": "2026-02-23T08:00:00",
        "end": "2026-02-23T08:50:00",
        "subject": "POS",
        "teachers": ["SZ", "WZR"],
        "room": "C5.10",
    },
    ...
]
```

Neben dem Lesen können autorisierte Benutzer (z.B. Administratoren) Ressourcen verändern:

- **POST /api/v1/classes:** Erstellt eine neue Klasse.

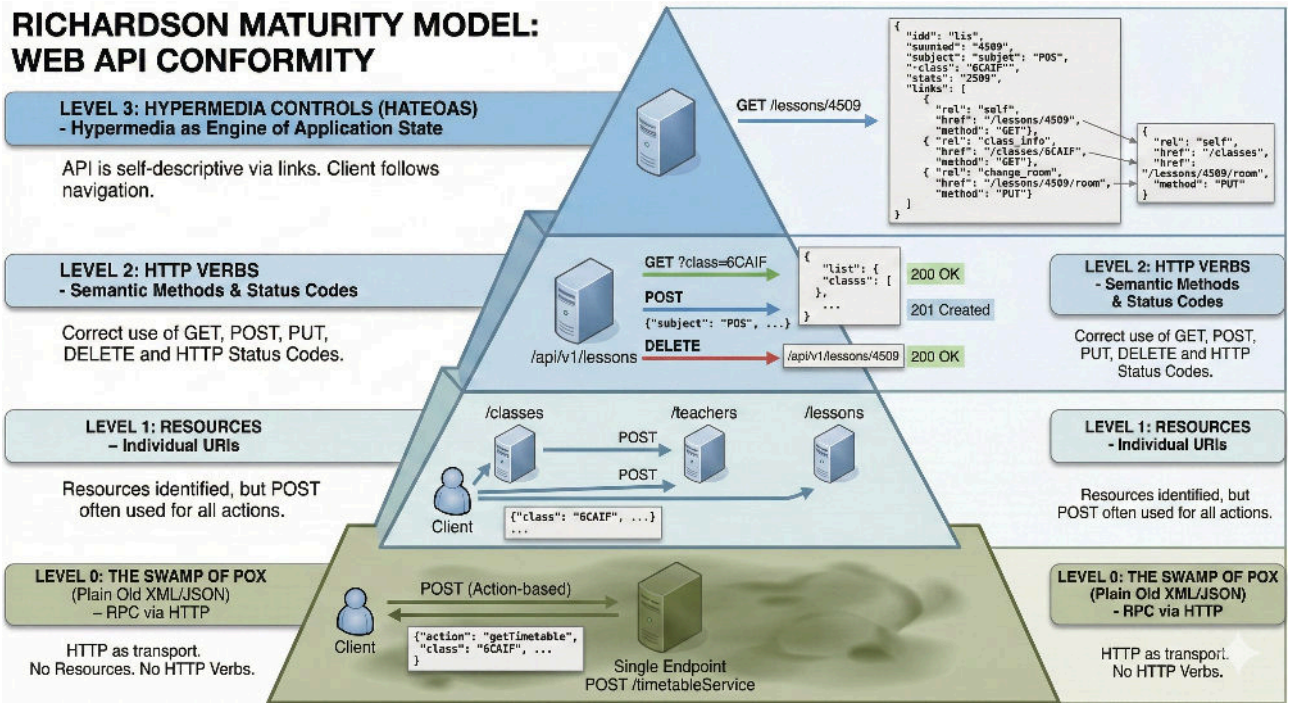
- **PATCH /api/v1/lessons/4509:** Ändert den Raum einer spezifischen Stunde (Raumwechsel).

Die HTTP Verben im Überblick

Befehl	Beschreibung
GET	Fordert die angegebene Ressource vom Server an. GET weist keine Nebeneffekte auf. Der Zustand am Server wird nicht verändert.
POST	Fügt eine neue (Sub-)Ressource unterhalb der angegebenen Ressource ein. Da die neue Ressource noch keinen URI besitzt, adressiert der URI die übergeordnete Ressource. Als Ergebnis wird der neue Ressourcenlink dem Client zurückgegeben. POST kann im weiteren Sinne auch dazu verwendet werden, Operationen abzubilden, die von keiner anderen Methode abgedeckt werden.
PUT	Die angegebene Ressource wird angelegt. Wenn die Ressource bereits existiert, wird sie geändert.
PATCH	Ein Teil der angegebenen Ressource wird geändert. Hierbei sind Nebeneffekte erlaubt.
DELETE	Löscht die angegebene Ressource.
HEAD	Fordert Metadaten zu einer Ressource an.
OPTIONS	Prüft, welche Methoden auf einer Ressource zur Verfügung stehen.

Das Richardson Maturity Model

> The Richardson Maturity Model (RMM) is a maturity model suggested in 2008 by Leonard Richardson which classifies Web APIs based on their adherence and conformity to each of the model's four levels. The aim of the research of the model as stated by the author was to find out the relationship between the constraints of REST and other forms of web services.
>
> It divides the principal parts of RESTful design into three steps: resource identification (URI), HTTP verbs, and hypermedia controls (e.g. hyperlinks).
>
> The RMM has been cited useful in evaluating the quality of particularly RESTful Web API design (even though it is not restricted to REST alone) and criticized for not addressing how a system could achieve the highest maturity levels of the model as well as for considering a limited number of quality attributes.
>
> Quelle: https://en.wikipedia.org/wiki/Richardson_Maturity_Model



Richardson Maturity Model. Quelle: Google Gemini KI

Level 0: The Swamp of POX (Plain Old XML/JSON)

Auf diesem Level wird HTTP lediglich als Transportmedium für Remote Procedure Calls (RPC) genutzt. Es gibt meist nur einen einzigen Endpunkt, und alle Aktionen werden über POST abgewickelt.

- **Endpunkt:** POST /timetableService
- **Anfrage (Body):** {"action": "getTimetable", "class": "6CAIF", "date": "2026-02-23"}
- **Charakteristik:** Keine Nutzung von HTTP-Methoden oder URIs zur Ressourcenidentifikation.

Level 1: Resources (Ressourcen)

Hier werden individuelle URIs für verschiedene Ressourcen eingeführt. Man spricht nicht mehr einen zentralen Service an, sondern einzelne Objekte.

- /classes
- /teachers
- /lessons

Wenn wir den Stundenplan der 6CAIF abrufen wollen, könnte eine Anfrage so aussehen:

- **Endpunkt:** POST /lessons
- **Anfrage (Body):** {"class": "6CAIF", "date": "2026-02-23"}
- **Charakteristik:** Ressourcen sind unterscheidbar, aber es wird oft immer noch nur POST für alles verwendet.

Level 2: HTTP Verbs (HTTP-Methoden)

In diesem Level werden die HTTP-Methoden korrekt und semantisch genutzt (GET zum Lesen, POST zum Erstellen, DELETE zum Löschen). Zudem werden korrekte HTTP-Statuscodes (z. B. 200 OK, 201 Created, 404 Not Found) zurückgegeben.

Als Beispiel dient der oben beschriebene GET Request an */api/v1/lessons?class=6CAIF*.

Level 3: Hypermedia Controls (HATEOAS)

Das höchste Level führt HATEOAS (Hypermedia as the Engine of Application State) ein. Die Antwort des Servers enthält Links, die dem Client mitteilen, welche Aktionen als Nächstes möglich sind.

- **Anfrage:** GET /lessons/4509

Antwort (JSON):

```
{
    "id": 4509,
    "start": "2026-02-23T08:00:00",
    "end": "2026-02-23T08:50:00",
    "subject": "POS",
    "teachers": ["SZ", "WZR"],
    "room": "C5.10",
    "links": [
        {"rel": "self", "href": "/lessons/4509", "method": "GET"},
        {"rel": "class_info", "href": "/classes/6CAIF", "method": "GET"},
        {"rel": "change_room", "href": "/lessons/4509/room", "method": "PUT"},
    ]
}
```

Charakteristik: Die API ist "selbsterklärend". Der Client muss die URL-Struktur nicht im Voraus kennen, sondern folgt den Links.

Zusammenfassung

- **Level 0:** Ein Endpunkt für alles.
- **Level 1:** Viele Endpunkte für verschiedene Dinge.
- **Level 2:** Richtige Nutzung von GET, POST, etc.
- **Level 3:** Die API liefert "Navigationsanweisungen" (Links) mit.

In der Praxis erreichen die meisten modernen APIs Level 2. Level 3 ist der "reine" REST-Gedanke, wird aber aufgrund der höheren Komplexität bei der Implementierung seltener vollständig umgesetzt.

RESTful APIs mit ASP.NET Core

Frameworks zur Entwicklung von RESTful APIs

Die Entwicklung von RESTful APIs hat sich über die Jahrzehnte von monolithischen Java-Schwergewichten hin zu flexiblen, Cloud-nativen Microservices stark gewandelt. Je nach Ökosystem (JavaScript, Java oder .NET) bieten die verschiedenen Frameworks unterschiedliche Ansätze in Bezug auf Konfiguration, Performance und Struktur.

JavaScript/TypeScript: Express und Node.js

Express gilt als der „Großvater“ der Node.js-Frameworks. Es wurde 2010 veröffentlicht und verfolgt einen minimalistischen, unvoreingenommenen (unopinionated) Ansatz. Entwickler haben hier die volle Freiheit, wie sie Middleware, Routing und Datenbanken integrieren. Eine API wird in Express manuell durch das Definieren von Routen (z. B. *app.get('/api/users', ...)*) aufgebaut. Es ist ideal für reine Backend-Dienste und Microservices, erfordert aber viel Eigenregie bei der Projektstruktur.

Java: Spring Boot

Spring Boot revolutionierte die Java-Welt bei seinem Erscheinen im Jahr 2014. Zuvor war die Entwicklung von Java-Webanwendungen (Java EE) berüchtigt für extrem komplexe XML-Konfigurationen und langsame Deployments. Spring Boot führte das Prinzip „Convention over Configuration“ ein: Es bietet fertige Starter-Pakete, die einen eingebetteten Server (wie Tomcat) mitbringen. REST-APIs werden hier sehr deklarativ mittels Annotationen wie @RestController und @GetMapping umgesetzt. Es ist heute der Industriestandard für robuste, hochskalierbare Enterprise-Anwendungen.

C#: ASP.NET Core

Die Geschichte von ASP.NET Core begann 2016 als kompletter Rewrite des alten, Windows-gebundenen .NET Frameworks. Microsoft öffnete sich damit für Open Source und Cross-Platform-Entwicklung (Linux/macOS). Für RESTful APIs nutzt man in ASP.NET Core meist den Web API-Controller-Ansatz oder die seit .NET 6 populären Minimal APIs, die den Code-Overhead massiv reduzieren. Es zeichnet sich durch eine extrem hohe Performance aus und bietet durch die starke Typisierung von C# eine hohe Typsicherheit, die besonders in großen Teams geschätzt wird.

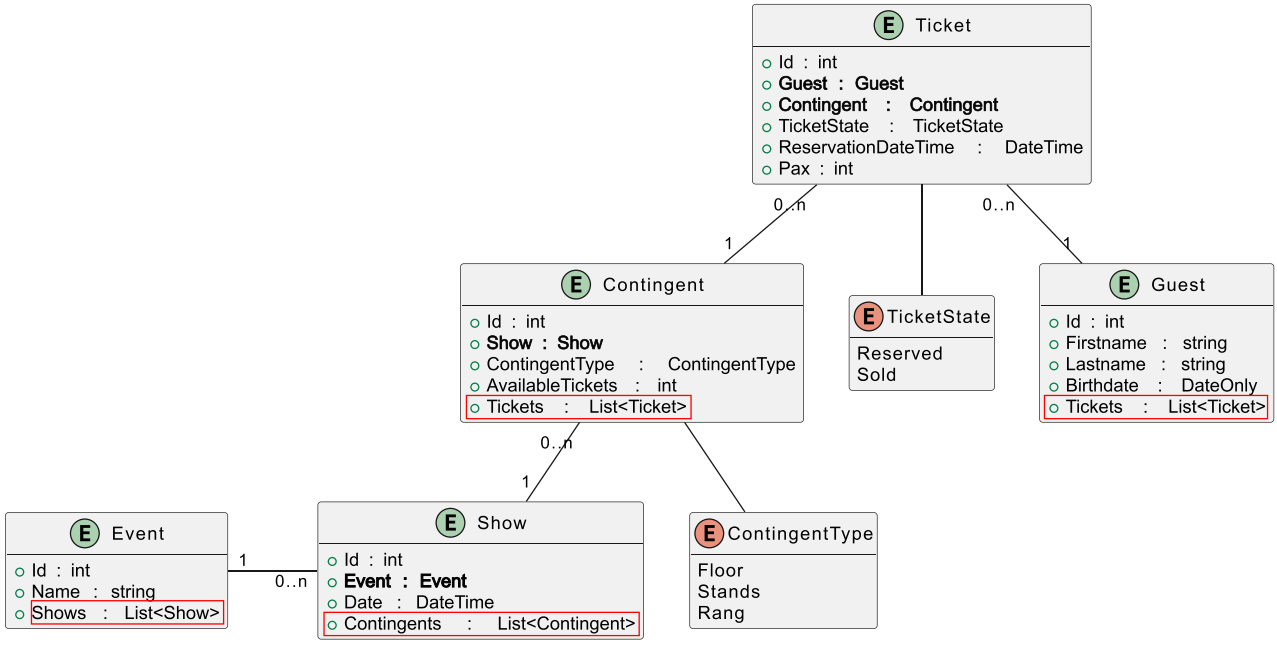
Anlegen unseres ersten ASP.NET Core Projektes

Im Kapitel *Services* haben wir für ein Verwaltungssystems für Events Abfragen und Servicemethoden geschrieben. Nun wollen wir Endpunkte bereitstellen. Dafür legen wir in der Konsole ein leeres .NET 10 Projekt an.

```
mkdir EventManager
```

```
cd EventManager
mkdir EventManager.Application
cd EventManager.Application
dotnet new classlib
dotnet add package Bogus --version 35.6.5
dotnet add package Microsoft.EntityFrameworkCore.Sqlite --version 10.*
dotnet add package Microsoft.EntityFrameworkCore.Sqlserver --version 10.*
cd ..
mkdir EventManager.Api
cd EventManager.Api
dotnet new webapi
dotnet add reference ..\EventManager.Application
dotnet add package Swashbuckle.AspNetCore.SwaggerUI --version 10.*
cd ..
dotnet new sln -n EventManager -f sln
dotnet sln add EventManager.Api
dotnet sln add EventManager.Application
start EventManager.sln
```

Die Modelklassen, den DbContext in Infrastructure und die Serviceklassen können wir 1:1 aus dem Beispiel übernehmen. Sie stehen im Archiv → Eventmanager.Application.7z zum Download bereit.



Der Ordner *Eventmanager.Api* hat folgende Dateien, die wir nun besprechen:

```
 Properties
    launchSettings.json
 appsettings.json
 Eventmanager.Api.csproj
 Program.cs
```

Konfiguration unserer API

Die Datei appsettings.json

Das O in den *SOLID* Principles steht für "open for extension, closed for modification". ASP.NET Core setzt dieses Prinzip unter anderen durch eine *zentrale Konfiguration* um. Wir können in der Datei *appsettings.json* verschiedene Dinge Konfigurieren:

- Connectionstrings zu Datenbanken
- Loggereinstellungen
- Einstellungen für externe Services (OAuth, etc.)
- u.v.m.

Zu Beginn setzen wir den Namen der SQLite Datenbank unter *ConnectionStrings*. Im Logger stellen wir ein, dass EF Core nicht jede SELECT Anweisung protokolliert, sondern nur Warnungen und Fehler.

```
{
  "ConnectionStrings": {
    "Default": "DataSource=events.db"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Die Datei lauchSettings.json (Ordner Properties)

Wie bei jedem Webserver können wir den Port und das Verhalten des Servers einstellen. Zuerst legen wir mit der *applicationUrl* fest, dass unser Server nur auf *localhost*, Port 5080 hört. *http* bedeutet, dass Klartext HTTP verwendet wird. Die Umgebungsvariable *ASPNETCORE_ENVIRONMENT* legen wir auf *Development* fest. Sie kann später in der *Program.cs* ausgelesen werden und ermöglicht es, bestimmte Dinge nur im *Development Mode* anzubieten.

```
{
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "http://localhost:5080",
```

```
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Die Datei Program.cs

Wir ersetzen nun die generierte Datei *Program.cs* durch diesen Inhalt.

```
using Eventmanager.Infrastructure;
using Microsoft.AspNetCore.Builder;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;

public class Program
{
    private static void Main(string[] args)
    {
        // STEP 1: Configuring ASP.NET Core Services
        var builder = WebApplication.CreateBuilder(args);
        builder.Services.AddOpenApi();
        builder.Services.AddControllers();

        // Configure Datebase with settings from appsettings.json (section
ConnectionStrings)
        builder.Services.AddDbContextFactory<EventContext>(options =>
            options.UseSqlite(builder.Configuration.GetConnectionString("Default")));

        // STEP 2: Configuring ASP.NET Core request pipeline
        var app = builder.Build();
        if (app.Environment.IsDevelopment() || app.Environment.IsEnvironment(
"Testing"))
        {
            // Map http://localhost:5080/openapi/v1.json
            app.MapOpenApi();
            app.UseSwaggerUI(options =>
            {
                options.SwaggerEndpoint("/openapi/v1.json", "API v1");
            });
            using (var scope = app.Services.CreateScope())
            using (var service = scope.ServiceProvider.GetRequiredService<
EventContext>())
            {
                service.Database.EnsureDeleted();
                service.Database.EnsureCreated();
                if (app.Environment.IsDevelopment()) service.Seed();
            }
        }
        // Assign controllers to routes.
```

```
        app.MapControllers();
        app.Run();
    }
}
```

Die Konfiguration folgt dem klassischen **Builder-Pattern**, das in zwei Hauptphasen unterteilt ist:

1. **Service-Registrierung**: Definition der Abhängigkeiten (Dependency Injection).
2. **Middleware-Pipeline**: Konfiguration der Request-Verarbeitung.

Schritt 1: Service-Konfiguration (Builder-Phase)

In diesem Abschnitt werden die benötigten Komponenten im *IServiceCollection*-Container registriert.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddOpenApi();     ①
builder.Services.AddControllers(); ②
```

① **AddOpenApi**: Registriert die integrierte Unterstützung für die Generierung von OpenAPI-Dokumenten (Standard in neueren .NET Versionen).

② **AddControllers**: Ermöglicht die Nutzung von Controller-Klassen (*[ApiController]*) für das Routing und die Logik.

Datenbank-Setup

```
builder.Services.AddDbContextFactory<EventContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("Default")));
```

Hier wird der *EventContext* (Entity Framework Core) registriert. Er nutzt **SQLite** und bezieht den Verbindungsstring aus der *appsettings.json* (Sektion *ConnectionStrings:Default*).

Schritt 2: Request-Pipeline & Laufzeitverhalten

Nach *builder.Build()* wird das Verhalten der Anwendung bei eingehenden HTTP-Requests definiert.

Entwicklung vs. Testumgebung

Das Code-Snippet enthält eine spezifische Logik für Nicht-Produktionsumgebungen:

```
if (app.Environment.IsDevelopment() || app.Environment.IsEnvironment("Testing"))
{
    app.MapOpenApi(); // Stellt das JSON-Dokument bereit
```

```
    app.UseSwaggerUI(options => { ... }); // Visualisiert die API-Doku
}
```

Datenbank-Initialisierung (Seeding)

Innerhalb der Development/Testing-Bedingung wird ein **Scope** erstellt, um auf registrierte Services zuzugreifen:

- **EnsureDeleted/Created**: Löscht die Datenbank und erstellt sie neu (nützlich für einen sauberen State beim Testen).
- **Seed()**: Füllt die Datenbank mit Testdaten, sofern man sich im *Development*-Modus befindet.

Schritt 3: Endpunkte und Start

```
app.MapControllers(); // Aktiviert das Routing zu den Controllern
app.Run(); // Startet den Webserver (Kestrel)
```

Controller in ASP.NET Core: GET Routen

Zusammenspiel zwischen Routing und Controller

Wie im Kapitel RESTful APIs beschrieben, werden Ressourcen über URLs (Uniform Resource Locators) eindeutig identifiziert. Die Grundprinzipien bei der Gestaltung von RESTful URLs sind

- **Ressourcenorientiert:** Die URL steht für eine Ressource, nicht für eine Aktion. Deswegen bestehen sie auch aus Substantiven (Hauptwörtern), nicht aus Verben. Richtig: */api/timetables* Falsch: */api/getTimetable*
- **Hierarchie & Struktur:** Die URL verwendet hierarchische Pfade zur Darstellung von Zusammenhängen. */api/customers/123/orders* gibt die Bestellungen des Kunden mit der ID *123* an.
- **Pluralisierung:** Verwende konsistent Pluralformen für Ressourcen, also */api/customers* statt */api/customer*.

Aufgabe des *Routing Systems* von ASP.NET Core ist es, den URLs entsprechenden Programmcode zuzuordnen. Dabei werden 2 Ansätze angeboten:

- Minimal API
- Controller-based API

Die *Minimal API* richtet sich an Entwickler, die Erfahrung mit dem Express Paket von Node.js oder Flask aus dem Python Ökosystem haben und wenige Endpunkte z. B. für ein Microservice anbieten möchten. Auf https://learn.microsoft.com/en-us/aspnet/core/tutorials/min-web-api gibt es eine genaue Beschreibung.

Wir werden allerdings den Ansatz mit **Controllerklassen** wählen. Bei größeren Projekten fällt es leichter, den Code thematisch zu trennen und merge Konflikte zu reduzieren.

Der Controller in ASP.NET Core

Ein Controller ist - syntaktisch betrachtet - eine "normale" C# Klasse mit Attributes. Die Attributes steuern, welche URL welcher Methode zugeordnet wird.

In der Datei *Program.cs* haben wir bereits beim Anlegen des Projektes die Grundlagen geschaffen, dass ASP.NET Core diese Verknüpfung herstellen kann:

```
// STEP 1: Configuring ASP.NET Core Services
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
// ....
// STEP 2: Configuring ASP.NET Core request pipeline
var app = builder.Build();
app.MapControllers();
// ....
```

```
app.Run();
```

Die Methode *AddControllers()* durchsucht alle Klassen mit den Attribute *[ApiController]* und baut sich das Adressmapping auf. *MapControllers()* sorgt dafür, dass der Request durch die Methode im Controller abgearbeitet wird.

Fehlt einer dieser Teile, wird ASP.NET Core die Adressen nicht zuordnen können und es wird *404 not found* geliefert.

Zuerst legen wir im *Application Projekt* einen Ordner *Dtos* an und definieren 3 DTO Klassen. Sie geben an, welche Informationen wir zurückgeben wollen.

```
public record EventDto(int Id, string Name, int ShowsCount);
public record EventWithShowsDto(int Id, string Name, List<ShowDto> Shows);
public record ShowDto(int Id, DateTime Date);
```

Nun wollen wir im *API Projekt* einen ersten Controller anlegen:

Eventmanager.Api/Controllers/EventsController.cs

```
using Eventmanager.Application.Dtos;
using Eventmanager.Infrastructure;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace Eventmanager.Api.Controllers;

[Route("api/[controller]")]
[ApiController]
public class EventsController : ControllerBase
{
    private readonly EventContext _db;

    public EventsController(EventContext db)
    {
        _db = db;
    }
    [HttpGet]
    [ProducesResponseType<List<EventDto>>(StatusCodes.Status200OK)]
    public async Task<ActionResult<List<EventDto>>> GetAllEvents()
    {
        var events = await _db.Events
            .Select(e => new EventDto(e.Id, e.Name, e.Shows.Count))
            .ToListAsync();
        return Ok(events);
    }
```

```
    [HttpGet("{id}")]
    [ProducesResponseType<EventDto>(StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<ActionResult<EventDto>> GetEventById(
        [FromRoute] int id,
        [FromQuery] bool includeShows = false)
    {
        var eventData = await _db.Events
            .Where(e => e.Id == id)
            .Select(e => new EventWithShowsDto(
                e.Id, e.Name,
                includeShows
                    ? e.Shows.Select(s => new ShowDto(s.Id, s.Date)).ToList()
                    : new List<ShowDto>()))
            .FirstOrDefaultAsync();
        if (eventData is null) return Problem("Event not found", statusCode: 404);
        return Ok(eventData);
    }
}
```

Wir sehen 3 Attributes:

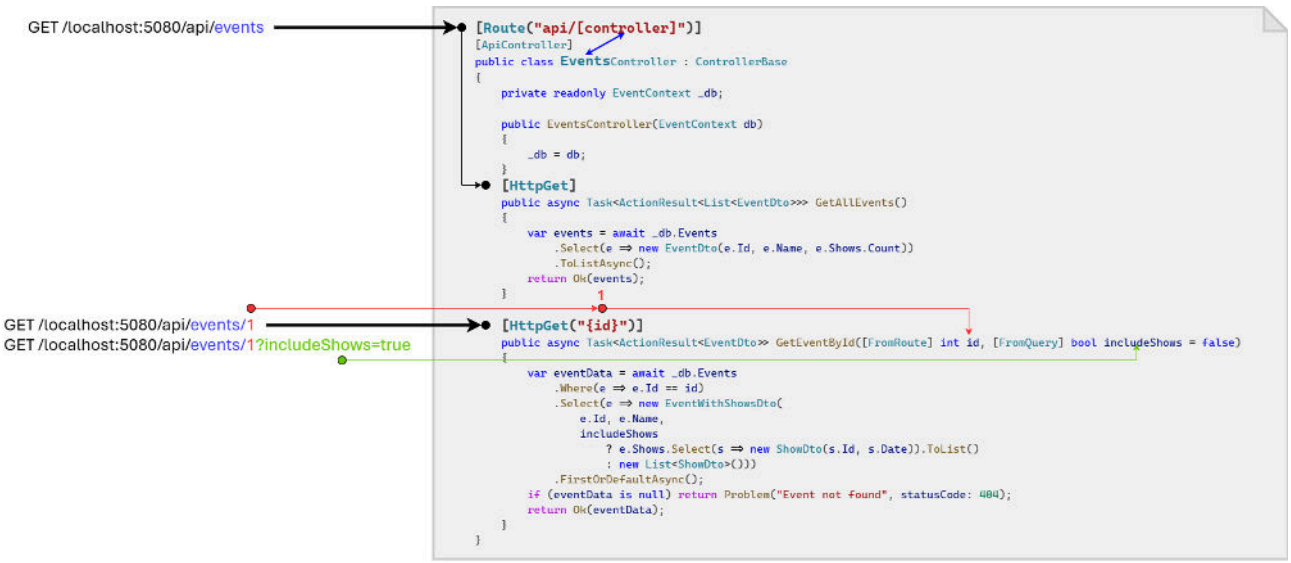
- Route
- ApiController
- HttpGet

Das Attribute ***Route*** weist die *Basisadresse* für die Controllerklasse zu. Wir können z. B. *[Route("api/events")]* schreiben, wenn der Controller auf Adressen, die mit *api/events* beginnen, hören soll. Damit wir allerdings flexibler sind, können wir statt *[Route("api/events")]* auch *[Route("api/[controller]")]* schreiben. Hier wird der Name aus dem Namen der Controllerklasse extrahiert. *EventsController* ordnet die Basisadresse *api/events* zu, *TimetablesController* die Adresse *api/timetables*, etc.

Auf Methodenebene kommt das Attribute ***HttpGet*** zum Einsatz. Ohne weitere Parameter wird es auf *GET* Requests an die Adresse des Controllers reagieren. Damit wir weitere IDs in der Adresse angeben können, schreiben wir z. B. *[HttpGet("{id}")]* Es wird den Wert nach *events* auslesen und in der Variable *id* bereitstellen.

Achte darauf, dass der Parameter von *HttpGet* genau dem Parameternamen der Methode entspricht (case sensitive)!

Die folgende Grafik zeigt die Zuordnung verschiedener Adressen.



Dependency Injection

Eine Besonderheit von ASP.NET Core ist die Unterstützung von *Dependency Injection*. Anstatt den Datenbankcontext im Controller mit *new* zu instanzieren und zu konfigurieren, haben wir eine flexiblere Möglichkeit. Der *Serviceprovider* ist dafür zuständig, Instanzen von registrierten Typen zu erzeugen. In der Datei *Program.cs* haben wir mit *AddDbContext<T>()* eine solche "Anleitung" an den Serviceprovider gegeben, wie eine Instanz der Klasse *EventContext* zu erzeugen ist.

```
// Configure Datebase with settings from appsettings.json (section ConnectionStrings)
builder.Services.AddDbContext<EventContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("Default")));
```

Nun können wir in unserem Controller einfach im Konstruktor den Typ *EventContext* anfordern. ASP.NET Core wird bei jedem Request eine Instanz nach dieser Vorschrift erzeugen und sie dem Controller übergeben.

Der primary constructor in C# 12

Unser Controller definiert einen Konstruktor, der die *private readonly* Variable *_db* initialisiert. Seit C# 12 kann die Deklaration und Initialisierung mit Hilfe eines *primary constructors* vereinfacht werden. Wir schreiben bei der Klassendeklaration direkt die Argumente des Konstruktors. C# generiert dann eine *private readonly* Variable mit diesem Namen und weist diese zu.

```
[Route("api/[controller]")]
[ApiController]
public class EventsController(EventContext db) : ControllerBase    ①
{
    [HttpGet]
    [ProducesResponseType<List<EventDto>>(StatusCodes.Status200OK)]
    public async Task<ActionResult<List<EventDto>>> GetAllEvents()
    {
```

```
        var events = await db.Events    ②
            .Select(e => new EventDto(e.Id, e.Name, e.Shows.Count))
            .ToListAsync();
        return Ok(events);
    }
    /* .... */
}
```

① Der Konstruktor wird direkt bei der Deklaration der Klasse angegeben. Das Schreiben eines eigenen Konstruktors entfällt.

② Die Variable *db* steht in der Klasse zur Verfügung.

Die Klassen *Task* und *ActionResult*

Bisher haben wir die Datenbank *blockierend* mit *ToList()*, *FirstOrDefault()* abgefragt. Besser ist es, wenn der Webserver, während er auf die Antwort der Datenbank wartet, in der Zwischenzeit andere Requests annimmt. Deswegen gibt es in C# - wie in Javascript - das Konzept der *Tasks* (in Javascript: Promise). Die Regeln sind fast ident mit Javascript, wenn man *Promise* durch *Task* ersetzt:

- Wird *await* verwendet, muss die Methode *async* sein.
- Der Rückgabewert einer *async* Methode ist *Task<T>*.
- Die Methoden *ToListAsync()* und *FirstOrDefaultAsync()* geben einen Task zurück, auf den gewartet werden kann.

Der Rückgabewert der Controllermethode ist daher einmal *Task<T>*. *T* ist wieder ein generischer Typ mit dem Namen *ActionResult*. Ein *ActionResult* ist eine Kombination von *StatusCode* und den Daten. Mit *Ok(data)* wird ein ActionResult mit Status Code *200 OK* und den übergebenen Daten zurückgegeben. Möchten wir Fehlermeldungen zurückgeben, verwenden wir die Methode *Problem(message, statusCode: code)*.

Erstellen eines HTTP Files zum Testen des Controllers

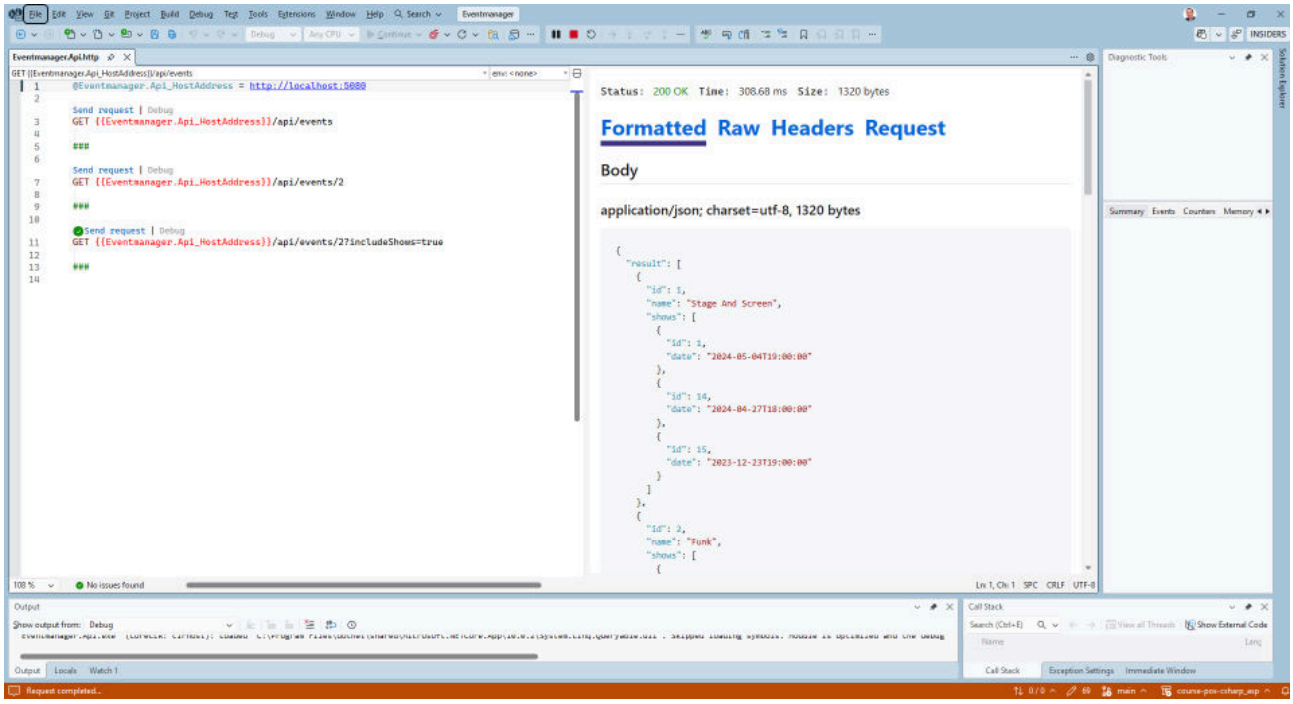
Wir erstellen nun eine Datei *Eventmanager.Api.http* mit folgendem Inhalt:

Eventmanager.Api/Eventmanager.Api.http

```
@baseUrl = http://localhost:5080

GET {{baseUrl}}/api/events

###

GET {{baseUrl}}/api/events/2

###

# Request should fail (404 not found)
GET {{baseUrl}}/api/events/999
```

```
###

GET {{baseUrl}}/api/events/2?includeShows=true

###
```

Öffnen wir diese Datei in Visual Studio, können wir mit *Send request* einen Request an diese Adresse senden. Dafür starten wir zuerst die Applikation und klicken danach auf *Send request*.



Unter *View* → *Other Windows* → *Endpoints Explorer* können diese Einträge automatisch erstellt werden.

Weitere Informationen: https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api

POST Routen, Services und Validierung

Das Event Service

Wir möchten nun mit Hilfe unserer API Kontingente für unsere Events anlegen können. Dabei gibt es verschiedene Regeln, die sogenannte *Business Logik*, zu beachten. Diese Logik sollte aus folgenden Gründen nicht direkt im Controller implementiert werden:

- **DRY (don't repeat yourself)**: Prüfungen, etc. müssen in jeder Controllermethode implementiert werden.
- **Clean Architecture**: Der Servicelayer soll ein eigenständiger Layer sein. Siehe Clean Architecture in a Nutshell und Understanding Clean Architecture.
- **Schlechte Testbarkeit**: Der Servicelayer kann nicht separat getestet werden und er kann nicht durch Mockingframeworks abgetrennt werden.

Als konkretes Beispiel wollen wir nun ein EventService anlegen. Bei größeren Projekten empfiehlt sich das Anlegen mehrerer Serviceklassen, die thematisch zusammenhängen. Das kann z. B. ein *TicketService*, ein *CustomerService*, ein *InvoiceService*, etc. sein. Wir legen das Service im *Application Projekt* an, da es keine Abhängigkeiten zu ASP.NET hat und auch keine haben soll.

→ Eventmanager.Application/Services/EventService.cs

```
using Eventmanager.Application.Commands;
using Eventmanager.Infrastructure;
using Eventmanager.Model;
using Microsoft.EntityFrameworkCore;
using System;
using System.Linq;
using System.Threading.Tasks;

namespace Eventmanager.Services;

public class EventService(EventContext db, bool isDevelopment)    ①
{
    public IQueryable<Contingent> Contingents => db.Contingents.AsQueryable();  ②

    public async Task<Contingent> CreateContingent(CreateContingentCmd cmd)     ③
    {
        if (!Enum.TryParse<ContingentType>(
            cmd.ContingentType, ignoreCase: true, out var contingentType))
            throw new EventServiceException(
                $"{cmd.ContingentType} is not a valid contingent type.");

        var show = await db.Shows.FirstOrDefaultAsync(s => s.Id == cmd.ShowId)
            ?? throw new EventServiceException($"Show {cmd.ShowId} not found.");

        if (await db.Contingents.AnyAsync(
            c => c.Show.Id == cmd.ShowId && c.ContingentType == contingentType))
            throw new EventServiceException(
                $"Show has already a contingent for {cmd.ContingentType}.");
```

```
        var contingent = new Contingent(show, contingentType, cmd.AvailableTickets);
        db.Add(contingent);
        await SaveChanges();
        return contingent;
    }

    private async Task SaveChanges()    ④
    {
        try
        {
            await db.SaveChangesAsync();
        }
        catch (DbUpdateException e)
        {
            var message = isDevelopment
                ? e.InnerException?.Message ?? e.Message
                : "Cannot write changes to database.";
            throw new EventServiceException(message);
        }
    }
}
```

① Über *dependency injection* und dem *primary constructor* (C# 12) soll später ASP.NET Core eine Instanz des *EventContext* bereitstellen. Wir erzeugen niemals im Service mit *new* eine Instanz der Datenbankverbindung! Die Variable *isDevelopment* wird später bei der Registrierung des Services gesetzt. Sie ermöglicht im Inneren des Services das Ausführen von speziellem Code, der nur im Development Mode verfügbar sein soll.

② In unserem Controller wollen wir direkten Zugriff auf den Datenbankkontext vermeiden. Da wir im Controller auch von der Datenbank lesen wollen, können wir mit folgender Idee einen lesenden Zugriff auf die Datenbank bereitstellen: Der Typ *IQueryable* stellt Funktionen zur Auswertung von Abfragen bereit. Er wird im *DbSet* implementiert, das bedeutet wir können aus dem *Set<Contingent>* mit *AsQueryable()* einen "read only" Zugriff auf das DbSet *Contingents* ermöglichen. Wenn andere DbSets benötigt werden, können diese ebenfalls als Property vom Typ *IQueryable* angeboten werden.

③ Die Methode *CreateContingent* implementiert die Businesslogik zur Erstellung eines Kontingentes. Als Parameter bekommt sie ein sogenanntes *command object*. Später dazu mehr.

④ Um Codeduplizierungen zu vermeiden, geben wir die Fehlerbehandlung für *SaveChangesAsync()* in eine eigene Methode. Eine *DbUpdateException* entsteht, wenn Aktualisierungen durch *INSERT*, *UPDATE* oder *DELETE* nicht von der Datenbank verarbeitet werden können. Dies kann durch Constraints, Regeln der referentiellen Integrität, etc. passieren. Wir reagieren darauf, indem wir die Fehlermeldung der Datenbank als Servicefehler weitergeben. **Hinweis:** Es werden sehr konkrete Fehlermeldungen an den Client zurückgegeben, die auf den internen Aufbau der Datenbank schließen lassen. Daher verwenden wir sie nur im Development Mode.

Das Interface IEventService

Das D in SOLID steht für *dependency inversion principle*. Unser Controller sollte nicht eine konkrete Implementierung des Services anfordern, sondern nur die Funktionalität, also das Inter-

face.

Über Refactoring in der IDE können wir leicht ein Interface aus unserer Serviceklasse erstellen. Es beinhaltet alle *public* Methoden und Properties. Später können wir durch Mocking Frameworks eine eigene Implementierung zum Testen der Controller erzeugen.

→ Eventmanager.Application/Services/IEventService.cs

```
using Eventmanager.Application.Commands;
using Eventmanager.Model;
using System.Linq;
using System.Threading.Tasks;

namespace Eventmanager.Services;

public interface IEventService
{
    IQueryable<Contingent> Contingents { get; }

    Task<Contingent> CreateContingent(CreateContingentCmd cmd);
}
```

Das Command Object

Im *Application Projekt* erstellen wir einen Ordner mit dem Namen *Commands*. Der Namespace *System.ComponentModel.DataAnnotations* stellt uns verschiedene Attribute bereit, um Constraints zu definieren. Dies ist vergleichbar mit den Constraints der *CREATE TABLE* Anweisung. ASP.NET Core kann diese Attributes später auslesen und entsprechend darauf reagieren. Die Validierung selbst hat mit ASP.NET Core noch nichts zu tun.

Es gibt verschiedene vorgegebene Attribute.

AllowedValues	Specifies a list of values that should be allowed in a property.
StringLength	Specifies the minimum and maximum length of characters that are allowed in a data field.
Range	Specifies the numeric range constraints for the value of a data field.
RegularExpression	Specifies that a data field value must match the specified regular expression.
EmailAddress	Validates an email address.
Url	Provides URL validation.

Dies ist nur ein kleiner Ausschnitt. Auf https://learn.microsoft.com/en-us/dotnet/api/system.componentmodel.dataannotations?view=net-10.0 werden alle Attribute aufgelistet.

Das Required Attribute

In der Doku gibt es auch ein Attribut für *Required*. Da wir aber das *nullable* Feature in C# verwenden, wird folgende Regel angewandt:

> The validation system treats non-nullable parameters or bound properties as if they had a [Required(AllowEmptyStrings = true)] attribute. By enabling Nullable contexts, MVC implicitly starts validating non-nullable properties or parameters as if they had been attributed with the [Required(AllowEmptyStrings = true)] attribute.
>
> Quelle: https://learn.microsoft.com/en-us/aspnet/core/mvc/models/validation?view=aspnetcore-10.0#non-nullable-reference-types-and-required-attribute

Leerstrings werden durch dieses Verhalten zugelassen! Wenn du dies nicht möchtest, definiere mit *StringLength* eine minimale Länge von 1. Wertetypen (structs) wie *int*, *bool*, *DateTime*, etc. werden auf den Standardwert gesetzt, wenn kein Wert übertragen wurde. Beachte, dass *DateTime* auch ein Wertetyp ist und den 1.1.0001 als Wert annimmt, wenn kein Wert übertragen wurde.

Nun wollen wir für den Parameter der Methode *CreateContingent* ein command object mit dem Namen *CreateContingentCmd* erstellen:

→ Eventmanager.Application/Commands/CreateContingentCmd.cs

```
using Eventmanager.Model;
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;

namespace Eventmanager.Application.Commands;

public record CreateContingentCmd(
    int ShowId,
    [StringLength(255, MinimumLength = 1, ErrorMessage = "Invalid contingent type.")]
①
    string ContingentType,
    [Range(1,9999)]    ②
    int AvailableTickets) : IValidatableObject    ③
{
    public IEnumerable<ValidationResult> Validate(ValidationContext validationContext)
    {
        if (!Enum.IsDefined(typeof(ContingentType), ContingentType))
        {
            yield return new ValidationResult(             ④
                "Invalid value for contingent type",
                new string[] { nameof(ContingentType) });
        }
    }
}
```

① Im Attribute *StringLength* definieren wir minimale und maximale Länge. Die minimale Länge ist wichtig, da wir keine leeren Strings ("") akzeptieren wollen. Die maximale Länge ist wichtig, damit ein Client nicht einen String mit 1 Million Zeichen übermitteln kann. Über *ErrorMessage* können wir die Fehlermeldung, die an den Client gesendet wird, steuern.

② Da *int* ein Wertetyp ist, würde, wenn *AvailableTickets* nicht übertragen wird, dieser Wert auf

0 gesetzt werden. Daher legen wir das Minimum auf 1 fest.

③ Für komplexere Validierungen können wir das Interface *IValidatableObject* implementieren. Es definiert die Methode *Validate*, die nach der Validierung der Attributes aufgerufen wird. Sie liefert einen Enumerator (also eine Liste) von Fehlermeldungen zurück. Mit *yield return* können wir in C# ein Element im Enumerator zurückliefern. Dadurch können wir auch mehrere Validierungen in diese FUnktion schreiben.

④ Das *ValidationResult* besteht aus der Fehlermeldung und einer Liste von Properties, die diesen Fehler verursacht haben. Diese Information wird an den Client gesandt und hilft ihm, z. B. die betreffenden Formularelemente hervorzuheben.

Registrieren des Services in ASP.NET Core

Bis jetzt haben wir im *Application Projekt* gearbeitet, da die implementierte Logik noch nichts mit APIs zu tun hat. Nun wollen wir in ASP.NET Core auf unser Service zugreifen. Zuerst müssen wir das Service registrieren, damit wir über dependency injection aus dem Controller heraus zugreifen können.

Es gibt verschiedene Arten von Services

Transient Service	Es wird bei jeder Verwendung des Services durch DI eine neue Instanz erzeugt.
Scoped Service	Es wird bei jedem Request eine neue Instanz erzeugt.
Singleton Service	Es wird zu Programmstart eine Instanz erzeugt, diese wird während der gesamten Laufzeit verwendet. Diese Services müssen thread safe sein und erfordern viel Erfahrung.

Wir werden immer ein *Scoped Service* anlegen, da unsere Services schnell instanziert werden können und wir keine Seiteneffekte berücksichtigen müssen. Für die Registrierung gibt es die Methode *AddScoped* des Service Providers, die wir in der Datei *Program.cs* verwenden.

→ Eventmanager.Api/Program.cs

```
public class Program
{
    private static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
        // ...
        // Configure Datebase with settings from
        // appsettings.json (section ConnectionStrings)
        builder.Services.AddDbContext<EventContext>(options =>
            options.UseSqlite(builder.Configuration.GetConnectionString("Default")));

        builder.Services.AddScoped<IEventService, EventService>(
            (provider) => new EventService(
                db: provider.GetRequiredService<EventContext>(),
                isDevelopment: builder.Environment.IsDevelopment()));
        // ...
    }
```

```
}
```

AddScoped() hat mehrere Überladungen. Die hier verwendete hat 2 Typparameter: Das Interface, das wir im Controller anfordern (*IEventService*) und das Service selbst. Wenn wir mit *builder.Services.AddScoped<IEventService, EventService>();* die Registrierung durchführen möchten, entsteht folgende Exception:

```
System.AggregateException: 'Some services are not able to be constructed
(Error while validating the service descriptor 'ServiceType:
Eventmanager.Services.IEventService Lifetime:
Scoped ImplementationType: Eventmanager.Services.EventService':
Unable to resolve service for type 'System.Boolean' while attempting to activate
'Eventmanager.Services.EventService'.)'
```

ASP.NET Core kann alle Parameter des Services auflösen, die registriert wurden. Würde unser Service nur den *EventContext* verwenden, wäre dies möglich. User Service hat aber auch den Parameter *isDevelopment* vom Typ *bool*. Die Fehlermeldung sagt aus, dass ASP nicht weiß, wie der Wert für *isDevelopment* abzurufen ist. Daher müssen wir ASP helfen und selbst den Konstruktor des Services angeben.

Mit *provider.GetRequiredService<EventContext>()* rufen wir eine Instanz des *EventContext* ab und übergeben ihn als ersten Parameter. Mit *builder.Environment.IsDevelopment()* befüllen wir den Parameter *isDevelopment*.

Die Klasse ContingentsController

Der Controller unterscheidet sich vom Aufbau her nicht vom *EventsController* des letzten Kapitels. Wir haben weiterhin *Route*, *ApiController* und *Http* Attributes für das Routing. Über Dependency Injection wird nun eine Implementierung von *IEventService* abgerufen. ASP wird - wie vorher definiert - das *EventService* mit den richtigen Parametern instanzieren und übergeben.

Wir haben keine Dependency zur Datenbank durch den *EventContext*, da dadurch Prüflogik im Service umgangen werden könnte und das clean code Schema durchbrochen wird.

→ Eventmanager.Api/Controllers/ContingentsController.cs

```
using Eventmanager.Application.Commands;
using Eventmanager.Application.Dtos;
using Eventmanager.Services;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Linq;
using System.Threading.Tasks;

namespace Eventmanager.Api.Controllers;
```

```
[Route("api/[controller]")]
[ApiController]
public class ContingentsController(IEventService eventService) : ControllerBase
{
    [HttpGet("{id}")]
    [ProducesResponseType<ContingentDto>(StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<ActionResult<ContingentDto>> GetContingentById(int id)
    {
        var contingent = await eventService.Contingents
            .Where(c => c.Id == id)
            .Select(c => new ContingentDto(
                c.Id, c.Show.Id, c.Show.Date,
                c.Show.Event.Name, c.ContingentType.ToString(), c.AvailableTickets))
            .FirstOrDefaultAsync();
        if (contingent is null)
            return Problem("Contingent not found.", statusCode: 404);
        return Ok(contingent);
    }

    [HttpPost]
    [ProducesResponseType(StatusCodes.Status201Created)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    public async Task<IActionResult> CreateContingent(
        [FromBody] CreateContingentCmd cmd)
    {
        try
        {
            var contingent = await eventService.CreateContingent(cmd);
            // Return 201 created and set the location header to the uri of the created
resource (/api/contingents/{id})
            return CreatedAtAction(
                nameof(GetContingentById),      // Name of GET Method to retrieve the
resource.
                new { id = contingent.Id },     // Routing parameter (id)
                new { id = contingent.Id });    // Created resource.
        }
        catch (EventServiceException e)
        {
            return Problem(e.Message, statusCode: 400);
        }
    }
}
```

Die Neuerung ist die Implementierung der *POST Route* mit dem Attribute *HttpPost*. Der Controller wird daher auf POST Requests, die an die Controlleradresse gehen (*POST /api/contingents*) reagieren. Als Parameter der Methode *CreateContingent* wird das command object *CreateContingentCmd* übergeben. ASP.NET Core erzeugt es automatisch aus den Daten des *request bodies*. Dieser Mechanismus heißt *model binding*. Eine genaue Dokumentation ist auf https://learn.microsoft.com/en-us/aspnet/core/mvc/models/model-binding?view=aspnetcore-10.0 abrufbar.

Im Gegensatz zur GET Route liefert der Controller im Erfolgsfall den Code *201 created*. Zusätz-

lich werden noch 2 Informationen übermittelt:

- Ein **location header** mit der URL der neu erstellten Ressource (z. B. http://localhost:5080/api/Contingents/74)
- Eine Antwort mit der ID der neuen Ressource als JSON. Da die ID in der Datenbank generiert wurde, schicken wir dem Client diese Information zurück.

Die Methode *CreatedAtAction()* erzeugt für uns diese Information. Dafür müssen wir als ersten Parameter den Namen der Methode angeben, die die Details der neu angelegten Ressource abrufen kann (also *GetContingentById*). Der zweite Parameter gibt den Parameter für *GetContingentById* an. Diese Methode erwartet sich einen Parameter mit dem Namen *id*. Dies ist für den korrekten Aufbau der Adresse im location header wichtig. Der dritte Parameter ist ein beliebiges Objekt, das an den Client zurückgesendet wird.

Test mit dem HTTP File

→ Eventmanager.Api/Eventmanager.Api.http

```
@baseUrl = http://localhost:5080

# Request should work (show 1 has no contingent for floor)
POST {{baseUrl}}/api/contingents
Content-Type: application/json
{
    "showId": "1",
    "contingentType": "Floor",
    "availableTickets": 100
}

###

# Request should not work (wrong enum value)
POST {{baseUrl}}/api/contingents
Content-Type: application/json
{
    "showId": "1",
    "contingentType": "XXX",
    "availableTickets": 100
}

###

# Request should not work (show 1 already has a contingent for stands)
POST {{baseUrl}}/api/contingents
Content-Type: application/json
{
    "showId": "1",
    "contingentType": "Stands",
    "availableTickets": 100
}

###
```

Verhalten bei Fehlern

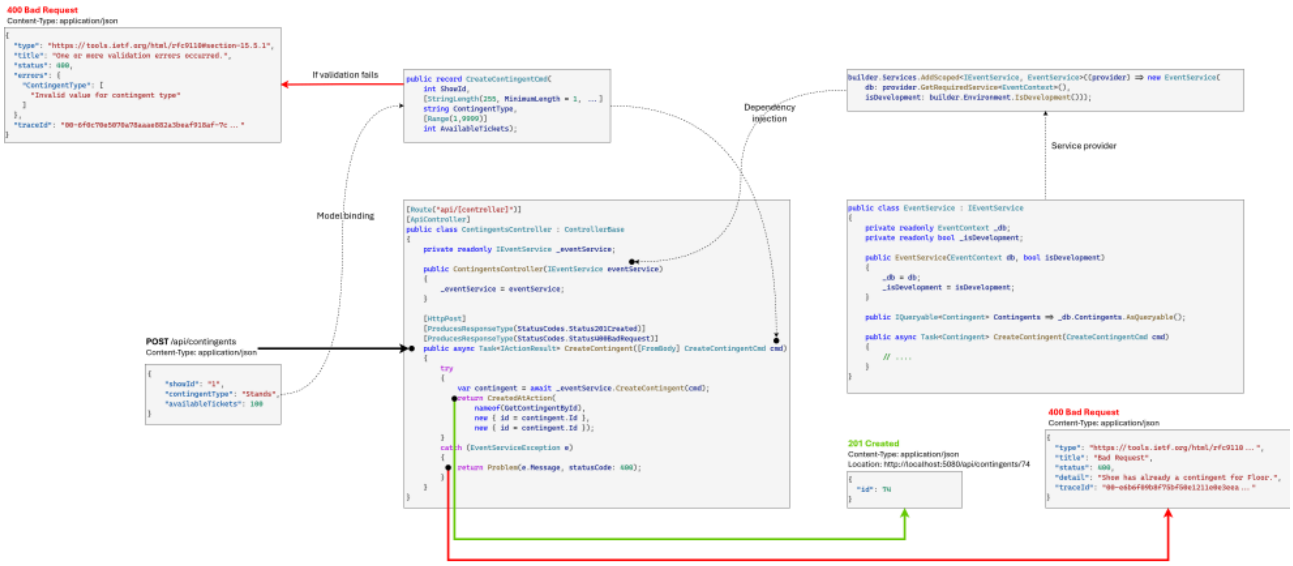
Wenn im request body falsche Daten, d. h. Daten die nicht den Validierungsregeln von *CreateContingentCmd* entsprechen, übermittelt werden, wird automatisch *400 bad request* mit einer Problembeschreibung gesendet:

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "ContingentType": [
      "Invalid value for contingent type"
    ]
  },
  "traceId": "00-4600e170ba6f4c2d767cc43afbda64c6-a60d243dcf47d7b5-00"
}
```

Wenn unser Service eine Exception wirft, dann senden wir mit der Methode *Problem()* ebenfalls eine solche Problembeschreibung. Sie beinhaltet dann die Fehlermeldung des Services:

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "Bad Request",
  "status": 400,
  "detail": "Show has already a contingent for Floor.",
  "traceId": "00-fe8eb95e7d3a8f42e07fb294f4f42565-681abcd1367f2728-00"
}
```

Das folgende Bild zeigt alle Teile, die bei der Bearbeitung des Requests beteiligt sind.



PUT, PATCH und DELETE

Der Time Provider

Wie bei der letzten Übung kommt auch hier der Time Provider zum Einsatz. Er wird in der Datei *Program.cs* als *singleton service* registriert.

```
builder.Services.AddSingleton<TimeProvider>((provider) => TimeProvider.System);
```

Im Service können wir mit *GetLocalNow()* bzw. *GetUtcNow()* die Systemzeit abrufen. Im Unittest können wir die Klasse *FakeTimeProvider* aus dem NuGet Paket *Microsoft.Extensions.TimeProvider.Testing* nutzen. Sie leitet sich von *TimeProvider* ab, und kann daher dem Service statt dem "echten" Time Provider übergeben werden. In diesem Beispiel wird die aktuelle Systemzeit auf den 28.2.2026 um 14:30 UTC (Offset = 0) gesetzt.

```
[Fact]
public void FakeTimeTest()
{
    // NuGet: Install Microsoft.Extensions.TimeProvider.Testing
    var timeProvider = new FakeTimeProvider(
        new DateTimeOffset(2026, 2, 28, 14, 30, 0, TimeSpan.Zero));
    var service = new EventService(db, timeProvider, isDevelopment: true);
}
```

Ressourcen ersetzen mit PUT

Technisch gesehen ist die Verarbeitung eines PUT Requests nicht schwer. Die benötigten Komponenten wie Command Object, Servicemethode und Endpunkt sind wie beim POST Request nötig und unterscheiden sich nur wenig.

Command Object und Servicemethode

Zuerst legen wir im Application Projekt einen record *UpdateContingentCmd* an. Natürlich muss er die Validierungsregeln, wie sie auch beim Anlegen eines Kontingentes gelten, beinhalten. Neu ist das Property *Id*. Da wir eine bestehende Ressource aktualisieren, brauchen wir eine eindeutige Identifikation. Das Property *Version* wird später besprochen. Es verhindert, dass Änderungen anderer User überschrieben werden.

→ Eventmanager.Application/Commands/UpdateContingentCmd.cs

```
using Eventmanager.Model;
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;

namespace Eventmanager.Application.Commands;
```

```
public record UpdateContingentCmd(
    [Range(1,int.MaxValue, ErrorMessage = "Invalid id.")]
    int Id,
    [Range(1,int.MaxValue, ErrorMessage = "Invalid show id.")]
    int ShowId,
    [StringLength(255, MinimumLength = 1, ErrorMessage = "Invalid contingent type.")]
    string ContingentType,
    [Range(1,9999)]
    int AvailableTickets,
    long Version) : IValidatableObject
{
    public IEnumerable<ValidationResult> Validate(ValidationContext validationContext)
    {
        if (!Enum.IsDefined(typeof(ContingentType), ContingentType))
        {
            yield return new ValidationResult(
                "Invalid value for contingent type",
                new string[] { nameof(ContingentType) });
        }
    }
}
```

Das Service wird durch eine Methode *UpdateContingent* ergänzt.

→ Eventmanager.Application/Services/EventService.cs

```
public class EventService(EventContext db, bool isDevelopment, TimeProvider
timeProvider) : IEventService
{
    // ....

    public async Task UpdateContingent(UpdateContingentCmd cmd)
    {
        if (!Enum.TryParse<ContingentType>(cmd.ContingentType, ignoreCase: true, out
var contingentType))
            throw new EventServiceException($"{cmd.ContingentType} is not a valid
contingent type.");

        var contingent = await db.Contingents
            .FirstOrDefaultAsync(c => c.Id == cmd.Id)
            ?? throw new EventServiceNotFoundException($"Contingent {cmd.Id} not
found.");

        if (contingent.Version != cmd.Version)
            throw new EventServiceException($"The contingent has already changed.");

        var show = await db.Shows
            .FirstOrDefaultAsync(s => s.Id == cmd.ShowId)
            ?? throw new EventServiceException($"Show {cmd.ShowId} not found.");

        contingent.Show = show;
        contingent.ContingentType = contingentType;
        contingent.AvailableTickets = cmd.AvailableTickets;
```

```
        contingent.Version = timeProvider.GetTimestamp();
        // UPDATE "Contingents" SET "AvailableTickets" = 80, "ContingentType" = Floor,
"Version" = 639080503086892609
        // WHERE "Id" = 1 AND "Version" = 0
        // RETURNING 1;
        await SaveChanges();
    }

    private async Task SaveChanges()
    {
        try
        {
            await db.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException e)
        {
            var message = isDevelopment ? e.InnerException?.Message ?? e.Message : "The
record has already changed.";
            throw new EventServiceException(message);
        }
        catch (DbUpdateException e)
        {
            var message = isDevelopment ? e.InnerException?.Message ?? e.Message :
"Cannot write changes to database.";
            throw new EventServiceException(message);
        }
    }
}
```

Die EventServiceNotFoundException

Wenn die ID nicht in der Datenbank gefunden wurde, soll unser Controller den HTTP Status 404 not found liefern. Bei anderen Fehlern soll HTTP 400 bad request zurückgegeben werden. Damit der Controller die Exceptions unterscheiden kann, legen wir einfach eine Klasse *EventServiceNotFoundException* an, die von *EventServiceException* erbt.

→ Eventmanager.Application/Services/EventServiceNotFoundException.cs

```
[Serializable]
public class EventServiceNotFoundException : EventServiceException
{
    public EventServiceNotFoundException() { }
    public EventServiceNotFoundException(string? message) : base(message) { }
    public EventServiceNotFoundException(string? message, Exception? innerException)
    : base(message, innerException) { }
}
```

Somit kann die folgende Anweisung dem Controller signalisieren, dass er HTTP 404 not found zurückgeben soll:

```
var contingent = await _db.Contingents
```

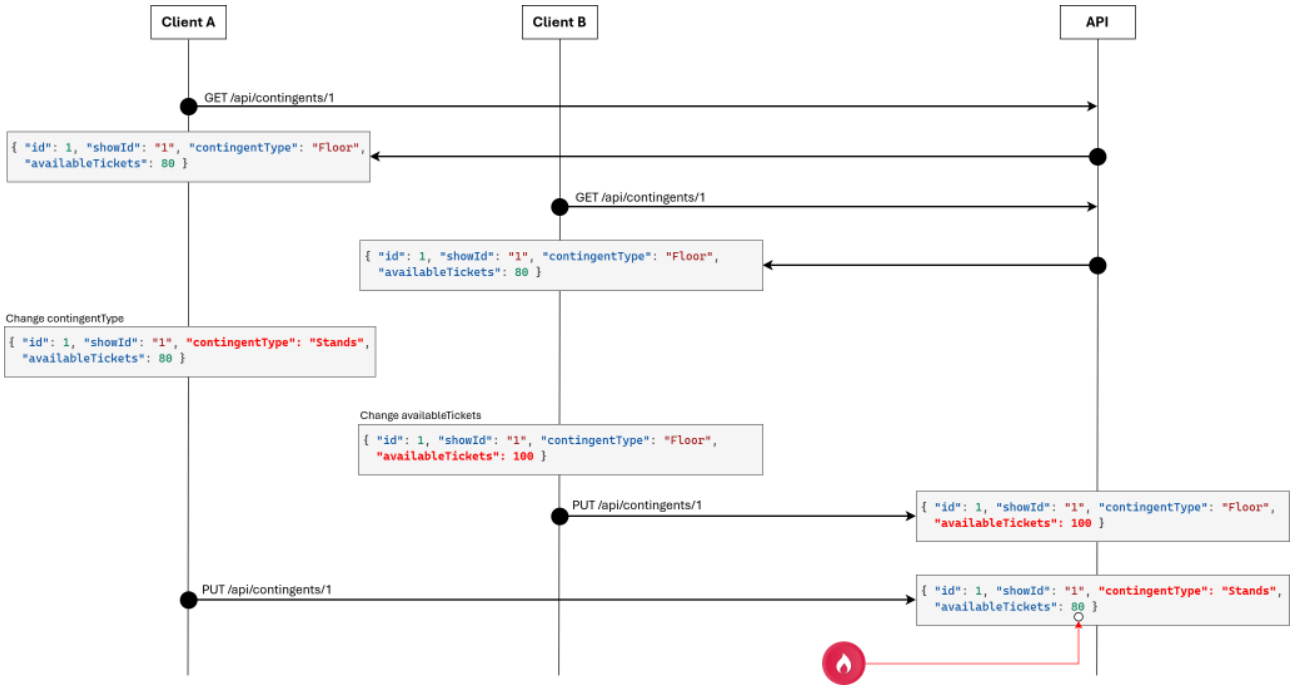
```
    .FirstOrDefaultAsync(c => c.Id == cmd.Id)
    ?? throw new EventServiceNotFoundException($"Contingent {cmd.Id} not found.");
```

Concurrency

In der Methode *UpdateContingent* befindet sich eine if Abfrage:

```
if (contingent.Version != cmd.Version)
    throw new EventServiceException($"The contingent has already changed.");
```

Um sie zu verstehen, müssen wir folgenden Fall analysieren: Was passiert, wenn Client A mit einem GET Request das Kontingent lädt und einige Zeit nachher den PUT Request absetzt. Nach dem Laden von Client A lädt Client B ebenfalls mit einem GET Request das Kontingent. Da Client A noch nicht geschrieben hat, bekommt Client B noch die alte Version. Nun führen beide Änderungen durch.

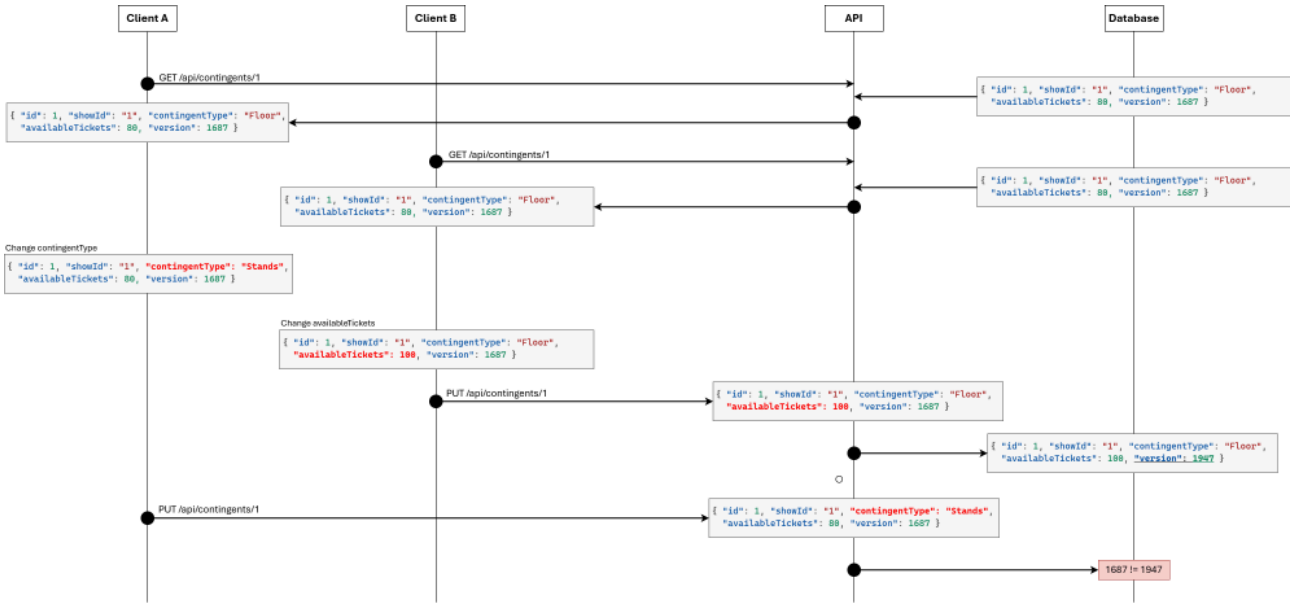


Dieses Verhalten bezeichnet man als *last one wins*. Nun kommt die Version ins Spiel. Wir ergänzen in der Modelklasse *Contingent* ein *long* Property mit dem Namen *Version*.

```
public class Contingent
{
    // EF Core constructor, public constructor
    public int Id { get; set; }
    public Show Show { get; set; }
    public ContingentType ContingentType { get; set; }
    public int AvailableTickets { get; set; }
    [ConcurrencyCheck]
    public long Version { get; set; }
    [JsonIgnore]
    public List<Ticket> Tickets { get; set; } = new();
```

```
}
```

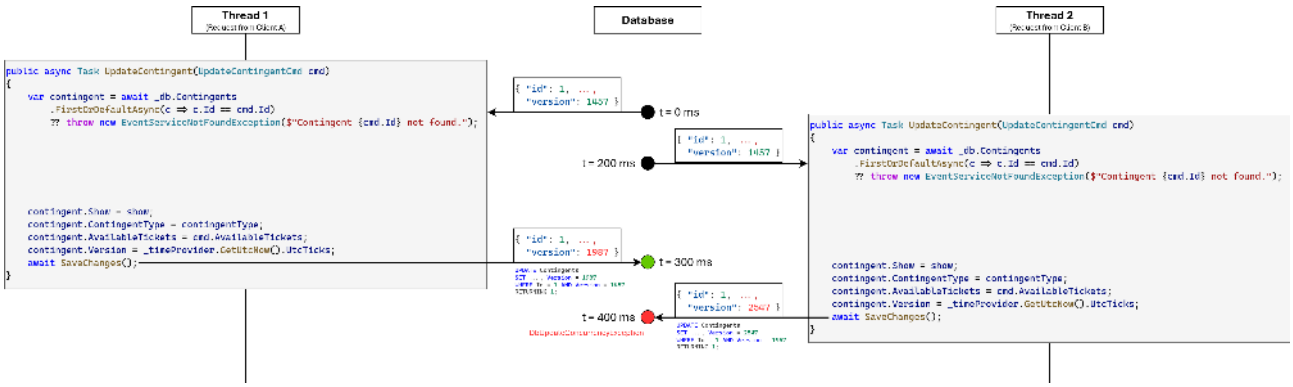
Wie wir die Version nutzen können, beschreibt die folgende Grafik:



Die if Abfrage verhindert also, dass eine geänderte, alte Version eine neuere Version in der Datenbank überschreibt.

Optimistic locking

Die Prüfung im Servicelayer ist allerdings noch nicht ausreichend. Es könnte auch sein, dass zwischen dem Lesen des Datensatzes in der Methode *UpdateContingent* und dem Absetzen des *UPDATE* Statements ein anderer Thread die Datenbank in der Zwischenzeit geändert hat. Da die Datenbank den Datensatz zwischen dem Lesen und dem Aktualisieren nicht sperrt, bezeichnet man dieses Verhalten als *optimistic locking*. Die folgende Grafik zeigt das Problem, das dabei auftreten kann:



Das Attribut *ConcurrencyCheck* in EF Core verfolgt eine einfache, aber wirksame Idee: Beim *UPDATE* Statement wird in der *WHERE* Klausel nicht nur die ID, sondern auch die Version angegeben. Hat der Datensatz in der Zwischenheit eine andere Version, wird dieser nicht gefunden. EF Core liefert dann eine *DbUpdateConcurrencyException*. In unserer verbesserten Methode *SaveChanges* im Service können wir darauf reagieren:

```
private async Task SaveChanges()
{
    try
    {
        await db.SaveChangesAsync();
    }
    catch (DbUpdateConcurrencyException e)
    {
        var message = _isDevelopment ? e.InnerException?.Message ?? e.Message : "The
record has already changed.";
        throw new EventServiceException(message);
    }
    catch (DbUpdateException e)
    {
        var message = _isDevelopment ? e.InnerException?.Message ?? e.Message : "Cannot
write changes to database.";
        throw new EventServiceException(message);
    }
}
```

Der PUT Endpunkt im Controller

Nach diesen vielen Vorüberlegungen ist der Controller der einfachste Teil der Implementierung. Er ruft einfach die Servicemethode auf und liefert je nach Fehler die entsprechende Antwort. Im Erfolgsfall wird *204 no content* geliefert, da der Client schon alle Informationen über die Ressource besitzt. Deswegen muss der Server keine Daten zurückliefern.

→ Eventmanager.Api/Controllers/ContingentsController.cs

```
[Route("api/[controller]")]
[ApiController]
public class ContingentsController(IEventService eventService) : ControllerBase
{
    /* ... */

    [HttpPut("{id}")]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<IActionResult> UpdateContingent([FromRoute] int id, [FromBody]
UpdateContingentCmd cmd)
    {
        if (id != cmd.Id) return Problem("Invalid id.", statusCode: 400);
        try
        {
            await eventService.UpdateContingent(cmd);
            return NoContent();
        }
        catch (EventServiceNotFoundException e)
        {
            return Problem(e.Message, statusCode: 404);
        }
```

```
        catch (EventServiceException e)
        {
            return Problem(e.Message, statusCode: 400);
        }
    }
}
```

Teilweises Aktualisieren mit PATCH

Wir haben gesehen, dass das Aktualisieren der gesamten Resource durch die mögliche Parallelität Probleme bereiten kann. Kommt in der Business Logik eine Aktualisierung eines einzigen Feldes häufiger vor, lohnt es sich, eine *PATCH* Route zu implementieren.

Wir möchten einen Endpunkt *PATCH /api/contingents/availableTickets* anbieten. Er soll den Wert in *AvailableTickets* aktualisieren.

Wie vorher erstellen wir zuerst ein command object, das die ID der Resource und den neuen Wert beinhaltet:

→ Eventmanager.Application/Commands/UpdateContingentAvailableTicketsCmd.cs

```
public record UpdateContingentAvailableTicketsCmd(
    [Range(1,9999,ErrorMessage = "Invalid id.")]
    int Id,
    [Range(1,9999)]
    int AvailableTickets);
```

Nun folgt die Service Methode im *EventService*. Sie beinhaltet auch Randbedingungen. So darf der neue Wert nicht unter den schon verkauften Tickets liegen. Natürlich bekommt der Datensatz auch eine neue Version. Da wir das Verhalten "last one wins" akzeptieren, prüfen wir allerdings nicht im Code die Version.

→ Eventmanager.Application/Services/EventService.cs

```
public async Task UpdateAvailableTickets(UpdateContingentAvailableTicketsCmd cmd)
{
    var contingent = await db.Contingents
        .Include(c => c.Tickets)
        .FirstOrDefaultAsync(c => c.Id == cmd.Id)
        ?? throw new EventServiceNotFoundException($"Contingent {cmd.Id} not found.");

    if (cmd.AvailableTickets < contingent.Tickets.Sum(t => t.Pax + 1))
        throw new EventServiceException($"The number of tickets sold or reserved
exceeds the number of tickets currently available.");

    contingent.AvailableTickets = cmd.AvailableTickets;
    contingent.Version = _timeProvider.GetTimestamp();
    await SaveChanges();
}
```

Im *ContingentsController* ist diese Methode mit dem entsprechenden *HttpPatch* Attribut implementiert:

→ Eventmanager.Api/Controllers/ContingentsController.cs

```
[Route("api/[controller]")]
[ApiController]
public class ContingentsController(IEventService eventService) : ControllerBase
{
    /* ... */
    [HttpPatch("{id}/availableTickets")]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<IActionResult> UpdateAvailableTickets([FromRoute] int id,
[FromBody] UpdateContingentAvailableTicketsCmd cmd)
    {
        if (id != cmd.Id) return Problem("Invalid id.", statusCode: 400);
        try
        {
            await eventService.UpdateAvailableTickets(cmd);
            return NoContent();
        }
        catch (EventServiceNotFoundException e)
        {
            return Problem(e.Message, statusCode: 404);
        }
        catch (EventServiceException e)
        {
            return Problem(e.Message, statusCode: 400);
        }
    }
}
```

Löschen mit DELETE

Beim Löschen müssen wir das Verhalten der relationalen Datenbank berücksichtigen. In der Tabelle *Tickets* wird auf die Contingent ID verwiesen. Falls nun für das Kontingent schon Tickets gekauft oder reserviert wurden, funktioniert das Löschen nur, wenn wir alle Verkäufe löschen. Unsere Business Logik sieht in diesem Fall einen Abbruch vor.

Ansonsten ist die Servicemethode sehr einfach aufgebaut, da kein command object benötigt wird. Wir brauchen nur die ID als eindeutige Identifizierung.

→ Eventmanager.Application/Services/EventService.cs

```
public async Task DeleteContingent(int id)
{
    var contingent = await db.Contingents
        .Include(c => c.Tickets)
        .FirstOrDefaultAsync(c => c.Id == id)
        ?? throw new EventServiceNotFoundException($"Contingent {id} not found.");
```

```
    if (contingent.Tickets.Any())
        throw new EventServiceException("The contingent has already sold or reserved
all the tickets.");

    db.Contingents.Remove(contingent);
    await SaveChanges();
}
```

Der Controller ruft wie gewohnt das Service auf und gibt *204 no content* im Erfolgsfall zurück.

```
[Route("api/[controller]")]
[ApiController]
public class ContingentsController(IEventService eventService) : ControllerBase
{
    /* ... */
    [HttpDelete("{id}")]
    [ProducesResponseType(StatusCodes.Status204NoContent)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<IActionResult> DeleteContingents([FromRoute] int id)
    {
        try
        {
            await eventService.DeleteContingent(id);
            return NoContent();
        }
        catch (EventServiceNotFoundException e)
        {
            return Problem(e.Message, statusCode: 404);
        }
        catch (EventServiceException e)
        {
            return Problem(e.Message, statusCode: 400);
        }
    }
}
```

Unsere Controller können wir mit dem folgenden HTTP File überprüfen:

```
# Request should not work: Contingent 999 not found.
PUT {{baseUrl}}/api/contingents/999
Content-Type: application/json
{
    "id": 999,
    "showId": "1",
    "contingentType": "Floor",
    "availableTickets": 80,
    "version": 0
}

###
```

```
# Request should work one time: 204 no content.
# After the first request: The contingent has already changed.
PUT {{baseUrl}}/api/contingents/1
Content-Type: application/json
{
    "id": 1,
    "showId": "1",
    "contingentType": "Floor",
    "availableTickets": 80,
    "version": 0
}

###

# Request should not work: The number of tickets sold or reserved exceeds ...
PATCH {{baseUrl}}/api/contingents/25/availableTickets
Content-Type: application/json
{
    "id": 25,
    "availableTickets": 35
}

###

# Request should work
PATCH {{baseUrl}}/api/contingents/25/availableTickets
Content-Type: application/json
{
    "id": 25,
    "availableTickets": 80
}

###

# Request should not work: The contingent has already sold ...
DELETE {{baseUrl}}/api/contingents/25

####

# Request should work
DELETE {{baseUrl}}/api/contingents/44

####
```

Servicetests mit der SQLite in-memory DB

Analyse der Dependencies

Unser *EventService* hat unter anderem eine Dependency zum Datenbankcontext *EventContext*. Möchten wir das Service testen, müssen wir es natürlich im Unittest einmal instanzieren. Dafür müssen wir allerdings alle Argumente des Konstruktors übergeben.

```
public class EventService(
    EventContext db, TimeProvider timeProvider, bool isDevelopment) : IEventService
{
}
```

Wir erkennen 3 Dependencies: Den Datenbankcontext, den Timeprovider und einen bool Wert, der angibt, ob der Server im Development Mode läuft. Die schlechteste Möglichkeit wäre ein Durchführen des Testes mit der originalen Konfiguration des Datenbankcontextes. Dadurch würde die Produktionsdatenbank geändert werden.

Führe nie Unittests gegen die Produktionsdatenbank durch. Die Tests wären nicht deterministisch und hätten fatale Folgen für den Datenbestand.

Der Time Provider und der FakeTimeProvider

In den Übungen wurde im *FitnessService* über dependency injection die Klasse *TimeProvider* aus dem .NET Framework angefordert. Das dazugehörige Service wurde in der Datei *Program.cs* registriert:

```
builder.Services.AddSingleton<TimeProvider>((provider) => TimeProvider.System);
```

Der Grund ist die Vermeidung von *DateTime.Now* bzw. *DateTime.UtcNow* im Service. Wenn wir z. B. bei einer Reservierung testen möchten, ob eine Show mindestens 14 Tage in der Zukunft liegt, würde dies nach klassischem Muster so aussehen:

```
public async Task<int> CreateReservation(int guestId, int contingentId, int pax,
DateTime dateTime)
{
    var contingent = await _db.Contingents
        .Include(c => c.Show).Include(c => c.Tickets)
        .FirstOrDefaultAsync(c => c.Id == contingentId)
        ?? throw new EventServiceException("Invalid contingent id.");
    if (contingent.Show.Date < DateTime.Now.AddDays(14))
        throw new EventServiceException("The show is too close in time.");
    /* ... */
}
```

Probleme ergeben sich beim Testen: Ein Unittest, der die Methode *CreateReservation* prüft und erfolgreich durchläuft, wird vielleicht in 1 Monat fehlschlagen. Lösungen wie das Verwenden von *DateTime.Now* im Unittest sind problematisch, da der Test dadurch nicht *deterministisch* aufgebaut ist.

Daher gibt es seit .NET 8 die abstrakte Klasse *TimeProvider* im Namespace *System*. Diese Klasse kann unser Service im Konstruktor anfordern. Im Service selbst sieht die Methode *CreateReservation* nun so aus:

```
public async Task<int> CreateReservation(int guestId, int contingentId, int pax,
DateTime dateTime)
{
    var contingent = await _db.Contingents
        .Include(c => c.Show).Include(c => c.Tickets)
        .FirstOrDefaultAsync(c => c.Id == contingentId)
        ?? throw new EventServiceException("Invalid contingent id.");
    if (contingent.Show.Date < timeProvider.GetLocalNow().DateTime.AddDays(14))    ①
        throw new EventServiceException("The show is too close in time.");
    /* ... */
}
```

① Die Methode *GetLocalNow()* liefert ein *DateTimeOffset* mit der aktuellen Systemzeit. Das Property *DateTime* ruft den Datums- und Zeitwert ohne Zeitzone ab.

Im Paket *Microsoft.Extensions.TimeProvider.Testing*, das über NuGet installiert werden muss, steht die Klasse *FakeTimeProvider* bereit. Wenn wir nun das Service testen möchten, können wir mit folgendem Code die Zeit fix auf den 28.2.2026 um 16:30 UTC (0h Offet) festlegen:

```
var timeProvider = new FakeTimeProvider(new DateTimeOffset(2026, 2, 28, 16, 30, 0,
TimeSpan.Zero));
var service = new EventService(..., timeProvider);
```

Aufbau der Testklasse

Zuerst erstellen wir im Testprojekt eine neue Klasse mit dem Namen *EventServiceWithInMemoryDbTests*. Nun definieren wir einen Konstruktor und implementieren das Interface *IDisposable*.

```
public class EventServiceWithInMemoryDbTests : IDisposable    ①
{
    private readonly SqliteConnection _conn;
    private readonly EventContext _db;
    private readonly FakeTimeProvider _timeProvider;
    private readonly EventService _service;

    public EventServiceWithInMemoryDbTests()
    {
        _conn = new SqliteConnection("DataSource=:memory:");    ②
```

```
        _db = new EventContext(
            new DbContextOptionsBuilder().UseSqlite(_conn).Options);    ③
        _db.Database.EnsureDeleted();    ④
        _db.Database.OpenConnection();
        _db.Database.EnsureCreated();

        _timeProvider = new FakeTimeProvider(
            new DateTimeOffset(new DateTime(2026, 3, 9, 16, 0, 0)));       ⑤

        _service = new EventService(
            db: _db, timeProvider: _timeProvider, isDevelopment: true);    ⑥
    }

    public void Dispose()    ⑦
    {
        _db.Dispose();
        _conn.Dispose();
    }
}
```

① Da wir die Datenbankverbindung selbst verwalten, müssen wir am Ende die Ressource selbst schließen. Dafür stellt das .NET Framework das Interface *IDisposable* zur Verfügung, das die Methode *Dispose()* beinhaltet.

② Wir erzeugen eine SQLite Connection mit dem Verbindungsstring *DataSource=:memory:*. Dies erzeugt eine in-memory Datenbank.

③ Der *EventContext* besitzt einen Konstruktor, der ein Argument vom Typ *DbContextOptions* besitzt. Über dieses Argument kann eine andere Zieldatenbank angegeben werden.

④ Nachdem die Datenbank zur Sicherheit gelöscht wurde, öffnen wir die Verbindung. Mit *EnsureCreated()* wird das Schema mittels *CREATE TABLE* Anweisungen erzeugt.

⑤ Im Paket Microsoft.Extensions.TimeProvider.Testing befindet sich die Klasse *FakeTimeProvider*. Sie erzeugt einen *TimeProvider*, der als aktuelle Zeit die angegebene Zeit zurückgibt. Dadurch können wir alle Aufrufe des Services, die die Zeit abrufen, von außen steuern.

⑥ Nun wird das *EventService* mit allen Dependencies instanziert.

⑦ Die *Dispose()* Methode schließt alle Ressourcen.

Da für jeden Unittest die Testklasse neu instanziert wird, können wir die beschriebenen Tätigkeiten im Konstruktor zentral erledigen.

Erstellen der Musterdaten

Da das Service mit einer leeren Datenbank nur schlecht arbeiten kann, fügen wir Musterdaten hinzu. Achte darauf, dass die Daten *minimal* sind. Sonst gibt es Seiteneffekte mit vorhandenen Daten, wenn z. B. ein neues Kontingent erstellt wird.

```
private Tentity[] AddEntities<Tentity>(params Tentity[] items) where Tentity : class
{
    _db.Set<Tentity>().AddRange(items);
    _db.SaveChanges();
    return items;
```

```
}

private void GenerateFixtures()
{
    var events = AddEntities(new Event("Event1") { Id = 1 });
    var shows = AddEntities(
        new Show(events[0], new DateTime(2026, 3, 9, 14, 0, 0)) { Id = 1 },
        new Show(events[0], new DateTime(2026, 3, 9, 16, 0, 0)) { Id = 2 });
    var contingents = AddEntities(
        new Contingent(shows[0], ContingentType.Rang, 10) { Id = 1 });
}
```

`Tentity[] AddEntities<Tentity>(params Tentity[] items) where Tentity : class`

Diese Methode nutzt das sogenannte *params Array* in C#. Ähnlich dem Rest Operator in JavaScript können alle angegebenen Argumente einer Methode als Array abgerufen werden. Dadurch können beliebig viele Argumente verarbeitet werden. Diese Daten werden in das entsprechende DbSet geschrieben, gespeichert und als Array zurückgegeben. *Tentity* ist der Typparameter. Über *type inference* erkennt C# durch den Aufruf, dass *Tentity* bei *AddEntities(new Event(...))* vom Typ *Event* sein muss. Es funktioniert wie Bearbeiten → Ersetzen in einem Editor. Da wir nur Klassen und nicht structs wie int, double, ... als Typparameter akzeptieren wollen, schränken wir mit *where Tentity : class* ein.

`void GenerateFixtures()`

Wenn wir in einem Unittest nun Testdaten haben möchten, können wir einfach diese Methode aufrufen. Somit vermeiden wir wiederholenden Code in den einzelnen Tests.

Schreiben des ersten Tests

Sind wir einmal soweit gekommen, ist der eigentliche Test sehr einfach. Wir erzeugen nach den Musterdaten ein neues *CreateContingentCmd* Objekt und übergeben es der Servicemethode. Im *Assert* prüfen wir mit der *Any* Funktion, ob der besagte Datensatz in der Datenbank gefunden wird. Da wir wissen, dass es keine anderen Datensätze mit dem Typ *Floor* gibt, würde auch ein einfacheres Any ausreichen. Wir können dies nutzen, weil wir stets einen *definierten* Zustand der Datenbank haben.

Unittests müssen immer unabhängig voneinander ablaufen. Es darf nicht sein, dass Test A die Datenbank ändert und Test B arbeitet mit der geänderten Datenbank weiter. Es darf nur *eine* Servicemethode getestet werden.

```
[Fact]
public async Task CreateContingentCreatesNewContingentTest()
{
    GenerateFixtures();
    await _service.CreateContingent(new CreateContingentCmd(1, "Floor", 10));
    _db.ChangeTracker.Clear();
    Assert.True(_db.Contingents.Any(c =>
        c.ContingentType == ContingentType.Floor && c.AvailableTickets == 10));
```

```
}
```

Parametrisierte Tests

In unserem Service gibt es mehrere Fehlerzustände. Um mit einer code coverage von 100% zu testen, müssen wir jeden fehlerhaften Fall erzeugen.

Konkret haben wir 3 Fälle, wo aufgrund fehlerhafter Daten im command object eine *EventServiceException* geworfen wird.

```
if (!Enum.TryParse<ContingentType>(
    cmd.ContingentType, ignoreCase: true, out var contingentType))
    throw new EventServiceException(
        $"{cmd.ContingentType} is not a valid contingent type.");

var show = await db.Shows.FirstOrDefaultAsync(s => s.Id == cmd.ShowId)
    ?? throw new EventServiceException($"Show {cmd.ShowId} not found.");

if (await db.Contingents.AnyAsync(c =>
    c.Show.Id == cmd.ShowId && c.ContingentType == contingentType))
    throw new EventServiceException(
        $"Show has already a contingent for {cmd.ContingentType}.");
```

Wir können nun 3 Einzeltests schreiben, oder *parametrisierte Tests* verwenden. Sie werden mit *Theory* definiert. In *InlineData* werden die Argumente angegeben, die dann vom Testframework übergeben werden. In diesem Beispiel werden 3 Unittests generiert und ausgeführt:

```
[Theory]
[InlineData(1, "x", "x is not a valid contingent type.")]
[InlineData(999, "Floor", "Show 999 not found.")]
[InlineData(1, "Rang", "Show has already a contingent for Rang")]
public async Task CreateContingentThrowsExceptionWithMessageTest(int showId, string contingentType, string expectedMessage)
{
    GenerateFixtures();
    var ex = await Assert.ThrowsAsync<EventServiceException>(() => _service.CreateContingent(
        new CreateContingentCmd(showId, contingentType, 10)));
    Assert.Contains(expectedMessage, ex.Message, StringComparison.OrdinalIgnoreCase);
}
```

```
public async Task CreateContingentThrowsExceptionWithMessageTest(1, "x", "x is not a valid contingent type.")
{ /* ... */ }
```

```
public async Task CreateContingentThrowsExceptionWithMessageTest(999, "Floor", "Show 999 not found.")
{ /* ... */ }
```

```
public async Task CreateContingentThrowsExceptionWithMessageTest(1, "Rang", "Show has already a contingent for Rang")
{ /* ... */ }
```

Es können nicht alle Datentypen in *InlineData* angegeben werden. Braucht man z. B. einen *DateTime* Wert als Parameter, muss man diesen als String z. B. mit "2026-03-07T16:57:00" definieren, als String im Argument der Testmethode übergeben und mit *DateTime.Parse()* im Test parsen.

Nun sieht unsere gesamte Testklasse so aus. Sie testet die Methoden *CreateContingent()* und

UpdateContingent() mit 100% code coverage.

→ *Eventmanager.Test/EventServiceWithInMemoryDbTests.cs*

```
using Eventmanager.Application.Commands;
using Eventmanager.Application.Services;
using Eventmanager.Infrastructure;
using Eventmanager.Model;
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Time.Testing;
using System;
using System.Linq;
using System.Threading.Tasks;

namespace Eventmanager.Test;

public class EventServiceWithInMemoryDbTests : IDisposable
{
    private readonly SqliteConnection _conn;
    private readonly EventContext _db;
    private readonly FakeTimeProvider _timeProvider;
    private readonly EventService _service;

    public EventServiceWithInMemoryDbTests()
    {
        _conn = new SqliteConnection("DataSource=:memory:");

        _db = new EventContext(
            new DbContextOptionsBuilder().UseSqlite(_conn).Options);
        _db.Database.EnsureDeleted();
        _db.Database.OpenConnection();
        _db.Database.EnsureCreated();

        _timeProvider = new FakeTimeProvider(
            new DateTimeOffset(new DateTime(2026, 3, 9, 16, 0, 0)));

        _service = new EventService(
            db: _db, timeProvider: _timeProvider, isDevelopment: true);
    }
    /// <summary>
    /// Hilfsmethode, die die übergebenen Parameter in die Datenbank
    /// schreibt und diese als Array zurückgibt.
    /// </summary>
    private Tentity[] AddEntities<Tentity>(params Tentity[] items)
        where Tentity : class
    {
        _db.Set<Tentity>().AddRange(items);
        _db.SaveChanges();
        return items;
    }
    /// <summary>
    /// Definiert zentrale Musterdaten, mit denen jeder Unittest arbeitet.
    /// </summary>
    private void GenerateFixtures()
```

```
    {
        var events = AddEntities(new Event("Event1") { Id = 1 });
        var shows = AddEntities(
            new Show(events[0], new DateTime(2026, 3, 9, 14, 0, 0)) { Id = 1 },
            new Show(events[0], new DateTime(2026, 3, 9, 16, 0, 0)) { Id = 2 });
        var contingents = AddEntities(
            new Contingent(shows[0], ContingentType.Rang, 10) { Id = 1 });
    }
    /// <summary>
    /// Testet den Erfolgsfall der Methode CreateContingent()
    /// </summary>
    [Fact]
    public async Task CreateContingentCreatesNewContingentTest()
    {
        GenerateFixtures();
        await _service.CreateContingent(new CreateContingentCmd(1, "Floor", 10));
        _db.ChangeTracker.Clear();
        Assert.True(_db.Contingents.Any(c =>
            c.ContingentType == ContingentType.Floor && c.AvailableTickets == 10));
    }
    /// <summary>
    /// Testet die geworfene Exception von CreateContingent
    /// basierend auf den Daten des Command Objects.
    /// </summary>
    [Theory]
    [InlineData(1, "x", "x is not a valid contingent type.")]
    [InlineData(999, "Floor", "Show 999 not found.")]
    [InlineData(1, "Rang", "Show has already a contingent for Rang")]
    public async Task CreateContingentThrowsExceptionWithMessageTest(
        int showId, string contingentType, string expectedMessage)
    {
        GenerateFixtures();
        var ex = await Assert.ThrowsAsync<EventServiceException>(
            () => _service.CreateContingent(
                new CreateContingentCmd(showId, contingentType, 10)));
        Assert.Contains(
            expectedMessage, ex.Message, StringComparison.OrdinalIgnoreCase);
    }
    /// <summary>
    /// Testet den Erfolgsfall der Methode UpdateContingent()
    /// </summary>
    [Fact]
    public async Task UpdateContingentUpdatesContingentTest()
    {
        GenerateFixtures();
        var expectedVersion = _timeProvider.GetTimestamp();
        await _service.UpdateContingent(
            new UpdateContingentCmd(1, 1, "Rang", 20, 0));
        _db.ChangeTracker.Clear();
        Assert.True(_db.Contingents.Any(c =>
            c.Id == 1 && c.Show.Id == 1 && c.ContingentType == ContingentType.Rang
            && c.AvailableTickets == 20 && c.Version == expectedVersion));
    }
    /// <summary>
    /// Testet die geworfene Exception von UpdateContingent
    /// basierend auf den Daten des Command Objects.
```

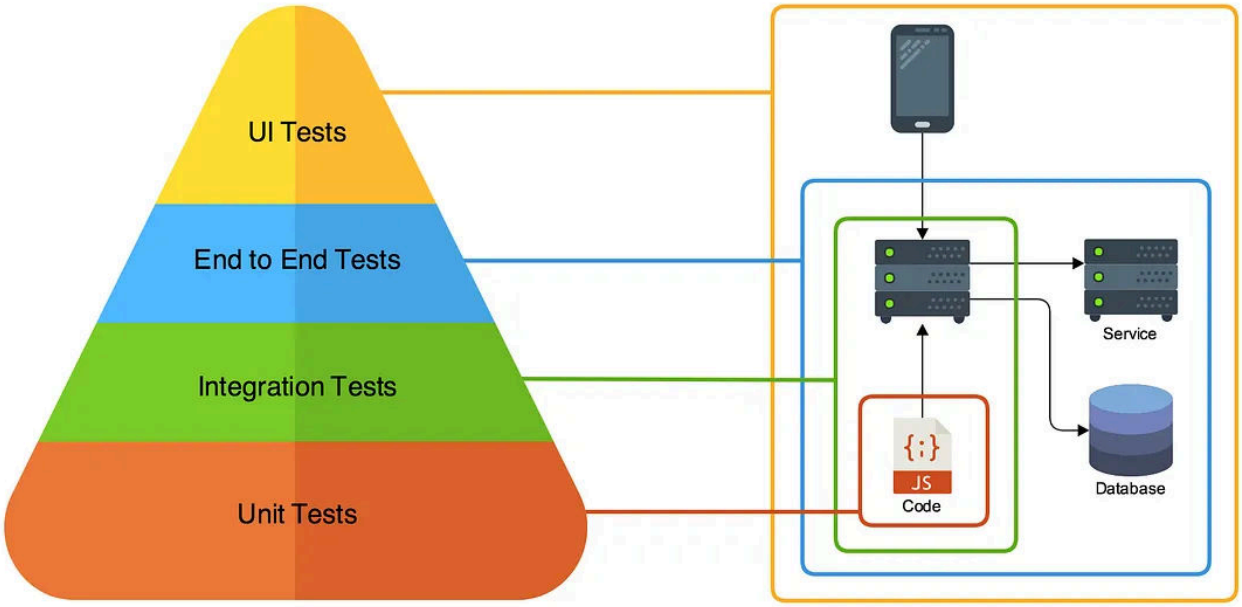
```
    /// </summary>
    [Theory]
    [InlineData(999, 1, 0, "Rang", "EventServiceNotFoundException",
                                "Contingent 999 not found.")]
    [InlineData(1, 1, 0, "x", "EventServiceException",
                            "x is not a valid contingent type.")]
    [InlineData(1, 1, 999, "Rang", "EventServiceException",
                                "The contingent has already changed.")]
    [InlineData(1, 999, 0, "Rang", "EventServiceException",
                                "Show 999 not found.")]
    public async Task UpdateContingentThrowsExceptionWithMessageTest(
        int contingentId, int showId, long version, string contingentType,
        string errorType, string expectedMessage)
    {
        GenerateFixtures();
        var ex = await Assert.ThrowsAnyAsync<EventServiceException>(
            () => _service.UpdateContingent(new UpdateContingentCmd(
                    contingentId, showId, contingentType, 20, version)));
        Assert.Equal(errorType, ex.GetType().Name);
        Assert.Contains(
            expectedMessage, ex.Message, StringComparison.OrdinalIgnoreCase);
    }
    public void Dispose()
    {
        _db.Dispose();
        _conn.Dispose();
    }
}
```

Integration Tests mit ASP.NET Core

Die Testpyramide

Bisher haben wir Unittests kennengelernt. Sie prüfen, ob eine Methode ein definiertes Verhalten zeigt. Der Test läuft auf Codeebene ab, d. h. eine Klasse wird instanziert, die Methode aufgerufen und das Ergebnis wird verglichen. Das Triple A Schema (Arrange - Act - Assert) ist hier zu erwähnen.

Diese Testart kann allerdings nicht alles prüfen. In ASP.NET Core haben wir z. B. mittels Attributes das Routingsystem definiert, und mit dem Serviceprovider werden abhängige Klassen zur Verfügung gestellt. Ein reines Instanzieren der Controllerklasse und überprüfen der Methode deckt keine Fehler in diesem Bereich auf. Wir brauchen also höherwertige Tests:



Quelle: https://medium.com/@nathankpeck/microservice-testing-introduction-347d2f74095e

Der **Ingetration Test** ist - technisch gesehen - ein Unittest, der folgenden Ablauf hat:

- Eine Datenbank wird für die Verwendung vorbereitet.
- Ein Testserver wird gestartet. Der Serviceprovider wird so angepasst, dass er auf die Testdatenbank verweist.
- Ein HTTP Request wird an den Testserver gesandt.
- Die Antwort des Servers wird ausgewertet.

Das Schema *Arrange - Act - Assert* ist auch hier erkennbar.

Integration Tests mit ASP.NET Core

Das Paket *Microsoft.AspNetCore.Mvc.Testing*

Das Durchführen eines Integration Tests erfordert mehr Vorbereitung als der klassische Unittest auf Methodenebene. Der Testserver soll automatisiert mit dem Unittest gestartet werden und einen zufälligen, freien Port belegen. Microsoft stellt mit dem Paket *Microsoft.AspNetCore.Mvc.Testing* die Klasse *WebApplicationFactory* bereit, die diese Aufgaben erledigt. Sie ist als *Basisklasse* für eine eigene Implementierung gedacht und besitzt die zentrale Methode *ConfigureWebHost*. Als *virtual* Methode kann sie überschrieben werden.

Folgende Implementierung einer eigenen Testklasse *TestWebApplicationFactory* zeigt den Einsatz.

```
public class TestWebApplicationFactory<Tcontext>(string environment = "Testing")
    : WebApplicationFactory<Program> where Tcontext : DbContext    ①
{
    /* ... */
    protected override void ConfigureWebHost(IWebHostBuilder builder)    ②
    {
        builder.ConfigureServices(services =>
        {
            foreach (var substituteService in _substituteServices)
            {
                var descriptor = services.First(d =>
                    d.ServiceType == substituteService.Key);
                services.Remove(descriptor);
                substituteService.Value(services);
            }
        });
        builder.UseEnvironment(environment);
    }

    public void SubstituteService<T>(    ③
        Action<DbContextOptionsBuilder>? builderAction) where T : DbContext
    { /* ... */ }
    public void SubstituteService<T>(    ④
        Func<IServiceProvider, T> factoryFuncion,
        ServiceLifetime lifetime = ServiceLifetime.Scoped) where T : class
    { /* ... */ }
}
```

① Die Klasse ist generisch und hat den Typparameter *Tcontext*. Später wird mit *new TestWebApplicationFactory<EventContext>()* ein konkreter Datenbankkontext als Typ übergeben. + Über den primary constructor wird eine Variable *environment* angelegt. Dadurch kann der Server gezielt im Environment *Testing* oder *Development* gestartet werden.

② In *ConfigureWebHost* wird nun der Serviceprovider abgeändert. Der Testserver startet zunächst mit den in der Datei *Program.cs* definierten Services. Danach suchen wir einzelne Services und tauschen sie aus bzw. entfernen diese. So können wir erreichen, dass der Datenbankcontext z. B. auf die in-memory Datenbank verweist.

③ Damit die Registrierung des DbContext einfach ausgetauscht werden kann, bietet die Methode eine einfache Möglichkeit an, die Konfiguration des Testcontextes anzugeben.

④ Damit andere Services einfach ausgetauscht werden kann, bietet die Methode eine einfache Möglichkeit an, z. B. einen eigenen Time Provider zu registrieren.

Requestmethoden in *TestWebApplicationFactory*

In → TestWebApplicationFactory.cs gibt es eine Implementierung, die auch bei Prüfungen zur Verfügung steht. Sie hat folgende Methoden, die wir danach in unseren Integration Tests nutzen:

`void InitializeDatabase(Action<Tcontext> action)`

Initialisiert die Datenbank mit einer vorgegebenen Methode. Da wir über den Serviceprovider auf die Datenbank zugreifen müssen, führt diese Methode folgende Schritte vor der Ausführung der übergebenen Funktion durch:

Laden des Datenbankservices über den Service Provider.
Löschen und neu Erstellen der Datenbank.

Konkret wird diese Methode z. B. so verwendet:

```
_factory.InitializeDatabase(db =>
{
    var @event = new Event("Event1");
    var show = new Show(
        @event, new DateTime(2026, 3, 7, 14, 0, 0, DateTimeKind.Utc));
    var contingent = new Contingent(show, ContingentType.Rang, 10);
    db.AddRange(show, contingent);
    db.SaveChanges();
});
```

`Tout QueryDatabase<Tout>(Func<Tcontext, Tout> query)`

Erlaubt das Abfragen der Datenbank durch den DbContext. Die Methode ruft das Service aus dem Service Provider ab und führt die übergebene Methode aus.

Konkret wird diese Methode z. B. so verwendet:

```
var contingentFromDb = _factory.QueryDatabase(
    db => db.Contingents.First(c => c.Id == 2));
```

`Task<(HttpResponseMessage, T?)> GetHttpContent<T>(string requestUrl)`

`Task<(HttpResponseMessage, JsonElement)> GetHttpContent(string requestUrl)`

Sendet einen GET Request an die angegebene Adresse. Die Adresse muss keine Domain beinhalten, da der Port dynamisch zugewiesen wird. Wenn ein typisiertes Ergebnis geliefert wird, kann die DTO Klasse bzw. eine Liste der DTO Klasse als Typparameter angegeben werden. Wird kein Typparameter angegeben, wird ein JSON Element, also die "rohe" JSON Antwort, geliefert. Die Methode liefert ein Tupel aus 2 Elementen zurück.

Konkret wird diese Methode z. B. so verwendet:

```
var (response, content) =
    await _factory.GetHttpContent<ContingentDto>("/api/contingents/1");
```

In *response* ist die *HttpResponseMessage* gespeichert. Sie beinhaltet den Status Code und andere Header Informationen. In *content* werden die eigentlichen Daten gespeichert. Da wir einen Typparameter (*ContingentDto*) angegeben haben, hat diese Variable den Typ *ContingentDto?*.

`Task<(HttpResponseMessage, JsonElement)> PostHttpContent<Tcmd>(string requestUrl, Tcmd payload)`

`Task<(HttpResponseMessage, JsonElement)> PutHttpContent<Tcmd>(string requestUrl, Tcmd payload)`

`Task<(HttpResponseMessage, JsonElement)> PatchHttpContent<Tcmd>(string requestUrl, Tcmd payload)`

Sendet einen POST, PUT oder PATCH Request. Der Parameter *requestUrl* ist wieder die Adresse. Der Parameter *paylad* ist die Instanz eines command objects.

Konkret wird diese Methode z. B. so verwendet:

```
var (response, content) = await _factory.PostHttpContent(
    "/api/contingents",
    new CreateContingentCmd(1, "Floor", 10));
```

`Task<(HttpResponseMessage, JsonElement)> DeleteHttpContent(string requestUrl)`

Sendet einen DELETE Request. Der Parameter *requestUrl* ist wieder die Adresse.

Konkret wird diese Methode z. B. so verwendet:

```
var (response, content) = await _factory.DeleteHttpContent("/api/contingents/1");
```

`bool ContentContainsErrorMessage(JsonElement content, string message)`

Prüft, ob eine problem detail Antwort eine definierte Fehlermeldung enthält. Die RFC problem detail kann 2 Formen haben.

- *{ "detail": "Error message" }* wenn mit *return Problem(...)* gearbeitet wurde.
- *{ "errors": "MyProperty": ["Validation message"] }* wenn Validierungsfehler auftreten.

  Es werden Teilstrings verglichen (nicht case sensitive), d. h. *not found* ist für alle Fehlermeldungen zugreffend, die *not found* in der Fehlermeldung in *detail* oder in *errors* haben.

Arbeiten mit JsonElement (System.Text.Json)

Oft liefern API-Aufrufe (besonders bei POST, PUT oder DELETE) dynamische oder kleine Datenstrukturen zurück, für die sich das Anlegen eines eigenen DTOs nicht lohnt (z. B. `{ "id": 12 }`). In diesen Fällen liefert die Test-Factory ein `JsonElement` zurück, mit dem die "rohe" JSON-Ant-

wort dynamisch ausgewertet werden kann.

Folgende Methoden sind für das Extrahieren von Daten in den `Assert`-Prüfungen wichtig:

`JsonElement GetProperty(string propertyName)`

Sucht auf einem JSON-Objekt nach einer Eigenschaft mit dem angegebenen Namen und gibt deren Wert als neues `JsonElement` zurück. Damit navigiert man im JSON-Baum.

```
var idElement = content.GetProperty("id");
```

`int GetInt32()`

Wandelt den aktuellen JSON-Zahlenwert in einen C#-`int` um. Nützlich zur Überprüfung von IDs oder Zählern.

```
Assert.Equal(12, content.GetProperty("id").GetInt32());
```

`string? GetString()`

Liest den aktuellen JSON-String-Wert aus und gibt ihn als C#-`string` zurück.

```
Assert.Equal("Floor", content.GetProperty("contingentType").GetString());
```

`JsonElement.ArrayEnumerator EnumerateArray()`

Wenn das `JsonElement` ein JSON-Array darstellt (`[ ··· ]`), liefert diese Methode einen Enumerator, um durch die einzelnen Array-Elemente zu iterieren. Jedes Element ist wiederum ein `JsonElement`.

```
foreach (JsonElement item in content.EnumerateArray())
{
    Assert.True(item.GetProperty("id").GetInt32() > 0);
}
```

`JsonElement.ObjectEnumerator EnumerateObject()`

Wenn das `JsonElement` ein JSON-Objekt darstellt (`{ ··· }`), kann hiermit durch alle Key-Value-Paare des Objekts iteriert werden. Jedes Element der Schleife ist ein `JsonProperty` mit einem Namen (`Name`) und einem Wert (`Value`). Dies ist besonders nützlich, um dynamische Validierungsfehler auszuwerten.

Konkrete Verwendung im Integration Test

Die folgende Klasse zeigt den Test des → ContingentsControllers mit Hilfe der Hilfsklasse → TestWebApplicationFactory.

→ *Eventmanager.Test/ContingentsControllerTests.cs*

```
using Eventmanager.Application.Commands;
using Eventmanager.Application.Dtos;
using Eventmanager.Infrastructure;
using Eventmanager.Model;
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Time.Testing;
using System;
using System.Linq;
using System.Threading.Tasks;

namespace Eventmanager.Test;

public class ContingentsControllerTests : IDisposable
{
    private readonly TestWebApplicationFactory<EventContext> _factory;
    private readonly TimeProvider _timeProvider =
        new FakeTimeProvider(
            new DateTimeOffset(2026, 2, 28, 16, 30, 0, TimeSpan.Zero));
    private readonly SqliteConnection _connection =
        new SqliteConnection("DataSource=:memory:");
    public ContingentsControllerTests()
    {
        _factory = new TestWebApplicationFactory<EventContext>();
        _factory.SubstituteService<EventContext>(
            opt => opt.UseSqlite(_connection));
        _factory.SubstituteService<TimeProvider>(
            opt => _timeProvider, ServiceLifetime.Singleton);
    }
    /// <summary>
    /// Creates global test data for all unit tests.
    /// </summary>
    private void GenerateFixtures()
    {
        _factory.InitializeDatabase(db =>
        {
            var @event = new Event("Event1");
            var show = new Show(
                @event, new DateTime(2026, 3, 7, 14, 0, 0, DateTimeKind.Utc));
            var contingent = new Contingent(show, ContingentType.Rang, 10);
            db.AddRange(show, contingent);
            db.SaveChanges();
        });
    }
    /// <summary>
    /// Checks whether GET /api/contingents/{id} returns HTTP 200 and a ContingentDto.
    /// </summary>
    [Fact]
    public async Task GetContingentByIdReturns200WithDataTest()
    {
        GenerateFixtures();
        var (response, contingent) =
```

```
            await _factory.GetHttpContent<ContingentDto>("/api/contingents/1");
        Assert.Equal(200, (int)response.StatusCode);
        Assert.NotNull(contingent);
        Assert.True(contingent.Id != default);
    }
    /// <summary>
    /// Checks whether POST /api/contingents returns HTTP 201 with ID
    /// and modifies the database.
    /// </summary>
    [Fact]
    public async Task CreateContingentReturns201Test()
    {
        GenerateFixtures();
        var (response, content) = await _factory.PostHttpContent(
            "/api/contingents",
            new CreateContingentCmd(1, "Floor", 10));
        Assert.Equal(201, (int)response.StatusCode);
        Assert.True(_factory.QueryDatabase(
            db => db.Contingents.Any(c => c.Id == 2)));
        // Create returns a JSON: { id : 12 }.
        // You can read the JSON with JsonElement Methods.
        Assert.True(content.GetProperty("id").GetInt32() != default);
    }
    /// <summary>
    /// Checks whether PUT /api/contingents/{id} returns HTTP 204
    /// and modifies the database.
    /// </summary>
    [Fact]
    public async Task UpdateContingentReturns204Test()
    {
        GenerateFixtures();
        var (response, content) = await _factory.PutHttpContent(
            "/api/contingents/1",
            new UpdateContingentCmd(1, 1, "Floor", 10, 0));
        Assert.Equal(204, (int)response.StatusCode);
        Assert.True(_factory.QueryDatabase(
            db => db.Contingents.Any(
                c => c.Id == 1 && c.ContingentType == ContingentType.Floor)));
    }
    /// <summary>
    /// Checks whether DELETE /api/contingents/{id} returns HTTP 204
    /// and modifies the database.
    /// </summary>
    [Fact]
    public async Task DeleteContingentReturns204Test()
    {
        GenerateFixtures();
        var (response, content) =
            await _factory.DeleteHttpContent("/api/contingents/1");
        Assert.Equal(204, (int)response.StatusCode);
        Assert.False(_factory.QueryDatabase(
            db => db.Contingents.Any(c => c.Id == 1)));
    }
    /// <summary>
    /// Checks whether POST /api/contingents returns HTTP 400
    /// with the corresponding error message in the problem detail.
```

```
    /// </summary>
    [Theory]
    [InlineData("xxx", 1, "invalid value for contingent type")]
    [InlineData("Floor", 999, "show 999 not found")]
    [InlineData("Rang", 1, "show has already a contingent for")]
    public async Task CreateContingentReturns400WithProblemDetailTest(
        string contingentType, int showId, string errorMessage)
    {
        GenerateFixtures();
        var (response, content) = await _factory.PostHttpContent(
            "/api/contingents",
            new CreateContingentCmd(showId, contingentType, 10));
        Assert.Equal(400, (int)response.StatusCode);
        Assert.True(_factory.ContentContainsErrorMessage(content, errorMessage));
    }
    public void Dispose()
    {
        _factory.Dispose();
        _connection.Dispose();
    }
}
```

○ Mocking mit NSubstitute

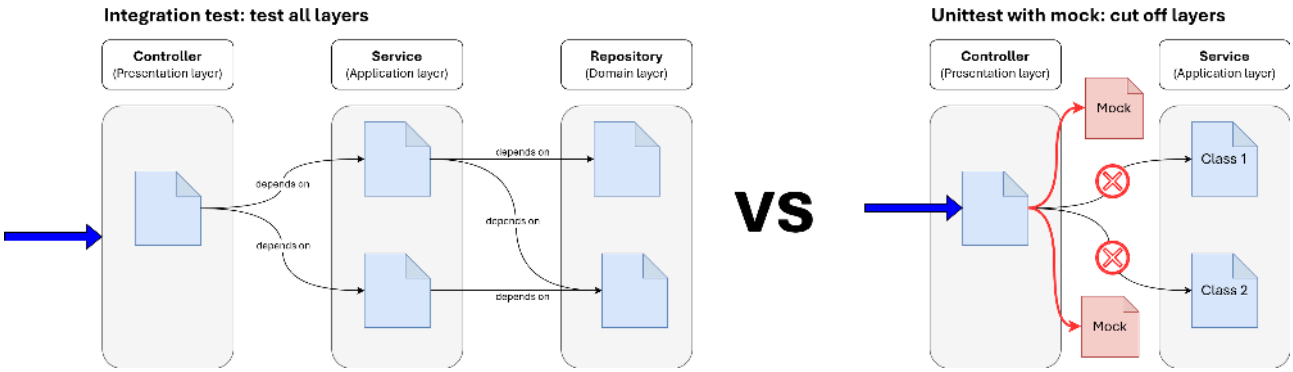
Warum Mocking?

Integration Tests prüfen das gesamte System inklusive Datenbank und Routing. Das ist wichtig, aber diese Tests sind vergleichsweise langsam und aufwendig aufzusetzen, besonders wenn wir gezielt Randfälle oder Fehlerzustände (z. B. "Datenbank nicht erreichbar", "Eintrag nicht gefunden") testen wollen.

Echte Unit Tests zeichnen sich durch absolute Isolation aus. Wenn wir z. B. eine Controllerklasse testen wollen, wollen wir nur die Logik *des Controllers* testen, nicht die Logik der darunterliegenden Schichten.

Da unser Controller aber über Dependency Injection von einem Service (*IEventService*) abhängig ist, müssen wir diesen Service "vortäuschen". Dieser Vorgang wird *Mocking* genannt. Ein Mock ist ein Dummy-Objekt, das das Interface implementiert, aber keine echte Logik besitzt. Wir können dem Mock genau vorschreiben, wie er sich verhalten soll.

Die Grafik stellt den Integration Test und den Unittest mit Mocking gegenüber. Der Integration Test testet alle Schichten, während mit Hilfe von Mocking darunterliegende Schichten "weggeschnitten" werden.



Das Paket NSubstitute

Für C# gibt es verschiedene Mocking-Frameworks. Wir verwenden *NSubstitute*, da es sich durch eine besonders klare, lesbare und flüssige Syntax auszeichnet.

Um NSubstitute in einem Testprojekt zu verwenden, und *IQueryable* zu mocken, müssen folgende Dependencies installiert werden:

```
<PackageReference Include="NSubstitute" Version="5.*" />
<PackageReference Include="MockQueryable.NSubstitute" Version="10.*" />
```

Kurzreferenz zu NSubstitute

Beispiel	Erklärung
`var mock = Substitute.For<IEventService>()`	Erstellt einen Mock

`mock`
`.CreateContingent(Arg.Any<CreateContingentCmd>())`
`.Returns(new Contingent(...));`
	Liefert für jedes Argument, mit dem *CreateContingent* aufgerufen wurde, einen bestimmten Wert zurück.

`mock`
`.CreateContingent(Arg.Is<CreateContingentCmd>(`
`  cmd => cmd.ShowId == 10))`
`.Returns(new Contingent(...));`
	Liefert für Argumente, dessen ShowId den Wert 10 hat, einen bestimmten Wert zurück.

`mock`
`.CreateContingent(Arg.Any<CreateContingentCmd>())`
`.Returns(opt => new Contingent(`
`  ...,`
`  opt.Arg<CreateContingentCmd>().AvailableTickets))`
	Liefert für jedes Argument, mit dem *CreateContingent* aufgerufen wurde, ein *Contingent* mit dem Wert *AvailableTickets*, der im Parameter definiert wurde, zurück.

`mock`
`.CreateContingent(Arg.Any<CreateContingentCmd>())`
`.Throws(info =>`
`  new EventServiceException("Error message"));`
	Wirft für jedes Argument, mit dem *CreateContingent* aufgerufen wurde, eine *EventServiceException*.

`mock`
`.Received()`
`.CreateContingent(Arg.Any<CreateContingentCmd>());`
	Prüft, ob *CreateContingent* mit einem beliebigen Argument aufgerufen wurde. Wirft eine *ReceivedCallsException*, falls die Methode nie aufgerufen wurde. Wird im *assert* Teil verwendet.

Dies ist nur ein kleiner Ausschnitt der Möglichkeiten. Mehr darüber gibt es auf nsubstitute.github.io.

Mocken des Services im Controllertest

Das Prinzip des Mockens von Dependencies können wir jetzt auf unseren Controller übertragen. Der *CategoriesController* hat als Dependency das *EventService*. Da der Controller das Interface *IEventService* anfordert, können wir mit NSubstitute eine Implementierung dieses Interfaces erstellen.

```
public interface IEventService
{
    IQueryable<Contingent> Contingents { get; }
    Task<Contingent> CreateContingent(CreateContingentCmd cmd);
    Task UpdateContingent(UpdateContingentCmd cmd);
    Task UpdateAvailableTickets(UpdateContingentAvailableTicketsCmd cmd);
    Task DeleteContingent(int id);
}
```

Während die Methoden einfach mit NSubstitute zu mocken sind, müssen wir das Property *Contingent* genauer betrachten. Es ist vom Typ *IQueryable<Contingent>*. Dieser Typ wird von EF Core zum Bereitstellen von Datenbankabfragen verwendet. Daher brauchen wir hier wieder unser Paket *MockQueryable.NSubstitute*. Die Methode *BuildMock()* kann eine Liste in ein *IQueryable* umwandeln, das als Rückgabewert für das Property verwendet werden kann. Die GET Endpunkte des Controllers greifen auf *Contingents* zu und erzeugen darüber ihre Daten.

Eine konkrete Implementierung der Integrationtests zeigt die Verwendung. Die Tests sind überraschend kurz. Da wir den Servicelayer mocken, müssen wir die Dependencies des Services wie Datenbank, Timeprovider, etc. *nicht berücksichtigen*. Wir müssen für jeden Endpunkt nur 2 Tests schreiben:

- Liefert der Controller, wenn die Methode *CreateContingent* keine Exception wirft, einen positiven Statuscode (2xx)?
- Liefert der Controller, wenn die Methode *CreateContingent* eine Exception wirft, einen entsprechenden 4xx Statuscode (client error)?

Die Businesslogik des Services kümmert uns nicht. Wir erstellen einfach eine Methode *CreateContingent*, die entweder einen Wert oder eine Fehlermeldung liefert. Damit wir bei den GET Endpunkten Daten im response body bekommen, erstellen wir einige (wenige) Musterdaten.

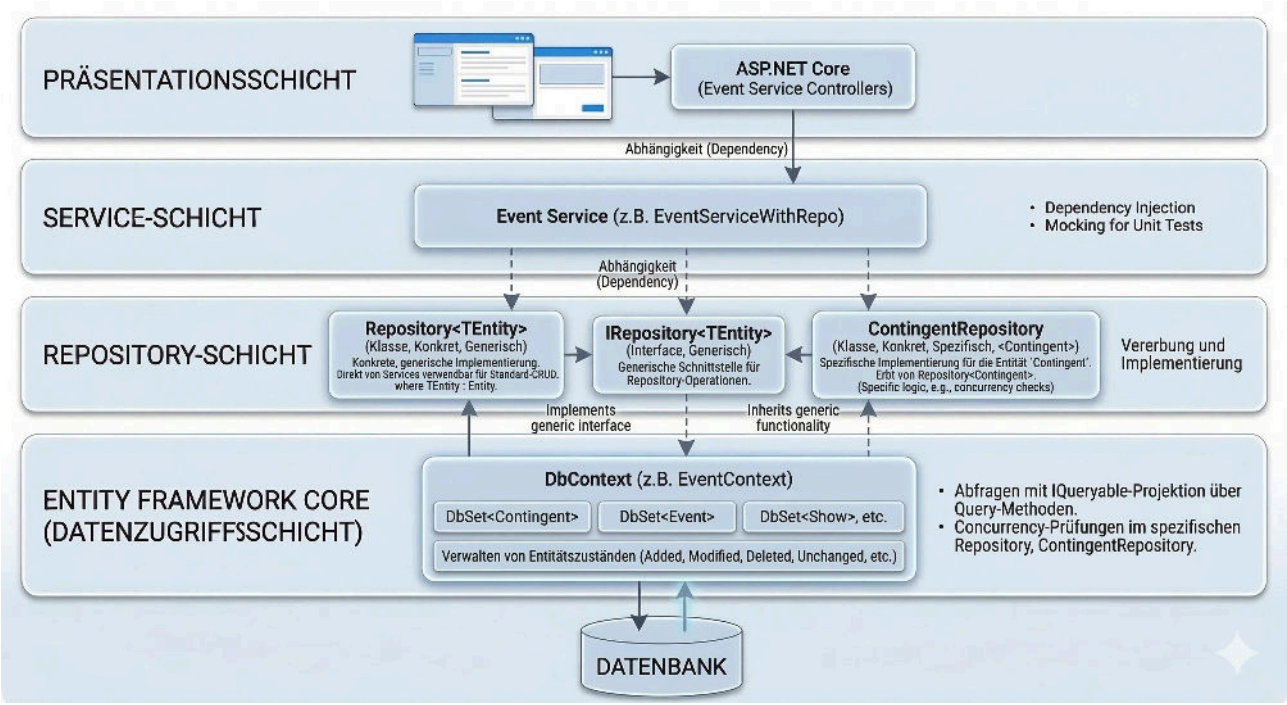
→ Eventmanager.Test/ContingentsControllerWithServiceMockTests.cs

```
using Eventmanager.Application.Commands;
using Eventmanager.Application.Dtos;
using Eventmanager.Application.Services;
using Eventmanager.Infrastructure;
using Eventmanager.Model;
using MockQueryable;
using NSubstitute;
using NSubstitute.ExceptionExtensions;
using System;
using System.Collections.Generic;
using System.Threading.Tasks;

namespace Eventmanager.Test;

public class ContingentsControllerWithServiceMockTests : IDisposable
{
    private readonly TestWebApplicationFactory<EventContext> _factory;
    private readonly IEventService _eventServiceMock =
```

```
        Substitute.For<IEventService>();
    public ContingentsControllerWithServiceMockTests()
    {
        _factory = new TestWebApplicationFactory<EventContext>();
        _factory.SubstituteService<IEventService>(provider => _eventServiceMock);
    }
    private void GenerateFixtures()
    {
        var events = new List<Event>
        {
            new Event("Event") { Id = 1 }
        };
        var shows = new List<Show>
        {
            new Show(events[0], new DateTime(2026, 3, 9, 14, 0, 0)) { Id = 1 }
        };
        var contingents = new List<Contingent>
        {
            new Contingent(shows[0], ContingentType.Rang, 10) { Id = 1 }
        };
        _eventServiceMock.Contingents.Returns(contingents.BuildMock());
    }
    /// <summary>
    /// Checks whether GET /api/contingents/{id} returns HTTP 200
    /// and a ContingentDto.
    /// </summary>
    [Fact]
    public async Task GetContingentByIdReturns200WithDataTest()
    {
        GenerateFixtures();
        var (response, contingent) =
            await _factory.GetHttpContent<ContingentDto>("/api/contingents/1");
        Assert.Equal(200, (int)response.StatusCode);
        Assert.NotNull(contingent);
        Assert.True(contingent.Id == 1);
    }
    /// <summary>
    /// Checks whether POST /api/contingents returns HTTP 201
    /// with ID and modifies the database.
    /// </summary>
    [Fact]
    public async Task CreateContingentReturns201Test()
    {
        var command = new CreateContingentCmd(1, "Floor", 10);
        _eventServiceMock.CreateContingent(
            Arg.Any<CreateContingentCmd>()).Returns(opt =>
                new Contingent(
                    new Show(new Event("Event"),
                    new DateTime(2026, 3, 9, 20, 0, 0)),
                    ContingentType.Floor, 10)
                { Id = 1 });

        var (response, content) = await _factory.PostHttpContent(
            "/api/contingents",
            command);
        await _eventServiceMock.Received().CreateContingent(
```

```
            Arg.Any<CreateContingentCmd>());
        Assert.Equal(201, (int)response.StatusCode);
    }
    /// <summary>
    /// Checks whether POST /api/contingents returns HTTP 400
    /// with the corresponding error message in the problem detail.
    /// </summary>
    [Fact]
    public async Task
CreateContingentReturns400IfCreateContingentThrowsEventServiceExceptionTest()
    {
        _eventServiceMock
            .CreateContingent(Arg.Any<CreateContingentCmd>())
            .Throws(info => new EventServiceException("Error message"));
        var (response, content) = await _factory.PostHttpContent(
            "/api/contingents",
            new CreateContingentCmd(1, "Floor", 10));
        await _eventServiceMock.Received().CreateContingent(
            Arg.Any<CreateContingentCmd>());
        Assert.Equal(400, (int)response.StatusCode);
        Assert.True(_factory.ContentContainsErrorMessage(content, "Error message"));
    }
    public void Dispose()
    {
        _factory.Dispose();
    }
}
```

○ Das Repository Pattern



Das Problem mit dem DbContext

Wenn wir unseren *EventService* betrachten, fordert es 3 Dependencies im Konstruktor an:

```
public class EventService(
    EventContext db, TimeProvider timeProvider, bool isDevelopment) { }
```

Dependencies sollen folgenden Kriterien entsprechen:

- Sie sollten minimal sein. Fordere nur Features an, die du auch brauchst. Das erleichtert das Mocken.
- Beziehe dich auf Interfaces, nicht auf konkrete Implementierungen.

Der *EventContext*, abgeleitet von *DbContext* ist eine sehr mächtige Klasse, die die gesamte Datenbank manipulieren kann. Es geht aus dieser Dependency nicht hervor, welche Features *genau* wir von der Datenbank brauchen.

Ein weiteres Problem sind die Properties vom Typ *DbSet* in *EventContext*.

```
public class EventContext : DbContext
{
    public DbSet<Contingent> Contingents => Set<Contingent>();
    public DbSet<Event> Events => Set<Event>();
    public DbSet<Guest> Guests => Set<Guest>();
    public DbSet<Show> Shows => Set<Show>();
    public DbSet<Ticket> Tickets => Set<Ticket>();
```

```
    public EventContext()
    { }
}
```

Im Kapitel *Mocking* haben wir zwar die Technik gelernt, wie Properties vom Typ *IQueryable* (den das *DbSet* unter anderem implementiert) gemockt werden können, dies ist jedoch mit viel Aufwand verbunden.

Daher können wir zwischen Service und Datenbank eine Zwischenschicht definieren: das *Repository*.

Implementierung eines Repositories

Das generische Repository

Wir beginnen einmal mit der Implementierung der Klasse *Repository*. Es bekommt als Dependency einen allgemeinen *DbContext*. Da wir das Feld *db* explizit mit der Sichtbarkeit *protected* definieren müssen (damit abgeleitete Klassen darauf zugreifen können), ist der klassische Konstruktor hier übersichtlicher als ein Primary Constructor, der Felder standardmäßig *private* anlegt

```
public class Repository<TEntity> : IRepository<TEntity> where TEntity : Entity
{
    protected readonly DbContext db;

    public Repository(DbContext db)
    {
        this.db = db;
    }
}
```

Repositories beziehen sich auf ein (1) Entity. Sie abstrahieren technisch gesehen das DbSet, nicht den DbContext.

Die Basisklasse *Entity* für alle Entities

Die *Repository* Klasse ist generisch, d. h. sie kann mit jedem Entity verwendet werden. Damit wir nicht beliebige Klassen verwenden können, schränken wir mit *where TEntity : Entity* auf alle Klassen ein, die von *Entity* abgeleitet sind. Diese Klasse ist im Projekt definiert und ist die Basisklasse aller Entities. Sie definiert das *Id* Property. Das ist wichtig, damit wir bei den lesenden Methoden auf *Id* zugreifen können. Die Modelklassen leiten sich dann von *Entity* ab.

→ Entity.cs

```
public abstract class Entity
{
```

```
    public int Id { get; set; }
}
```

Die Schreibmethoden im generischen Repository

Zuerst definieren wir in der generischen Klasse *Repository* die Methoden, die die Insert, Update und Delete Funktionen abbilden.

```
public class Repository<TEntity> : IRepository<TEntity> where TEntity : Entity
{
    /* ... */
    public virtual async Task<TEntity> CreateAndSave(TEntity entity)
    {
        db.Entry(entity).State = EntityState.Added;
        await db.SaveChangesAsync();
        return entity;
    }

    public virtual async Task UpdateAndSave(TEntity entity)
    {
        db.Entry(entity).State = EntityState.Modified;
        await db.SaveChangesAsync();
    }

    public virtual async Task DeleteAndSave(TEntity entity)
    {
        db.Entry(entity).State = EntityState.Deleted;
        await db.SaveChangesAsync();
    }
}
```

Den Aufruf von *SaveChangesAsync* in den einzelnen Methoden müssen wir kritisch betrachten. Es beginnt eine Transaktion mit einer Anweisung und beendet diese. Möchten wir im Service zwei zusammenhängende Änderungen durchführen, und die zweite Anweisung schlägt fehl, ist die erste Änderung schon geschrieben und wird nicht rückgängig gemacht. Eine Lösung wäre das Auslagern von *SaveChangesAsync* in eine eigene Methode im Repository. Deswegen enden die Methoden auch mit *AndSave*, damit dieses Verhalten klar hervorgeht.

Die Lesemethoden im generischen Repository

Zuerst diskutieren wir die grundlegenden Lesemethoden, wie sie klassisch in einem Repository vorhanden sind:

```
public class Repository<TEntity> : IRepository<TEntity> where TEntity : Entity
{
   /* ... */
    public virtual Task<TEntity?> FindByIdAsync(int id)
```

```
        => db.Set<TEntity>().FirstOrDefaultAsync(e => e.Id == id);
    public virtual Task<TEntity?> FindByIdAsync(int id, params string[]
includeNavigations)
        => FindByIdAsync<TEntity>(id, includeNavigations);
    public virtual Task<bool> ExistsAsync(Expression<Func<TEntity, bool>> predicate)
        => db.Set<TEntity>().AnyAsync(predicate);
}
```

Task<TEntity?> FindByIdAsync(int id)

Die häufigste Operation ist sicher das Suchen nach dem Primärschlüssel. Mit *await eventsRepo.FindByIdAsync(2)* kann z. B. nach dem Event mit der ID 2 gesucht werden.

Task<TEntity?> FindByIdAsync(int id, params string[] includeNavigations)

Etwas spezieller ist diese Methode. Oft wollen wir Entities inklusive ihrer Navigation-Properties (verbundene Daten) laden. Ohne diesen Include wären diese Properties null bzw. leere Listen. Mit *await eventsRepo.FindByIdAsync(2, nameof(Event.Shows))* können wir das Event mit der ID 2 samt ihren Shows laden.

Task<bool> ExistsAsync(Expression<Func<TEntity, bool>> predicate)

Für Existenzabfragen stellen wir eine Methode bereit. So kann z. B. mit *await eventsRepo.ExistsAsync(e ⇒ !e.Shows.AnyAsync())* geprüft werden, ob ein Event ohne Shows existiert. Die Methode liefert wie *AnyAsync* einen *boolean* Wert zurück.

Lesemethoden für andere Entities: Der Kompromiss

Wie immer im Leben müssen wir hier auch einen Kompromiss zwischen reiner Architektur und Praxis eingehen. Oft müssen wir z. B. in einer Servicemethode einen Fremdschlüssel auflösen. Mit *await db.Shows.FirstOrDefaultAsync(s ⇒ s.Id == cmd.ShowId)* wollen wir z. B. die Show mit einer bestimmten ID lesen. Steht uns nur das Repository für Contingents zur Verfügung, brauchen wir extra ein *ShowsRepository*. Beim Controller wird es überhaupt schwierig. Er verwendet LINQ, um auf die DTO Klassen zu projizieren.

Würden wir in den Repositories mit Listen arbeiten, können wir die Vorteile von *IQueryable* nicht nutzen: Er generiert SQL Klauseln, die nur die benötigten Daten liefern.

Daher definieren wir auch Lesemethoden für andere Entities mit dem Typparameter *TOther*.

```
public class Repository<TEntity> : IRepository<TEntity> where TEntity : Entity
{
    public virtual Task<TOtherEntity?> FindByIdAsync<TOtherEntity>(int id)
        where TOtherEntity : Entity
        => db.Set<TOtherEntity>().FirstOrDefaultAsync(e => e.Id == id);

    public virtual Task<TOtherEntity?> FindByIdAsync<TOtherEntity>(
        int id,
        params string[] includeNavigations)
        where TOtherEntity : Entity
    {
        var query = db.Set<TOtherEntity>().AsQueryable();
```

```
        foreach (var include in includeNavigations)
            query = query.Include(include);

        return query.FirstOrDefaultAsync(e => e.Id == id);
    }

    public virtual TResult Query<TResult>(Func<IQueryable<TEntity>, TResult> queryFunc)
        => queryFunc(db.Set<TEntity>());
}
```

Die Methode *FindByIdAsync()* kann nun auf andere Entities angewandt werden. So liest z. B. *await eventsRepo.FindById<Show>(2)* die *Show* mit der ID 2. *await eventsRepo.FindById<Show>(2, nameof(Show.Events))* findet die *Show* mit der ID 2 und lädt alle Events dazu.

Die Methode *Query* ist für die Weitergabe an den Controller gedacht. Mit *await eventsRepo.Query(event ⇒ events.Select(e ⇒ new EventDto(e.Id, e.Name)).ToListAsync())* kann ein Event auf eine DTO projiziert werden. Über *type inference* des Compilers wird der Rückgabetyp gemäß der übergebenen Funktion gesetzt, d. h. die *Query* Methode liefert in diesem Fall einen Task mit einer Liste von EventDto Instanzen zurück.

Damit auch der SQL Code entsprechend generiert werden kann, wird der *Query* Methode eine *Expression* übergeben. Bei einer *Func* würde EF Core diese Abfrage clientseitig ausführen.

Der gesamte Code und das Interface

→ IRepository.cs

```
using Eventmanager.Application.Model;
using System;
using System.Linq;
using System.Linq.Expressions;
using System.Threading.Tasks;

namespace Eventmanager.Application.Repositories
{
    public interface IRepository<TEntity> where TEntity : Entity
    {
        Task<TEntity?> FindByIdAsync(int id);
        Task<TEntity?> FindByIdAsync(
            int id, params string[] includeNavigations);
        Task<TOtherEntity?> FindByIdAsync<TOtherEntity>(
            int id) where TOtherEntity : Entity;
        Task<TOtherEntity?> FindByIdAsync<TOtherEntity>(
            int id, params string[] includeNavigations) where TOtherEntity : Entity;
        Task<bool> ExistsAsync(Expression<Func<TEntity, bool>> predicate);
        TResult Query<TResult>(Func<IQueryable<TEntity>, TResult> queryFunc);
        Task<TEntity> CreateAndSave(TEntity entity);
        Task DeleteAndSave(TEntity entity);
        Task UpdateAndSave(TEntity entity);
    }
}
```

→ *Repository.cs*

```
using Eventmanager.Application.Model;
using Microsoft.EntityFrameworkCore;
using System;
using System.Linq;
using System.Linq.Expressions;
using System.Threading.Tasks;

namespace Eventmanager.Application.Repositories;

public class Repository<TEntity> : IRepository<TEntity> where TEntity : Entity
{
    protected readonly DbContext db;

    public Repository(DbContext db)
    {
        this.db = db;
    }

    public virtual Task<TOtherEntity?> FindByIdAsync<TOtherEntity>(int id)
        where TOtherEntity : Entity
        => db.Set<TOtherEntity>().FirstOrDefaultAsync(e => e.Id == id);

    public virtual Task<TEntity?> FindByIdAsync(int id)
        => db.Set<TEntity>().FirstOrDefaultAsync(e => e.Id == id);

    public virtual Task<TEntity?> FindByIdAsync(
        int id, params string[] includeNavigations)
        => FindByIdAsync<TEntity>(id, includeNavigations);

    public virtual Task<TOtherEntity?> FindByIdAsync<TOtherEntity>(
        int id, params string[] includeNavigations) where TOtherEntity : Entity
    {
        var query = db.Set<TOtherEntity>().AsQueryable();
        foreach (var include in includeNavigations)
            query = query.Include(include);

        return query.FirstOrDefaultAsync(e => e.Id == id);
    }

    public virtual Task<bool> ExistsAsync(Expression<Func<TEntity, bool>> predicate)
        => db.Set<TEntity>().AnyAsync(predicate);

    public virtual TResult Query<TResult>(Func<IQueryable<TEntity>, TResult> queryFunc)
        => queryFunc(db.Set<TEntity>());

    public virtual async Task<TEntity> CreateAndSave(TEntity entity)
    {
        db.Entry(entity).State = EntityState.Added;
        await db.SaveChangesAsync();
        return entity;
    }

    public virtual async Task UpdateAndSave(TEntity entity)
```

```
    {
        db.Entry(entity).State = EntityState.Modified;
        await db.SaveChangesAsync();
    }

    public virtual async Task DeleteAndSave(TEntity entity)
    {
        db.Entry(entity).State = EntityState.Deleted;
        await db.SaveChangesAsync();
    }
}
```

Spezifische Repositories

Die Klasse *Repository* ist *nicht abstrakt*, d. h. sie kann in ASP.NET Core in *Program.cs* registriert werden:

```
builder.Services.AddScoped<IRepository<Show>, Repository<Show>>((provider) =>
    new Repository<Show>(db: provider.GetRequiredService<EventContext>()));
```

Nun kann jedes Service die Dependency *IRepository<Show>* anfordern und bekommt ein fertiges Show Repository.

Möchten wir spezifischen Code implementieren, können wir natürlich das Repository als Basisklasse verwenden und davon ableiten:

→ IContingentRepository.cs

```
using Eventmanager.Model;
using System.Threading.Tasks;

namespace Eventmanager.Application.Repositories;

public interface IContingentRepository : IRepository<Contingent>
{
    Task Update(Contingent entity, long version);
}
```

→ ContingentRepository.cs

```
using Eventmanager.Model;
using Microsoft.EntityFrameworkCore;
using System;
using System.Threading.Tasks;

namespace Eventmanager.Application.Repositories;

public class ContingentRepository(DbContext db, TimeProvider timeProvider) :
Repository<Contingent>(db), IContingentRepository
{
```

```
    public async Task Update(Contingent entity, long version)
    {
        if (entity.Version != version)
            throw new DbUpdateConcurrencyException($"The contingent has already
changed.");

        entity.Version = timeProvider.GetTimestamp();
        await base.UpdateAndSave(entity);
    }

    public override Task UpdateAndSave(Contingent entity) => Update(entity, default);
}
```

Hier sieht man auch schon, welcher Code in ein Repository kommt: Code, der bei *jeder* Update Operation ausgeführt werden soll. In unserem Fall ist es der Vergleich der Version.

Andere Fälle für Code, der im Repository implementiert werden kann, sind:

- Häufig benutzte Abfragen, meist parametrisiert.
- Starten von Datenbankprozeduren und Zugriff auf Objekte in der Datenbank.
- Speziell optimierter SQL Code, der mit den *SqlRaw* Funktionen in EF Core genutzt wird.
- Sehr tief liegende Prüfungen, die keine Serviceklasse umgehen darf.

Verwendung im Service

Das nachfolgende Beispiel zeigt, wie das *ContingentRepository* verwendet werden kann. Es erbt von *Repository<Contingent>* und hat daher alle oben beschriebenen Methoden enthalten. Natürlich kann auch *IRepository<Contingent>* angefordert werden, falls es kein eigenes *ContingentRepository* gibt. Achte auf die korrekte Registrierung im Serviceprovider in der *Program.cs*.

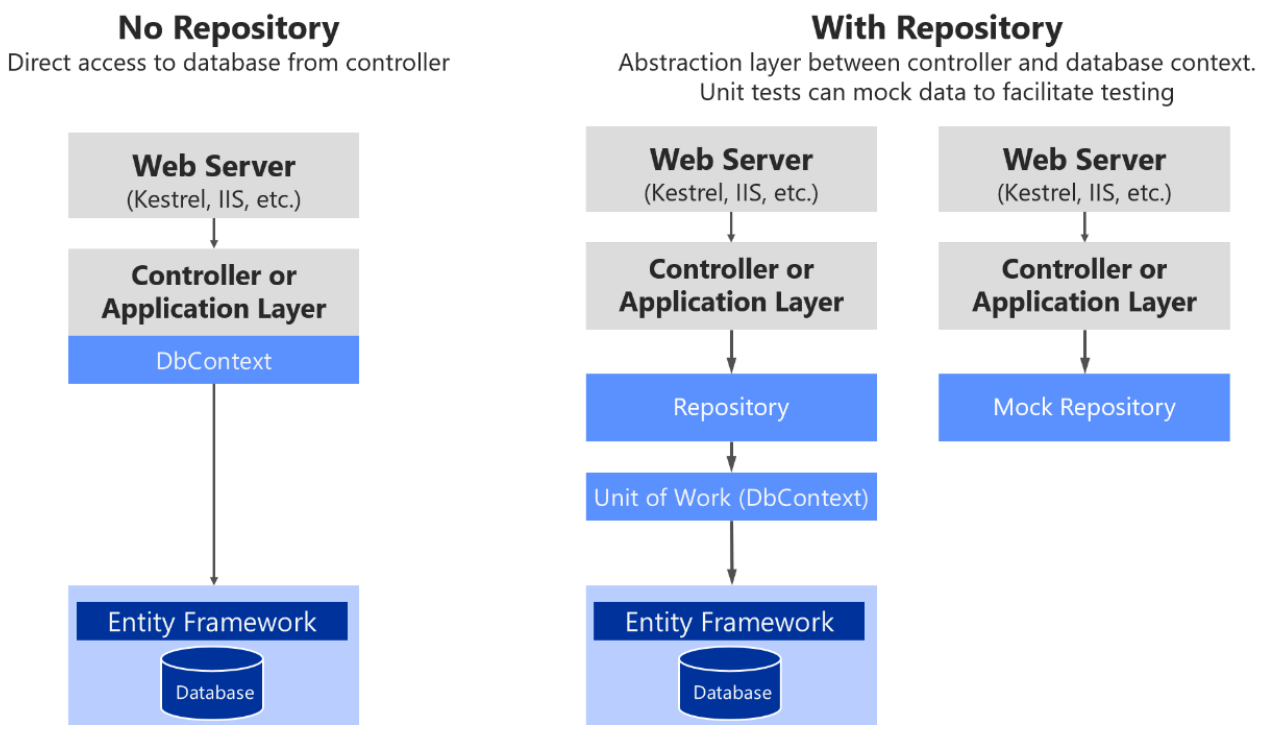
→ EventServiceWithRepo.cs

```
public class EventServiceWithRepo(IContingentRepository contingentRepo, bool
isDevelopment)
{
    public TResult QueryContingents<TResult>(Func<IQueryable<Contingent>, TResult>
queryFunc)
        => contingentRepo.Query(queryFunc);

    public async Task<Contingent> CreateContingent(CreateContingentCmd cmd)
    {
        /* ... */
        // Find show with type parameter.
        var show = await contingentRepo.FindByIdAsync<Show>(cmd.ShowId)
            ?? throw new EventServiceException($"Show {cmd.ShowId} not found.");

        // Use ExistsAsync
        if (await contingentRepo.ExistsAsync(c => c.Show.Id == cmd.ShowId && c
.ContingentType == contingentType))
            throw new EventServiceException($"Show has already a contingent for {cmd
.ContingentType}.");
```

```
        var contingent = new Contingent(show, contingentType, cmd.AvailableTickets);
        await RethrowDbException(() => contingentRepo.CreateAndSave(contingent));
        return contingent;
    }

    public async Task UpdateContingent(UpdateContingentCmd cmd)
    {
        /* ... */
        // Use overwritten Version of Update.
        await RethrowDbException(() => contingentRepo.Update(contingent, cmd.Version));
    }

    public async Task DeleteContingent(int id)
    {
        // Loading with Navigations
        var contingent = await contingentRepo.FindByIdAsync(id, nameof(Contingent
.Tickets))
            ?? throw new EventServiceNotFoundException($"Contingent {id} not found.");

        if (contingent.Tickets.Any())
            throw new EventServiceException("The contingent has already sold or
reserved all the tickets.");

        await RethrowDbException(() => contingentRepo.DeleteAndSave(contingent));
    }

    private async Task RethrowDbException(Func<Task> action)
    {
        try
        {
            await action();
        }
        catch (DbUpdateConcurrencyException e)
        {
            var message = isDevelopment ? e.InnerException?.Message ?? e.Message : "The
record has already changed.";
            throw new EventServiceException(message);
        }
        catch (DbUpdateException e)
        {
            var message = isDevelopment ? e.InnerException?.Message ?? e.Message :
"Cannot write changes to database.";
            throw new EventServiceException(message);
        }
    }
}
```

Unittest mit Mock des Repos

Erst beim Test wird die Stärke des Konzeptes deutlich. Wir können nun das Repository mit *NSubstitute* mocken. Das ist auch die häufigste Begründung für die Implementierung des Repository Patterns.



Quelle: https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-implementation-entity-framework-core

Eine komplette Testklasse, die die Methoden des Repositories mockt, ist hier abgebildet. Sie verwendet die folgenden Mocks:

Mock	Bedeutung
`_contingentRepo.FindByIdAsync<Show>(1).Returns(show);`	Für *CreateContingent*. Liefert die erste Show aus den Fixtures, wenn *FindByIdAsync<Show>* mit dem Parameter 1 aufgerufen wird.
`_contingentRepo.ExistsAsync( Arg.Any<Expression<Func<Contingent, bool>>>()).Returns(false);`	Für *CreateContingent*. Da die *ExistsAny* Funktion bei der Prüfung auf vorhandene Kontingente mit gleichem Namen aufgerufen wird, legen wir fest, dass sie false liefert.
`_contingentRepo.FindByIdAsync<Show>(999).Returns(null as Show);`	Für *CreateContingent*. Um das Verhalten bei einer ungültigen Show im command object zu prüfen. Das command object hat natürlich 999 als *showId*.
`_contingentRepo.FindByIdAsync(Arg.Any<int>(), nameof(Contingent.Tickets)).Returns(null as Contingent);`	Für *DeleteContingent*. Simuliert ein nicht vorhandenes Kontingent, das wir löschen wollen. Dabei liefert der Mock der Methode für jeden ID Wert *null*.
`_contingentRepo.FindByIdAsync(1, nameof(Contingent.Tickets)).Returns(contingent);`	Für *DeleteContingent*. Bei Delete werden die Navigation Tickets inkludiert. Wir müssen daher ein Kontingent mit Tickets erzeugen und dies als Rückgabewert festlegen.

→ *EventServiceWithRepoTest.cs*

```
using Eventmanager.Application.Commands;
using Eventmanager.Application.Repositories;
using Eventmanager.Application.Services;
using Eventmanager.Model;
using NSubstitute;
using System;
using System.Linq.Expressions;
using System.Threading.Tasks;

namespace Eventmanager.Test;

public class EventServiceWithRepoTest
{
    private readonly IContingentRepository _contingentRepo;
    private readonly EventServiceWithRepo _service;

    public EventServiceWithRepoTest()
    {
        _contingentRepo = Substitute.For<IContingentRepository>();
        _service = new EventServiceWithRepo(_contingentRepo, isDevelopment: true);
    }

    private (Event[] events, Show[] shows, Contingent[] contingents) GenerateFixtures()
    {
        var events = new Event[] { new Event("Event1") { Id = 1 } };
        var shows = new Show[]
        {
            new Show(events[0], new DateTime(2026, 6, 12, 18, 0, 0)) { Id = 1 }
        };
        var contingents = new Contingent[]
        {
            new Contingent(shows[0], ContingentType.Rang, 100)
        };
        return (events, shows, contingents);
    }
    [Fact]
    public async Task CreateContingent_WithValidData_CreatesAndSavesContingent()
    {
        // Arrange
        var show = GenerateFixtures().shows[0];
        var cmd = new CreateContingentCmd(1, "Floor", 10);
        _contingentRepo.FindByIdAsync<Show>(1).Returns(show);
        _contingentRepo.ExistsAsync(
            Arg.Any<Expression<Func<Contingent, bool>>>()).Returns(false);

        // Act
        var result = await _service.CreateContingent(cmd);

        // Assert
        Assert.NotNull(result);
        Assert.Equal(ContingentType.Floor, result.ContingentType);
        Assert.Equal(10, result.AvailableTickets);
        await _contingentRepo.Received(1).CreateAndSave(result);
```

```
    }


    [Fact]
    public async Task CreateContingent_WhenShowDoesNotExist_ThrowsException()
    {
        // Arrange
        var cmd = new CreateContingentCmd(999, "Floor", 10);
        _contingentRepo.FindByIdAsync<Show>(999).Returns(null as Show);

        // Act & Assert
        var ex = await Assert.ThrowsAsync<EventServiceException>(
            () => _service.CreateContingent(cmd));
        Assert.Equal("Show 999 not found.", ex.Message);
        await _contingentRepo.DidNotReceive().CreateAndSave(Arg.Any<Contingent>());
    }

    [Fact]
    public async Task DeleteContingent_WhenContingentHasNoTickets_DeletesAndSaves()
    {
        // Arrange
        var contingent = GenerateFixtures().contingents[0];
        _contingentRepo.FindByIdAsync(
            1, nameof(Contingent.Tickets)).Returns(contingent);

        // Act
        await _service.DeleteContingent(1);

        // Assert
        await _contingentRepo.Received(1).DeleteAndSave(contingent);
    }

    [Fact]
    public async Task DeleteContingent_WhenContingentHasTickets_ThrowsException()
    {
        // Arrange
        var contingent = GenerateFixtures().contingents[0];
        contingent.Tickets.Add(
            new Ticket(
                new Guest("first", "last", new DateOnly(2004, 2, 1)),
                contingent,
                TicketState.Sold,
                new DateTime(2026, 3, 19, 14, 0, 0), 2));
        _contingentRepo.FindByIdAsync(
            1, nameof(Contingent.Tickets)).Returns(contingent);

        // Act & Assert
        var ex = await Assert.ThrowsAsync<EventServiceException>(
            () => _service.DeleteContingent(1));
        Assert.Equal(
            "The contingent has already sold or reserved all the tickets.",
            ex.Message);
        // Sicherstellen, dass nichts gelöscht wird
        await _contingentRepo.DidNotReceive().DeleteAndSave(Arg.Any<Contingent>());
    }
```

```
    [Fact]
    public async Task
DeleteContingent_WhenContingentDoesNotExist_ThrowsNotFoundException()
    {
        // Arrange
        _contingentRepo.FindByIdAsync(Arg.Any<int>(), nameof(Contingent.Tickets))
            .Returns(null as Contingent);
        // Act & Assert
        var ex = await Assert.ThrowsAsync<EventServiceNotFoundException>(
            () => _service.DeleteContingent(1));
        Assert.Equal("Contingent 1 not found.", ex.Message);
        // Sicherstellen, dass nichts gelöscht wird
        await _contingentRepo.DidNotReceive().DeleteAndSave(Arg.Any<Contingent>());
    }
}
```

○ Codierung von ID Werten

Das Problem mit auto increment ID Werten

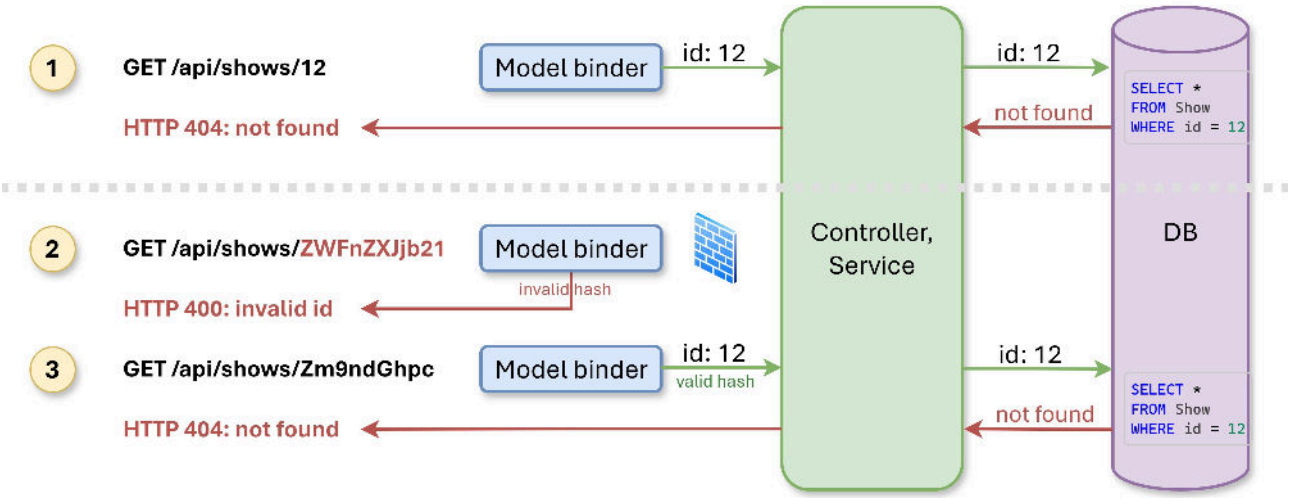
Bisher wurden unsere Ressourcen über den internen Auto-Increment-Wert der Datenbank eindeutig referenziert. Dies hat jedoch mehrere Probleme:

- Der User weiß: Wenn er die Ressource mit der ID 13 anfordert, wird es vermutlich auch eine Ressource mit der ID 12 geben. Dies kann als Angriffsvektor verwendet werden.
- Wenn der User viele Requests mit zufällig gewählter Zahl absendet, gelangt jeder Request bis zur Datenbank.

Daher wollen wir die internen ID Werte nicht herausgeben und sie vor dem Senden codieren. Diese Codierung soll bestimmte Eigenschaften haben:

- Der Server kann *ohne Befragung der Datenbank* feststellen, ob die ID syntaktisch gültig ist.
- Der User soll nicht selbst codierte IDs erstellen können.
- Die Codierung und Decodierung sollen möglichst von ASP selbst erledigt werden, wir wollen diese nicht im Service oder im Controller durchführen.

Folgendes Schema zeigt den Nutzen unserer Idee:



① Mit der klassischen integer ID gelangt *jeder* Request zur Datenbank.
② Besitzt unsere ID allerdings einen *Hashwert*, kann schon der Modelbinder durch den Parse Vorgang feststellen, dass diese manipuliert wurde. Er lehnt den Request sofort ab.
③ Ist die codierte ID syntaktisch gültig, geht der Request weiter. Nicht gefundene Ressourcen sind natürlich weiterhin möglich, aber nur wenn der Client die codierte ID weiß.

Die Id als eigener Typ

In C# können wir - im Gegensatz zu Java oder anderen Programmiersprachen - eigene *Wertetypen* anlegen. Diese werden wie *int*, *double*, etc. am Stack allokiert, können aber durch *Boxing*

auch am Heap angelegt werden. Die bekannteste struct im .NET Framework ist sicher *DateTime*. Structs werden angelegt, wenn ein Wert viele Methoden zur Bearbeitung anbieten soll. Andere Beispiele sind *Point*, *Position*, ...

Da wir hier einen int Wert verwalten und viele Methoden für die Codierung, Decodierung, etc. anbieten wollen, eignet sich daher eine struct hervorragend dafür. Konkret wird die Codierung mit folgender Idee realisiert:

- Mit HMACSHA256 generieren wir - unter Einbeziehung eines *Secrets* - einen Hash. Durch das Secret ist es dem Client nicht möglich, den Hashwert für eine ID selbst zu berechnen.
- Ein SHA256 Wert hat 32 Bytes. Wenn wir den int Wert für die ID zusätzlich speichern, kommen wir auf *36 Bytes* für ID + Hash.
- Die Berechnung soll mit den neuen .NET Methoden, die *zero allocations* am Heap benötigen, realisiert werden. Dazu gehören die Typen *Span*, *ReadOnlySpan* und entsprechende Methoden.

Bei der *Decodierung* berechnen wir den Hash aus den ersten 4 Bytes des codierten Wertes. Stimmen diese mit den restlichen Bytes des codierten Wertes überein, ist der Wert gültig. Einen Auszug der Implementierung zeigt dieses Listing.

Wichtig sind die Methoden *Parse* und *TryParse*. Wenn wir im Controller den Typ *Id* als *routing parameter* verwenden, wird der Modelbinder automatisch die Parse Methode aufrufen, um aus dem String im request body einen Wert vom Typ *Id* zu erzeugen.

→ IdEncoder.cs (Vollständiger Code)

```
public readonly struct Id : IParsable<Id>, IEquatable<Id>
{
    public static byte[] Secret; // Wird global beim Start gesetzt
    private const int _strLen = 48;

    public int Value { get; }
    public Id(int value) => Value = value;
    public bool HasValue => Value != 0;

    // --- KERNLOGIK: CODIERUNG ---
    public string EncodedValue => string.Create(_strLen, Value, (chars, val) =>
    {
        Span<byte> buffer = stackalloc byte[36]; // 4 (int) + 32 (hash)
        BinaryPrimitives.WriteInt32LittleEndian(buffer, val);
        HMACSHA256.HashData(Secret, buffer[..4], buffer[4..]);
        Base64Url.EncodeToChars(buffer, chars);
    });

    // --- KERNLOGIK: DECODIERUNG ---
    public static Id Decode(ReadOnlySpan<char> encodedValue)
    {
        if (encodedValue.Length != _strLen) return default;

        Span<byte> buffer = stackalloc byte[36];
        if (Base64Url.DecodeFromChars(encodedValue, buffer) != 36) return default;
```

```
        Span<byte> computedHash = stackalloc byte[32];
        HMACSHA256.HashData(Secret, buffer[..4], computedHash);

        // Schutz vor Timing-Angriffen beim Hash-Vergleich
        if (!CryptographicOperations.FixedTimeEquals(buffer[4..], computedHash))
            return default;

        return new Id(BinaryPrimitives.ReadInt32LittleEndian(buffer));
    }

    /* ... Implementierung von IParsable (Parse/TryParse), IEquatable,
           überschriebene Objektmethoden und Operatoren (==, !=, Casts) ...
           Details siehe verlinkte Source-Datei! ... */
}
```

Damit Werte vom Typ *Id* automatisch als String im JSON gesendet werden, müssen wir einen *Converter* schreiben. Er schreibt einfach den Base64 codierten Wert in das JSON. Beim Lesen wandeln wir den codierten Wert in ein int zurück um. Ist der Wert 0, wissen wir, dass der codierte Wert ungültig ist.

→ IdConverter.cs

```
public class IdConverter : JsonConverter<Id>
{
    public override Id Read(
        ref Utf8JsonReader reader, Type typeToConvert, JsonSerializerOptions options)
    {
        if (reader.HasValueSequence)
            return Id.Decode(reader.GetString() ?? string.Empty);

        Span<char> chars = stackalloc char[reader.ValueSpan.Length];
        int written = Encoding.UTF8.GetChars(reader.ValueSpan, chars);
        return Id.Decode(chars[..written]);
    }

    public override void Write(Utf8JsonWriter writer, Id id, JsonSerializerOptions
options) =>
        writer.WriteStringValue(id.EncodedValue);
}
```

Damit unser Converter funktioniert, müssen wir ihn in der *Program.cs* - also in der Serverkonfiguration von ASP - registrieren:

```
Id.Secret = Convert.FromBase64String(
    builder.Configuration["IdEncoderSecret"] ?? throw new Exception("Missing
IdEncoderSecret."));
builder.Services.AddControllers()
    .AddJsonOptions(opt => opt.JsonSerializerOptions.Converters.Add(new
IdConverter()));
```

Das *Secret* wird als Base64 codierter Wert in der Datei *appsettings.json* mit dem Key *IdEncoder-Secret* gespeichert oder als Umgebungsvariable bereitgestellt.

Verwendung im Controller

Die DTO Klasse verwendet für den Id Wert statt einem *int* Wert den Typ *Id*.

```
public record GuestDto(Id Id, string Firstname, string Lastname, DateOnly BirthDate);
```

Der nachfolgende Controller zeigt die Verwendung des Typs *Id*. Wir können ihn einfach durch Projektion mit der Methode *Id.From()* erzeugen.

→ *GuestsController.cs*

```
[Route("api/[controller]")]
[ApiController]
public class GuestsController(EventContext db) : ControllerBase
{
    [HttpGet]
    [ProducesResponseType<List<GuestDto>>(StatusCodes.Status200OK)]
    public async Task<ActionResult<List<GuestDto>>> GetGuests()
    {
        var guests = await db.Guests
            .Select(g => new GuestDto(
                Id.From(g.Id), g.Firstname, g.Lastname, g.BirthDate))    ①
            .ToListAsync();
        return Ok(guests);
    }

    [HttpGet("{id}")]
    [ProducesResponseType<GuestDto>(StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status404NotFound)]
    public async Task<ActionResult<GuestDto>> GetGuestById([FromRoute] Id id) ②
    {
        var guest = await db.Guests
            .Where(g => g.Id == id.Value)
            .Select(g => new GuestDto(
                Id.From(g.Id), g.Firstname, g.Lastname, g.BirthDate))
            .FirstOrDefaultAsync();

        if (guest is null) return Problem("Guest not found.", statusCode: 404);
        return Ok(guest);
    }
}
```

① Mit *Id.From()* wird der int Wert der ID codiert.

② Der Typ *Id* kann auch als routing parameter verwendet werden. Dafür haben wir die entsprechenden *Parse* Methoden im Typ *Id*.

Die Antwort auf den Request *GET /api/guests* sieht dann so aus:

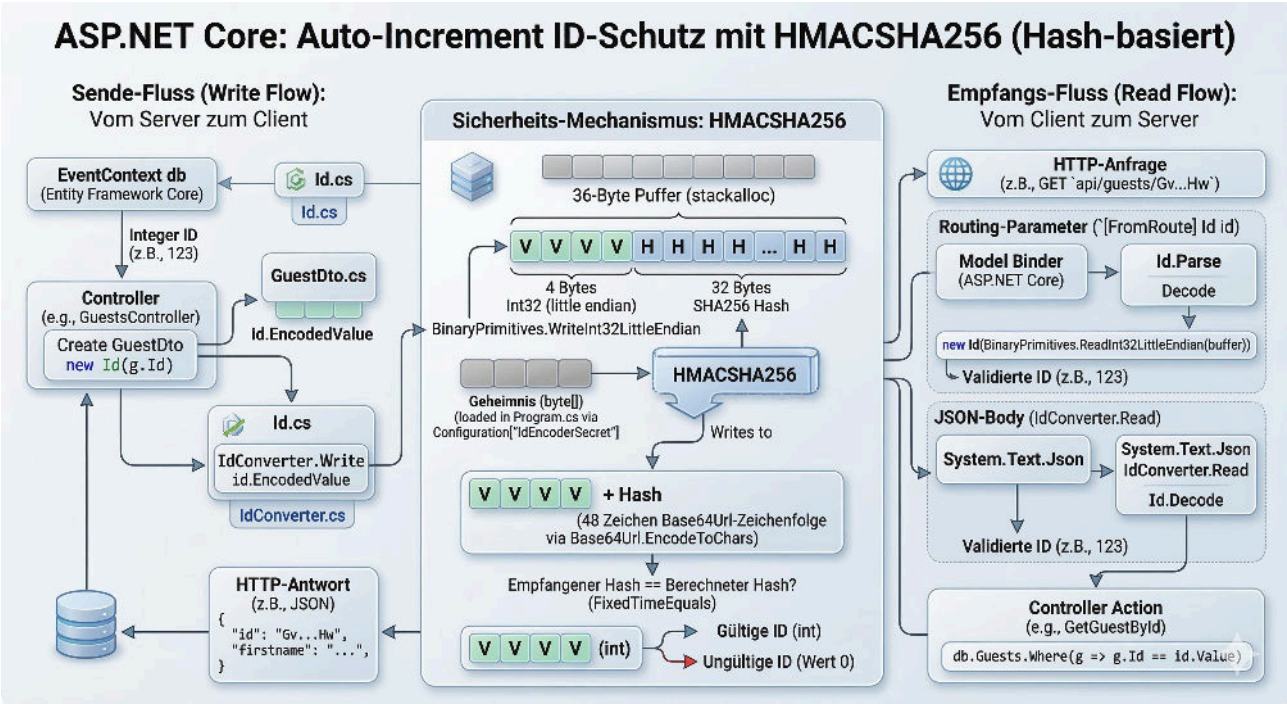
```
[
  {
    "id": "AQAAAGyqe0X_6-HDRGFIqLGetSQ9hZgXV_EMPGF01KKfYwr9",
    "firstname": "Estelle",
    "lastname": "Franzis",
    "birthDate": "2000-05-05",
    "links": {}
  }
]
```

Senden wir den Request *GET /api/guests/AgAAAGJ1xTf-ksLP5BI_kFQzKfou1osvxnXUnmxEw0KRd-VRr*, wird dieser verstanden und liefert die Daten des Gastes mit der ID 1:

```
{
  "id": "AgAAAGJ1xTf-ksLP5BI_kFQzKfou1osvxnXUnmxEw0KRdVRr",
  "firstname": "Rasmus",
  "lastname": "Dippl",
  "birthDate": "2003-04-27",
  "links": {}
}
```

Gesamtüberblick

Die beteiligten Komponenten werden in dieser Grafik dargestellt:



Kritische Betrachtungen

Das Codieren der ID ersetzt nicht das Prüfen des Userrechte auf diese Ressource! Gleiche ID Werte werden immer auf gleiche codierte Werte abgebildet. So steht z. B. *AgAAAGJ1xTf-ksLP5-*

BI_kFQzKfou1osvxnXUnmxEw0KRdVRr immer für den Wert 1. Diese Technik verhindert nur, dass ein Client Abfragen mit unbekannter ID an den Server senden kann.

○ Hypermedia und HATEOAS

Das Prinzip von Hypermedia

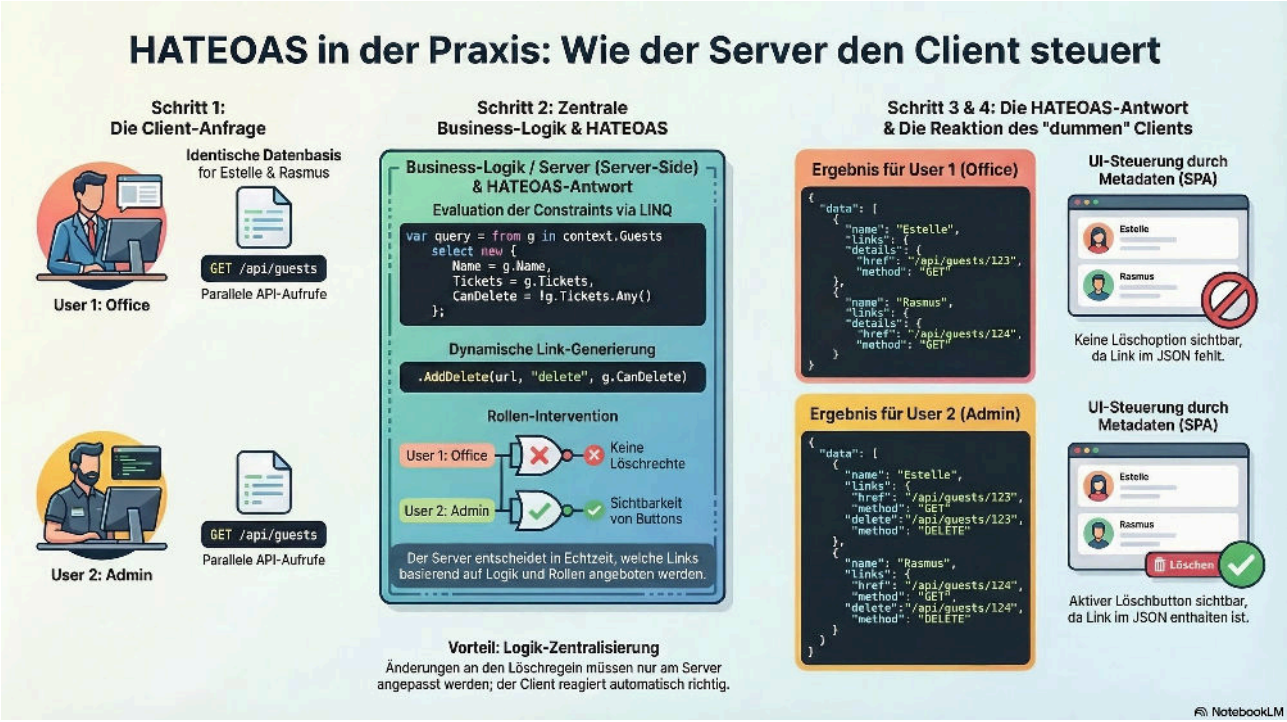
Ein wesentliches Merkmal einer ausgereiften REST-API (Richardson Maturity Model Level 3) ist die Verwendung von *Hypermedia as the Engine of Application State* (HATEOAS). Die Idee dahinter ist einfach: Eine API liefert nicht nur nackte Daten zurück, sondern reichert die Antworten mit Links an, die dem Client mitteilen, welche weiteren Aktionen (Zustandsübergänge) mit dieser Ressource aktuell möglich sind.

Vorteile von Hypermedia

Dies bringt entscheidende Vorteile für die Architektur von Client-Server-Anwendungen mit sich:

- **Zentrale Geschäftslogik am Server:** Der Server generiert die weiteren Möglichkeiten (Links) in Abhängigkeit von internen Constraints und aktuellen Userrechten.
- **Dumme Clients:** Ein Client, wie z. B. eine Single Page Application (SPA), muss diese komplexe Geschäftslogik nicht mehr selbst abbilden. Er verlässt sich rein auf die gelieferten Links. Ein Löschbutton in der UI wird beispielsweise nur dann gerendert und angezeigt, wenn der Server im JSON auch einen *delete*-Link für diese Ressource mitliefert.

Die folgende Illustration zeigt die generierten Hypermedia Links in Abhängigkeit des Users:



Der HAL-Standard und die Realität

Um diese Links im JSON zu strukturieren, haben sich verschiedene Entwürfe etabliert. Einer der bekanntesten ist der *HAL (Hypertext Application Language)* Standard. Dieser schlägt vor, Links

in einem speziellen *_links*-Objekt innerhalb der JSON-Antwort zu bündeln.

Es ist jedoch wichtig zu verstehen, dass es für das JSON keinen fixen, allgemein verbindlichen Aufbau gibt, der von allen Systemen zwingend genutzt werden muss. Wie Links benannt und strukturiert werden, obliegt letztlich dem API-Design der jeweiligen Anwendung.

Dazu hält Microsoft in den Best Practices für API-Design treffend fest:

> There are no general-purpose standards that define how to model the HATEOAS principle. The examples in this section illustrate one possible, proprietary solution.
>
> — Microsoft Azure Architecture Center

Um diesen fehlenden Standard zu veranschaulichen, lohnt sich ein Blick auf ein anderes Ökosystem: **Spring Boot (Java)**. Das Framework nutzt standardmäßig eine striktere Umsetzung des HAL-Standards. Ein dort generierter JSON-Auszug sieht beispielsweise so aus:

```
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/spring-security-rest/api/customers/10A"
    },
    "allOrders": {
      "href": "http://localhost:8080/spring-security-rest/api/customers/10A/orders"
    }
  }
}
```

Im Gegensatz zu unserer späteren eigenen Implementierung (die z. B. die HTTP-Methode explizit als String mitliefert und ein einfaches *links*-Property verwendet), nutzt Spring Boot das Präfix *_links* und verzichtet standardmäßig auf die Angabe der HTTP-Methode, da HAL primär von *GET*-Requests als Standard-Navigation ausgeht. Dies verdeutlicht sehr gut, dass unterschiedliche Frameworks stark voneinander abweichende Ausprägungen von HATEOAS erzeugen.

Table 1. Vergleich der Link-Strukturen

Merkmal	Spring Boot (HAL-Standard)	Eigene Implementierung (ASP.NET Core)
Container-Property	*_links* (mit Unterstrich)	*links* (ohne Unterstrich)
HTTP-Methode	Meist weggelassen (implizit *GET* als Standard)	Explizit angegeben (z. B. *"method": "DELETE"*)
Aufbau der Links	Relationen als Key, Objekt mit *href* -Property	Relationen als Key, Objekt mit *href* und *method*
Fokus	Strikte Einhaltung von Konventionen (HAL)	Pragmatismus, maßgeschneidert für moderne SPA-Frontends

Konkrete Implementierung in ASP.NET Core

Um Hypermedia in unserem Eventmanager zu realisieren, benötigen wir zunächst Basisklassen, die unsere DTOs um eine Link-Auflistung erweitern.

Basisklassen und DTOs

Wir definieren einen Record für den eigentlichen Link (*HypermediaLink*) und eine abstrakte Basisklasse (*HypermediaDto*), von der unsere Rückgabe-DTOs erben können. Ein Dictionary speichert die verfügbaren Links.

→ HypermediaDto.cs

```
public abstract record HypermediaDto
{
    private Dictionary<string, HypermediaLink> _links = new();

    [JsonInclude]
    public IReadOnlyDictionary<string, HypermediaLink> Links => _links.AsReadOnly();

    internal void AddLinkInternal(string rel, HypermediaLink link)
    {
        _links[rel] = link;
    }
}
```

Unser DTO für den Gast erbt nun von dieser Klasse:

→ GuestDto.cs

```
public record GuestDto(
    Id Id,
    string Firstname, string Lastname, DateOnly BirthDate) : HypermediaDto;
```

Einfaches Hinzufügen von Links via Extensions

Damit wir die Links elegant in unseren LINQ-Queries anhängen können, schreiben wir uns Extension Methods. Ein wichtiges Detail hierbei ist der Parameter *useLink*. Ist dieser *false*, wird der Link *nicht* hinzugefügt. Das ist der Schlüssel, um Rechte und Constraints direkt bei der Abfrage auszuwerten.

→ HypermediaExtensions.cs (Auszug)

```
public static class HypermediaExtensions
{
    public static T AddGet<T>(
      this T dto, string url, string rel, bool useLink = true)
      where T : HypermediaDto
        => dto.AddLink(url, rel, "GET", useLink);
```

```
    public static T AddDelete<T>(
      this T dto, string url, string rel, bool useLink = true)
      where T : HypermediaDto
        => dto.AddLink(url, rel, "DELETE", useLink);

    public static T AddLink<T>(
      this T dto, string url, string rel, string method, bool useLink = true)
      where T : HypermediaDto
    {
        if (string.IsNullOrEmpty(url) || !useLink) return dto;

        dto.AddLinkInternal(rel, new HypermediaLink(url, method));
        return dto;
    }
}
```

Verwendung im Controller

Im *GuestsController* sehen wir nun das Zusammenspiel in der Praxis. Wir wollen Gästen, die bereits Tickets besitzen, keinen *delete*-Link mitgeben, da sie aufgrund der referenziellen Integrität (oder unserer Geschäftslogik) nicht gelöscht werden dürfen.

→ *GuestsController.cs*

```
[HttpGet]
[ProducesResponseType<List<GuestDto>>(StatusCodes.Status200OK)]
public async Task<ActionResult<List<GuestDto>>> GetGuests(CancellationToken
cancellationToken)
{
    var guests = (
            await db.Guests
                .Select(g => new { Guest = g, CanDelete = !g.Tickets.Any() })   ①
                .ToListAsync(cancellationToken)
        ).Select(g => new GuestDto(
                Id.From(g.Guest.Id), g.Guest.Firstname,
                g.Guest.Lastname, g.Guest.BirthDate)
            .AddGet(
              GenerateResourceLink(Id.From(g.Guest.Id)), "details")       ②
            .AddDelete(
              GenerateResourceLink(Id.From(g.Guest.Id)), "delete", g.CanDelete)) ③
        .ToList();

    return Ok(guests);
}
```

① Zunächst ermitteln wir schon auf Datenbankebene, ob der Gast Tickets besitzt (*CanDelete*). Dies verhindert eine ineffiziente *Client Evaluation* im RAM.

② Jeder Gast bekommt einen Link zur Detailansicht (*GET /api/guests/{id}*).

③ Der *DELETE*-Link wird nur hinzugefügt, wenn das Constraint *g.CanDelete* den Wert *true* besitzt.

Die Logik, wann ein *DELETE* Link sichtbar ist, wurde hier zur besseren Veranschaulichung im Controller abgebildet. Natürlich kann auch eine Serviceklasse die Liste von *GuestDto* mit entsprechenden Links, die der Business Logik entsprechen, zurückgeben.

Das generierte JSON einer solchen Antwort sieht für den Client dann so aus:

```
[
  {
    "id": "AQAAAGyqe0X_6-HDRGFIqLGetSQ9hZgXV_EMPGF01KKfYwr9",
    "firstname": "Estelle",
    "lastname": "Franzis",
    "birthDate": "2000-05-05",
    "links": {
      "details": {
        "href": "http://localhost:5080/api/Guests/AQAAAGyqe0X_6-
HDRGFIqLGetSQ9hZgXV_EMPGF01KKfYwr9",
        "method": "GET"
      },
      "delete": {
        "href": "http://localhost:5080/api/Guests/AQAAAGyqe0X_6-
HDRGFIqLGetSQ9hZgXV_EMPGF01KKfYwr9",
        "method": "DELETE"
      }
    }
  },
  {
    "id": "AgAAAGJ1xTf-ksLP5BI_kFQzKfou1osvxnXUnmxEw0KRdVRr",
    "firstname": "Rasmus",
    "lastname": "Dippl",
    "birthDate": "2003-04-27",
    "links": {
      "details": {
        "href": "http://localhost:5080/api/Guests/AgAAAGJ1xTf-
ksLP5BI_kFQzKfou1osvxnXUnmxEw0KRdVRr",
        "method": "GET"
      },
      "delete": {
        "href": "http://localhost:5080/api/Guests/AgAAAGJ1xTf-
ksLP5BI_kFQzKfou1osvxnXUnmxEw0KRdVRr",
        "method": "DELETE"
      }
    }
  }
]
```

Der Client erkennt sofort, welche Aktionen erlaubt sind: Sowohl *Estelle* als auch *Rasmus* dürfen in diesem Zustand gelöscht werden (beide haben einen *delete*-Link), weshalb die SPA bei beiden den Löschbutton einblendet. Sobald ein Gast jedoch gebuchte Tickets besitzt, liefert der Server durch unsere Logik (*CanDelete* = *false*) keinen *delete*-Link mehr zurück, woraufhin die SPA den Löschbutton für diesen Eintrag ausblendet oder deaktiviert.

Einsatz in einer SPA (Next.js)

Um den wahren Nutzen unseres HATEOAS-Ansatzes zu demonstrieren, werfen wir einen Blick in das Frontend. In einer React/Next.js-Anwendung müssen wir die Geschäftslogik ("Darf ein Gast gelöscht werden?") nicht mehr duplizieren. Wir verlassen uns vollständig auf die Präsenz der Links im JSON.

Das folgende Beispiel zeigt eine Komponente, die einen einzelnen Gast in einer Liste darstellt. Der Löschbutton wird nur gerendert, wenn der Server den entsprechenden Link mitgeschickt hat. Ist der Link vorhanden, wird die *href* (also die Ziel-URL für den DELETE-Request) an eine separate Lösch-Komponente (*DeleteGuestButton*) übergeben.

```
import React from 'react';
import DeleteGuestButton from './DeleteGuestButton'; // Komponente, die den
eigentlichen DELETE-Request ausführt

interface HypermediaLink {
  href: string;
  method: string;
};

interface Guest {
  id: string;
  firstname: string;
  lastname: string;
  birthDate: string;
  links: {
    details?: HypermediaLink;
    delete?: HypermediaLink;
  };
};

export default function GuestListItem({ guest }: { guest: Guest }) {
  return (
    <div>
      <h3>{guest.firstname} {guest.lastname}</h3>
      <p>Geburtsdatum: {guest.birthDate}</p>

      <div>
        {/* Details-Link immer rendern, falls der Server ihn liefert */}
        {guest.links.details && (
          <a href={guest.links.details.href}>Details</a>
        )}

        {/* HATEOAS in Action: Lösch-Button nur rendern, wenn der Server den Link
anbietet */}
        {guest.links.delete && (
          <DeleteGuestButton deleteUrl={guest.links.delete.href}
          />
        )}
      </div>
    </div>
  );
```

```
}
```

Dieses einfache Muster entkoppelt das Frontend von komplexen Erlaubnis-Regeln und macht die Client-Architektur deutlich robuster gegenüber serverseitigen Regeländerungen.

○ GraphQL mit ASP.NET Core und HotChocolate

Geschichte und Motivation von GraphQL

Um GraphQL wirklich zu verstehen, hilft ein Blick auf seine Entstehungsgeschichte. Etwa im Jahr 2012 stand Facebook (heute Meta) vor einem massiven Problem: Die Nutzung von Smartphones explodierte, aber die mobilen Netzwerke (oft noch 3G) waren langsam und unzuverlässig. Gleichzeitig wurde die Facebook-App – insbesondere der News Feed – immer komplexer.

Facebook nutzte damals klassische REST-APIs. Das führte bei der Entwicklung der mobilen Apps zu zwei massiven Performance-Killern:

- **Under-fetching (zu viele Requests):** Um einen News-Feed-Eintrag anzuzeigen, musste die App zuerst den Post laden (*GET /posts/1*), dann den Autor (*GET /users/123*), dann die Kommentare (*GET /posts/1/comments*) und dann die Autoren der Kommentare. Dutzende Netzwerk-Requests verzögerten den Seitenaufbau enorm.
- **Over-fetching (zu viele Daten):** Alternativ baute das Backend-Team riesige Endpunkte (*GET /feed*), die **alles** zurückgaben. Die App brauchte vielleicht nur den Namen des Autors und den Text des Posts, bekam aber das komplette Profil des Autors inklusive Geburtsdatum, Freundesliste und Profilbild-URLs geschickt. Das verschwendete wertvolles mobiles Datenvolumen und Akku.

Die Lösung: Facebook entwickelte intern eine neue Abfragesprache, bei der nicht der Server bestimmt, welche Daten zurückkommen, sondern der **Client**. Die mobile App konnte nun in einem einzigen Request an einen einzigen Endpunkt exakt definieren, welche Felder sie für den aktuellen View benötigt. Nichts mehr und nichts weniger.

2015 wurde GraphQL als Open-Source-Projekt veröffentlicht und revolutionierte die API-Entwicklung. Seit 2018 wird der Standard von der unabhängigen **GraphQL Foundation** (unter dem Dach der Linux Foundation) gepflegt und heute von Tech-Giganten wie GitHub, Shopify, Netflix und Airbnb eingesetzt.

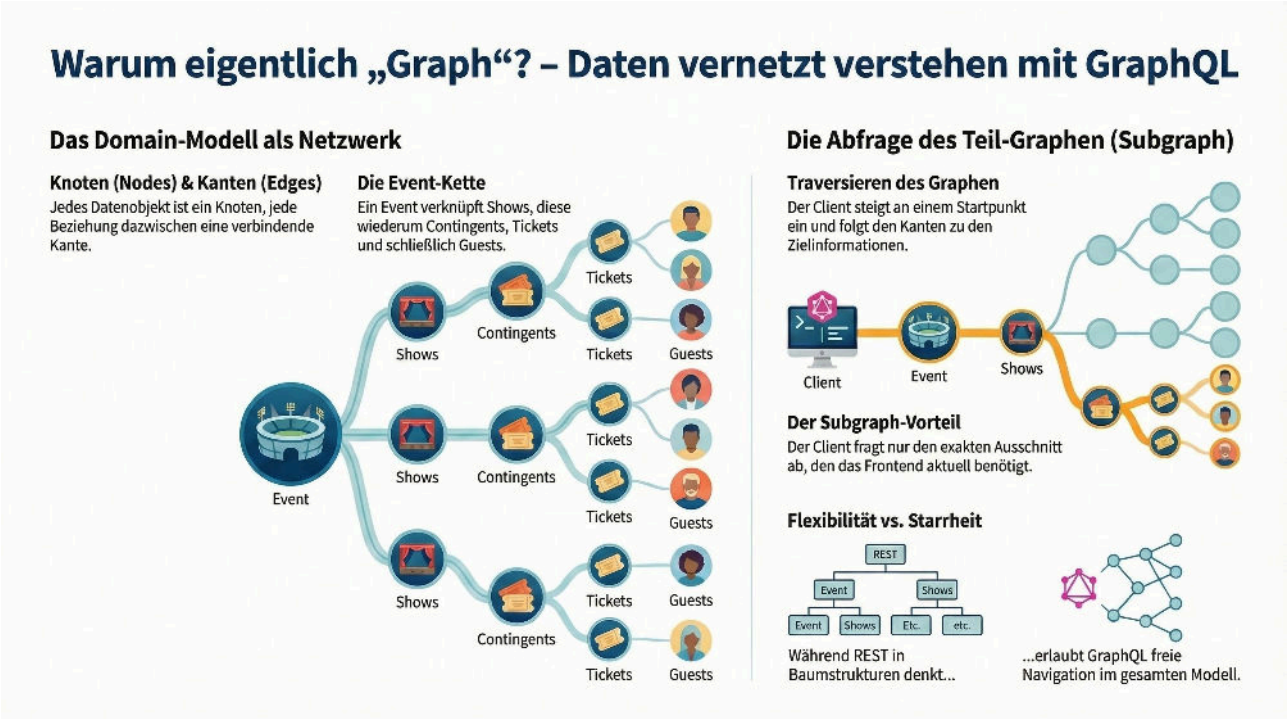
Warum eigentlich "Graph"?

Der Name "GraphQL" leitet sich nicht von grafischen Benutzeroberflächen oder Diagrammen (wie Balken- oder Tortendiagrammen) ab, sondern aus der **Graphentheorie** der Mathematik und Informatik.

In fast jeder Anwendung sind Daten stark relational miteinander verknüpft. In unserem Eventmanager-Beispiel hat ein *Event* mehrere *Shows*, eine *Show* hat *Contingents*, und dort gibt es *Tickets* für *Guests*. Stellt man diese Verbindungen visuell dar, entsteht ein komplexes Netz aus Knoten (Nodes) und Kanten (Edges) – ein sogenannter **Graph**.

Während klassische REST-APIs diese Daten oft in starre, baumartige Dokumente pro Endpunkt zwingen, betrachtet GraphQL das gesamte Domain-Modell als einen einzigen, großen, zusammenhängenden Graphen.

Mit einer Abfrage (Query) steigt der Client an einem bestimmten Knoten (z. B. einem *Guest*) in diesen Graphen ein und navigiert ("traversiert") entlang der Kanten zu den verknüpften Daten (z. B. zu seinen *Tickets* und deren *Shows*). GraphQL erlaubt es also, exakt den "Teil-Graphen" (Subgraph) abzufragen, den das Frontend gerade für die Darstellung benötigt.



GraphQL im Vergleich zu REST

Während wir bei REST-APIs für jede Ressource (z. B. Events, Shows, Guests) eigene Endpunkte definieren und aufrufen, verfolgt **GraphQL** einen fundamental anderen Ansatz.

Bei GraphQL gibt es in der Regel nur einen einzigen Endpunkt (meist */graphql*). Der Client sendet an diesen Endpunkt eine strukturierte Abfrage und definiert **exakt**, welche Datenfelder er zurückbekommen möchte.

Das löst die oben beschriebenen klassischen Probleme von REST (Over-fetching und Under-fetching) elegant.

Ein Rückschritt zu RPC? (Richardson Maturity Model)

Wenn man sich an das **Richardson Maturity Model (RMM)** für RESTful APIs erinnert, könnte man bei GraphQL stutzig werden: Ein einziger Endpunkt (*/graphql*), der für alle Lese- und Schreiboperationen per *POST* aufgerufen wird? Das klingt auf den ersten Blick stark nach **Level 0 (The Swamp of POX / RPC)**, wo HTTP lediglich als Tunnel für Funktionsaufrufe (Remote Procedure Calls) genutzt wurde.

Ist GraphQL also ein Rückschritt in die dunkle Zeit der RPC-APIs? **Nein, ganz im Gegenteil.**

GraphQL entzieht sich bewusst dem Richardson Maturity Model, da das RMM bewertet, wie gut eine API die Prinzipien von **REST und HTTP** (Verben, Caching, URIs) ausnutzt. GraphQL nutzt

HTTP aber explizit nur als **Transportmedium**, nicht als Architekturstil. Der Unterschied zu klassischem RPC liegt in der Intelligenz und Struktur:

- **Client-driven statt Server-driven:** Bei RPC definiert der Server starre Methoden (z. B. *getGuestWithTickets(id)*). Bei GraphQL definiert der Server nur ein zusammenhängendes Datenmodell (einen Graphen). Der Client navigiert selbst durch diesen Graphen und pickt sich die benötigten Daten heraus.
- **Typisierung und Introspektion:** Eine GraphQL-API hat ein stark typisiertes Schema. Der Client kann die API abfragen (Introspektion), um herauszufinden, welche Daten und Relationen überhaupt existieren. Das war bei alten RPC-Ansätzen ohne externe Dokumentation oft unmöglich.
- **Ressourcenorientierung:** Obwohl es nur einen Endpunkt gibt, dreht sich bei GraphQL alles um Ressourcen (Knoten im Graphen) und deren Beziehungen zueinander. Es ist konzeptionell sehr nah an REST, löst das Transportproblem aber auf seine eigene Weise.

Tabelle: GraphQL vs. REST Abfrage

REST	GraphQL
Endpunkte Meist viele (z. B. */api/events*, */api/guests/{id}*)	**Endpunkte** Nur einer (z. B. */graphql*)
Datenstruktur Wird fix vom Server durch DTOs (Data Transfer Objects) vorgegeben.	**Datenstruktur** Wird flexibel vom Client in der Abfrage (Query) definiert.
Beispiel-Request (Gast mit Tickets) Benötigt oft zwei Aufrufe oder stark angepasste DTOs: *GET /api/guests/1 GET /api/guests/1/tickets*	**Beispiel-Request (Gast mit Tickets)** Ein einziger Aufruf: `query { guests(where: { id: { eq: 1 } }) { firstname lastname tickets { ticketState } } }`

Implementierung in ASP.NET Core mit HotChocolate

Um GraphQL in ASP.NET Core zu integrieren, nutzen wir das Framework **HotChocolate**. Es integriert sich nahtlos in Entity Framework Core und macht es extrem einfach, bestehende Datenbankmodelle abzufragen.

Für unser Projekt benötigen wir folgende NuGet-Pakete:

- *HotChocolate.AspNetCore* (für das API-Projekt)
- *HotChocolate.Data.EntityFramework* (für Projektionen, Filterung und Sortierung im Application-Projekt)

Konfiguration des ASP Servers

In der Datei *Program.cs* müssen wir den GraphQL-Server registrieren und unsere Query-Klasse sowie die gewünschten Middleware-Komponenten (Projektionen, Filter, Sortierung) hinzufügen. Wir registrieren außerdem die *DbContextFactory*, damit HotChocolate effizient und threadsicher mit der Datenbank kommunizieren kann.

```
// STEP 1: Configuring ASP.NET Core Services
builder.Services.AddGraphQLServer()
    .AddQueryType<Eventmanager.Application.GraphQL.Query>()
    .AddType<Entity>()
    .AddProjections()
    .AddFiltering()
    .AddSorting()
    .RegisterDbContextFactory<EventContext>();
```

Im Bereich der Request-Pipeline weisen wir ASP.NET Core an, den Endpunkt bereitzustellen:

```
// STEP 2: Configuring ASP.NET Core request pipeline
app.MapGraphQL(); // Standardmäßig unter http://localhost:5080/graphql erreichbar
```

Die Query-Klasse anlegen

Der Kern von GraphQL sind die sogenannten **Queries** (Leseoperationen). Wir legen dafür eine Klasse *Query.cs* an. Dank HotChocolate können wir einfach *IQueryable<T>* direkt aus unserem *EventContext* zurückgeben.

Besonders wichtig sind die Attribute über den Methoden:

- *[UseProjection]*: Sorgt dafür, dass GraphQL-Queries in echte SQL *SELECT*-Statements übersetzt werden (es werden nur die Spalten aus der DB geladen, die der Client auch wirklich anfragt).
- *[UseFiltering]*: Erlaubt es dem Client, Daten zu filtern (z. B. nach Namen).
- *[UseSorting]*: Erlaubt es dem Client, die Ergebnisse zu sortieren.

```
using Eventmanager.Infrastructure;
using Eventmanager.Model;
using HotChocolate.Data;
using System.Linq;

namespace Eventmanager.Application.GraphQL;

public class Query
{
    [UseProjection]
    [UseFiltering]
    [UseSorting]
    public IQueryable<Event> GetEvents(EventContext db)
```

```
        => db.Events;

    [UseProjection]
    [UseFiltering]
    [UseSorting]
    public IQueryable<Show> GetShows(EventContext db)
        => db.Shows;

    [UseProjection]
    [UseFiltering]
    [UseSorting]
    public IQueryable<Guest> GetGuests(EventContext db)
        => db.Guests;
}
```

Codierung von IDs mit dem UseHashedIdAttribute

Im Eventmanager haben wir zusätzlich die Klasse *UseHashedIdAttribute* implementiert. Diese Klasse erbt von *ObjectFieldDescriptorAttribute* und übersetzt den internen Integer-Wert der ID in einen codierten Hash-String. Wir verwenden diese Klasse als Attribut in der Basisklasse *Model/Entity.cs* direkt über der Eigenschaft *Id*. Das ist besonders wichtig, damit die *int* ID bei Abfragen automatisch durch den Konverter codiert wird. Dadurch erscheint die ID im generierten GraphQL-Schema sicher und typisiert als *String!*. Hier ist der entsprechende Code aus der Basisklasse *Entity*:

→ Entity.cs

```
public abstract class Entity
{
    [UseHashedId]
    public int Id { get; set; }
}
```

Und hier ist die vollständige Implementierung des Attributs, das die Konvertierung bei der Schema-Generierung durchführt:

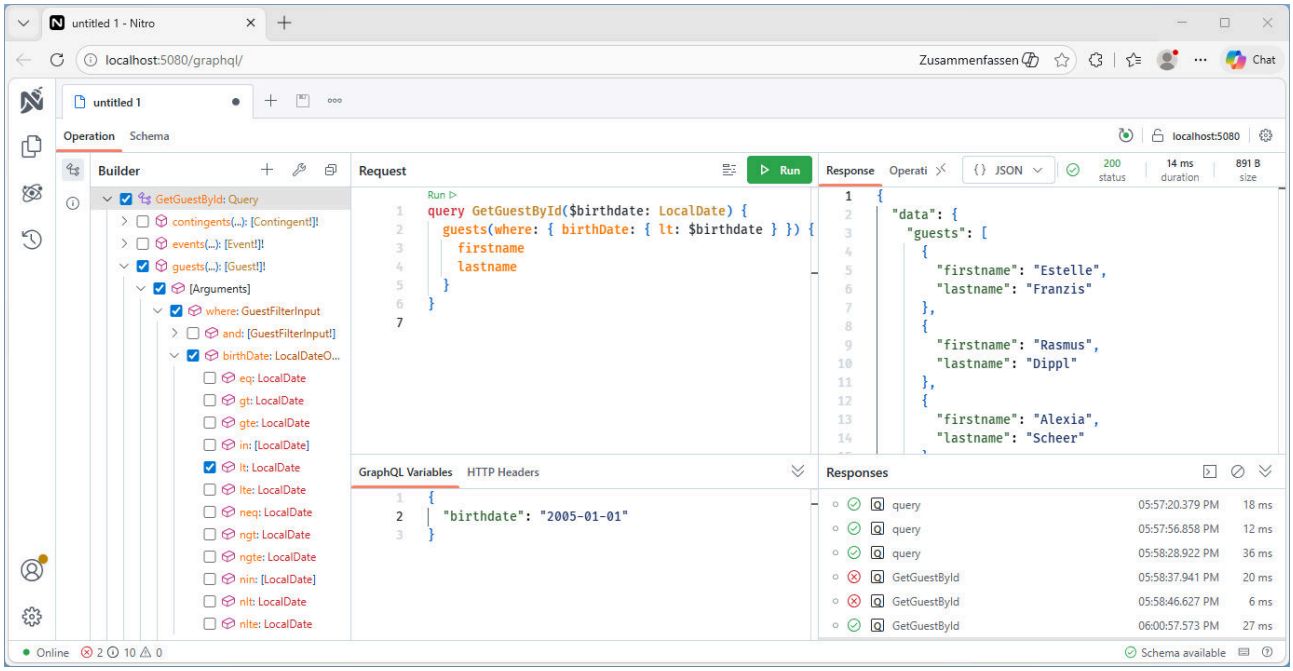
→ UseHashedIdAttribute.cs

```
public class UseHashedIdAttribute : ObjectFieldDescriptorAttribute
{
    protected override void OnConfigure(
        IDescriptorContext context, IObjectFieldDescriptor descriptor, MemberInfo
member)
    {
        descriptor
            .Type<NonNullType<StringType>>()  // int to string
            .Resolve(ctx =>
            {
                var entity = ctx.Parent<Entity>();
                return Id.From(entity.Id).EncodedValue;
            });
```

```
    }
}
```

Abfragebeispiele in Bezug auf das Programm

Wenn du das Projekt startest und im Browser *http://localhost:5080/graphql* öffnest, landest du in **Banana Cake Pop**, der integrierten GraphQL-IDE von HotChocolate. Hier kannst du Abfragen interaktiv testen.



Dank unserer Attribute (*[UseProjection]*, *[UseFiltering]*, *[UseSorting]*) aus der *Query*-Klasse sind sehr mächtige Abfragen möglich.

Beispiel 1: Events mit ihren Shows abfragen (Projektion) Wir holen uns alle Events und dazu direkt die verknüpften Shows (inklusive Datum).

```
query {
  events {
    name
    shows {
      date
    }
  }
}
```

Beispiel 2: Filtern und Sortieren von Gästen Wir suchen nach allen Gästen, deren Nachname mit "S" beginnt, und sortieren sie absteigend nach ihrem Geburtsdatum. Wir fragen nur Vornamen, Nachnamen und Geburtsdatum ab.

```
query {
  guests(
```

```
    where: { lastname: { startsWith: "S" } },
    order: { birthDate: DESC }
  ) {
    firstname
    lastname
    birthDate
  }
}
```

GraphQL Syntax und Möglichkeiten im Detail

GraphQL bietet eine stark typisierte und extrem flexible Abfragesprache. Im Gegensatz zu REST, wo der Server die Struktur der Antwort diktiert, bestimmt bei GraphQL der Client, wie die JSON-Antwort aussehen soll.

Hier sind die wichtigsten syntaktischen Konzepte von GraphQL:

1. Queries (Lesezugriffe)

Das Schlüsselwort *query* definiert einen Lesezugriff (vergleichbar mit einem GET-Request bei REST). Der Client gibt exakt an, welche Felder (*fields*) er von welchen Objekten haben möchte. Relationen werden einfach als verschachtelte Blöcke geschrieben.

```
query {
  events {
    id
    name
    shows {
      id
      date
    }
  }
}
```

Ergebnis: Eine Liste von Events, bei der jedes Event nur die *id*, den *name* und eine Liste seiner *shows* (nur mit *id* und *date*) enthält. Alle anderen Eigenschaften (wie z.B. Kontingente) werden ignoriert, was Bandbreite spart.

2. Argumente (Arguments)

Felder können Argumente entgegennehmen. Bei HotChocolate generieren die Attribute *[UseFiltering]* und *[UseSorting]* automatisch komplexe Argumente wie *where* oder *order* für uns.

```
query {
  guests(
    where: { lastname: { startsWith: "M" } }
    order: { birthDate: DESC }
  ) {
```

```
    firstname
    lastname
  }
}
```

3. Aliase (Aliases)

Manchmal möchte man dasselbe Feld (oder denselben Endpunkt) in einer Abfrage mehrfach mit unterschiedlichen Argumenten aufrufen. Da JSON keine doppelten Schlüssel (Keys) erlaubt, nutzt man **Aliase**, um das Ergebnis umzubenennen.

```
query {
  adultGuests: guests(where: { birthDate: { lt: "2003-01-01" } }) {
    firstname
    lastname
  }
  youngGuests: guests(where: { birthDate: { gte: "2003-01-01" } }) {
    firstname
    lastname
  }
}
```

Ergebnis: Ein JSON-Objekt mit den zwei Schlüsseln *"adultGuests"* und *"youngGuests"*.

4. Fragmente (Fragments)

Wenn Abfragen komplexer werden, fragt man oft an verschiedenen Stellen die gleichen Felder ab. **Fragmente** sind wiederverwendbare Bausteine, die den Code übersichtlich halten (DRY-Prinzip).

```
# Definition des Fragments auf dem Typ "Guest"
fragment GuestDetails on Guest {
  id
  firstname
  lastname
  birthDate
}

query {
  guests {
    ...GuestDetails
    tickets {
      ticketState
    }
  }
}
```

5. Datentypen und das Ausrufezeichen (!)

GraphQL ist stark typisiert. Das bedeutet, jedes Feld und jede Variable hat einen definierten Datentyp.

Die Basis bilden die sogenannten **Skalare** (primitive Datentypen):

- *Int* (Ganzzahl)
- *Float* (Gleitkommazahl)
- *String* (Zeichenkette)
- *Boolean* (Wahrheitswert)
- *ID* (Ein eindeutiger Identifikator, wird oft als String übertragen)

Zusätzlich zu diesen Standard-Skalaren erweitert HotChocolate das Schema automatisch basierend auf unseren C#-Klassen. So wird zum Beispiel ein C# *DateOnly* in GraphQL als *LocalDate* repräsentiert, und ein C# *DateTime* als *DateTime*.

Das Ausrufezeichen (!) Eine Besonderheit von GraphQL ist der Umgang mit Null-Werten. In GraphQL sind standardmäßig **alle** Felder und Argumente "nullable" (sie dürfen *null* sein). Wenn man sicherstellen will, dass ein Wert **niemals** *null* sein darf (es sich also um ein Pflichtfeld handelt), hängt man ein Ausrufezeichen an den Typ an:

- *String*: Ein String oder *null*.
- *String!*: Ein garantierter String (Pflichtfeld, darf nicht *null* sein).
- *[String]*: Eine Liste von Strings. Die Liste selbst kann *null* sein, und die Einträge darin könnten ebenfalls *null* sein.
- *[String!]!*: Eine garantierte Liste (nicht *null*), die garantiert nur Strings (und keine *null*-Werte) enthält.

6. Variablen (Variables)

In der Praxis baut man GraphQL-Strings im Frontend (z. B. in React oder Angular) nicht mit String-Konkatenation zusammen. Stattdessen definiert man Queries mit Variablen (gekennzeichnet durch ein *$*). Die eigentlichen Werte werden als separates JSON-Wörterbuch (Variables-Objekt) mitgeschickt. Hier sieht man auch sehr gut den Einsatz der Datentypen und des Ausrufezeichens.

Die Abfrage (Query):

```
# $birthdate ist eine Variable vom Typ LocalDate (das Ausrufezeichen ! bedeutet
"required/Pflichtfeld")
query GetGuestById($birthdate: LocalDate!) {
  guests(where: { birthDate: { lt: $birthdate } }) {
    firstname
    lastname
    birthDate
  }
```

```
}
```

Das zugehörige Variables-JSON (wird im Request-Body separat mitgesendet):

```
{
  "birthdate": "2005-01-01"
}
```

7. Mutationen (Mutations)

Während *query* zum Lesen gedacht ist, verwendet man das Schlüsselwort *mutation*, um Daten am Server zu verändern (Create, Update, Delete - analog zu POST, PUT, DELETE in REST).

Nach einer erfolgreichen Mutation erlaubt GraphQL es, die veränderten oder neu erstellten Daten sofort im selben Request wieder abzufragen.

```
mutation CreateNewGuest {
  # Angenommener Name der Mutations-Methode am Server
  addGuest(input: { firstname: "Max", lastname: "Mustermann", birthDate: "1990-01-01"
}) {
    # Rückgabe nach dem Erstellen:
    id
    firstname
  }
}
```

Hinweis zur Implementierung: Um Mutationen in HotChocolate zu nutzen, muss in ASP.NET Core eine separate Klasse (z. B. *Mutation.cs*) erstellt und in der *Program.cs* mit *.AddMutationType<Mutation>()* registriert werden.

8. Direktiven (Directives)

Direktiven sind Anweisungen an den GraphQL-Server, wie er die Abfrage ausführen soll. Sie beginnen mit einem @. Die bekanntesten sind *@include* und *@skip*, um Felder bedingt abzufragen.

```
query GetGuests($withTickets: Boolean!) {
  guests {
    firstname
    lastname
    # Das Feld tickets wird nur geladen, wenn die Variable $withTickets true ist
    tickets @include(if: $withTickets) {
      ticketState
    }
  }
}
```

Verwendung in einer SPA (Next.js)

Einer der größten Vorteile von GraphQL ist, dass wir für einfache Abfragen keine schweren Client-Bibliotheken (wie Apollo Client oder Relay) benötigen. Da GraphQL-Requests standardmäßig als gewöhnliche **HTTP POST-Requests** gesendet werden, können wir in modernen Frontends wie Next.js einfach die native *fetch*-API nutzen.

Der Body des Requests muss lediglich ein JSON-Objekt sein, das ein Property *query* (und optional *variables*) enthält. Die Antwort des Servers ist immer ein JSON-Objekt, das im Erfolgsfall ein *data*-Property und im Fehlerfall ein *errors*-Array besitzt.

Das folgende Beispiel zeigt eine typische **Next.js Server Component** (App Router), die Daten aus unserer HotChocolate-API abruft und darstellt:

```
import React from 'react';

// 1. Typdefinitionen für unser GraphQL-Ergebnis
interface Ticket {
  ticketState: string;
}

interface Guest {
  id: number;
  firstname: string;
  lastname: string;
  tickets: Ticket[];
}

interface GraphQLResponse {
  data?: {
    guests: Guest[];
  };
  errors?: any[];
}

// 2. Die asynchrone Fetch-Funktion
async function fetchGuests(): Promise<Guest[]> {
  // Die GraphQL-Abfrage als normaler String
  const query = _
    query {
      guests(order: { lastname: ASC }) {
        id
        firstname
        lastname
        tickets {
          ticketState
        }
      }
    }
  _;

  // POST-Request an unseren ASP.NET Core GraphQL Endpunkt
```

```
  const res = await fetch('http://localhost:5080/graphql', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({ query }),
    // Next.js Caching-Verhalten steuern (z. B. immer neu laden)
    cache: 'no-store'
  });

  const json: GraphQLResponse = await res.json();

  // Fehlerbehandlung: GraphQL liefert bei Queries oft trotzdem HTTP 200,
  // packt die Fehler aber in das "errors" Array im JSON.
  if (json.errors) {
    console.error("GraphQL Errors:", json.errors);
    throw new Error("Fehler beim Laden der Daten aus der GraphQL-API.");
  }

  return json.data?.guests || [];
}

// 3. Die Next.js Server Component
export default async function GuestsPage() {
  const guests = await fetchGuests();

  return (
    <main>
      <h1>Gästeliste (via GraphQL)</h1>
      <ul>
        {guests.map(guest => (
          <li key={guest.id}>
            <strong>{guest.lastname}, {guest.firstname}</strong>
            <br />
            Anzahl Tickets: {guest.tickets.length}
          </li>
        ))}
      </ul>
    </main>
  );
}
```

Wichtige Erkenntnisse für das Frontend:

- **Einziger Endpunkt:** Die URL (*http://localhost:5080/graphql*) bleibt immer gleich, egal welche Daten wir laden.
- **Fehlerbehandlung:** Selbst wenn eine GraphQL-Query fehlschlägt (z. B. wegen eines Tippfehlers im Query-String), antwortet der Server oft mit **HTTP 200 OK**. Der eigentliche Fehler muss im *errors*-Array des Response-JSONs geprüft werden.
- **Typsicherheit:** Das Frontend (*interface Guest*) spiegelt exakt die Struktur der Query wider, nicht zwingend die des gesamten Datenbankmodells.

○ Minimal APIs in ASP.NET Core: Moderne Web-Entwicklung

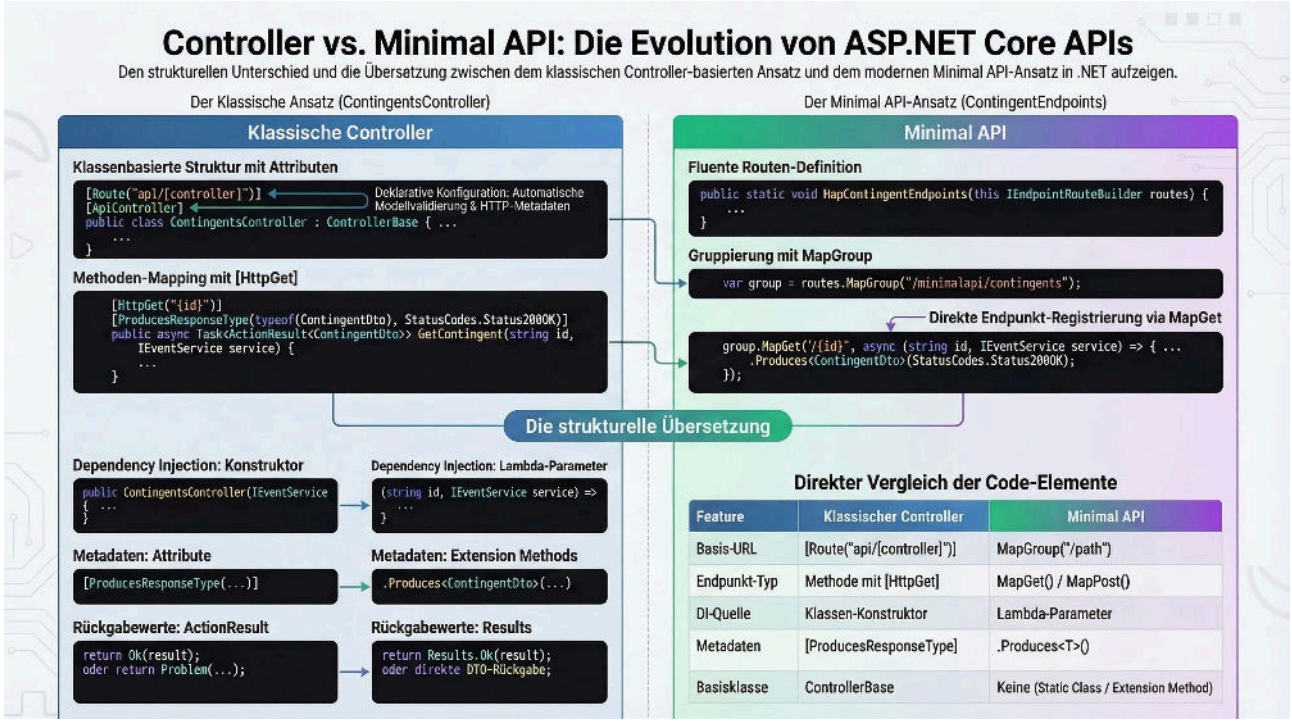
Minimal API vs. Controller

Mit der Einführung neuerer ASP.NET Core Versionen hat Microsoft das Konzept der Minimal APIs etabliert. Ziel war es, die Einstiegshürde für C#-Entwickler massiv zu senken und gleichzeitig die Performance des Webservers zu optimieren. Minimal APIs reduzieren die Anzahl der notwendigen Dateien und Zeilen Code, um einen funktionierenden Endpunkt bereitzustellen. Die Minimal APIs bestehen im Wesentlichen aus *WebApplication*, *WebApplicationBuilder* und Route Handlern. Diese Architektur ist besonders vorteilhaft für moderne Cloud-Native-Anwendungen und Microservices, die oft nur eine begrenzte Anzahl an Aufgaben erfüllen.

Klassische ASP.NET Core Anwendungen nutzen Controller-Klassen, die tief in das MVC-Framework integriert sind. Ein Controller ist eine syntaktisch normale C#-Klasse, die spezielle Attribute für das Routing verwendet. Minimal APIs hingegen brechen mit diesem objektorientierten Zwang und bieten eine funktionale Alternative ohne diesen Klassen-Overhead. Anstatt ganze Klassen zu instanziieren, werden Endpunkte direkt an Handler-Funktionen wie Lambdas gebunden. Diese Architektur führt zu einer flacheren Lernkurve für Einsteiger und einer spürbar schnelleren Ausführungszeit für den Server. Da der MVC-Filter-Pipeline-Overhead entfällt, wird zudem der Speicherverbrauch pro Anfrage deutlich minimiert.

Vergleichstabelle: Endpunkte vs. Controller im Detail

Controller-Ansatz	Minimal API-Ansatz
Basiert auf Klassen, die zwingend von *ControllerBase* erben müssen.	Nutzt kompakte Methoden-Aufrufe direkt am *WebApplication* oder *RouteGroup* Objekt.
Methoden innerhalb der Klasse verarbeiten die Anfragen in einem festen Kontext.	Lambda-Ausdrücke, lokale Funktionen oder statische Methoden dienen als Handler.
Nutzt deklarative Attribute wie *[HttpGet]* oder *[Route]* über den Methodenköpfen.	Nutzt explizite Methoden wie *MapGet()* oder *MapPost()* zur Definition der Routen.
Die Dependency Injection erfolgt klassisch über den Konstruktor beim Instanziieren der Klasse.	Abhängigkeiten werden direkt als Parameter in die jeweilige Handler-Methode injiziert.
Folgt dem strengen MVC-Muster, was oft zu einer Trennung von Logik und Routing führt.	Erlaubt einen funktionalen Ansatz für das gesamte API-Design.
Geeignet für komplexe Anwendungen mit sehr vielen Abhängigkeiten pro Ressource.	Perfekt für schlanke Dienste, die Agilität und maximale Performance priorisieren.



Location-Header: CreatedAtRoute vs. CreatedAtAction

In Controllern nutzen wir oft die Methode *CreatedAtAction*, um auf eine andere Methode innerhalb der aktuellen Klasse zu verweisen. Da Minimal APIs keine Klassen-Methoden im herkömmlichen Sinne besitzen, kann das Framework diesen Bezug nicht automatisch herstellen. Stattdessen verwenden wir in der Minimal-Welt eine angepasste Methode, um die Route anhand ihres Namens zu identifizieren. Um dies nutzen zu können, müssen wir dem Ziel-Endpunkt mit der Erweiterung *WithName()* einen Namen geben. Dieser Name fungiert als eindeutiger Bezeichner innerhalb der internen Routing-Tabelle von ASP.NET Core. Das Framework nutzt diesen Bezeichner zur Laufzeit, um die korrekte URL für den *Location*-Header im 201-Response zu generieren. Dies garantiert dem Client, dass er nach einer erfolgreichen Erstellung die Adresse der neuen Ressource zuverlässig mitgeteilt bekommt.

Implementierung von Endpunkten

In vielen Online-Tutorials werden alle Endpunkte direkt in der Datei *Program.cs* definiert. Bei realen Projekten mit vielen Ressourcen und komplexer Geschäftslogik führt das jedoch zu einer extrem unübersichtlichen Datei. Ein wesentliches Problem in professionellen Entwickler-Teams sind zudem die gefürchteten Merge-Konflikte in der Versionsverwaltung Git. Wenn fünf Entwickler gleichzeitig neue Routen oder Logik in die einzige *Program.cs* schreiben, lässt sich der Code kaum noch fehlerfrei zusammenführen. Minimal APIs müssen jedoch nicht zwingend in der *Program.cs* lokalisiert sein. Durch das Auslagern in separate, thematisch getrennte Dateien bleibt der Code modular und folgt dem Single Responsibility Principle. Dieses Vorgehen ermöglicht es Teams, parallel an verschiedenen Endpunkten zu arbeiten, ohne die zentrale Start-Datei der Applikation zu blockieren.

Exkurs: Was sind Extension Methods?

Um den Code für Minimal APIs in eigene Dateien auszulagern, nutzen wir sogenannte Extension Methods. Dieses Sprachfeature von C# erlaubt es uns, bestehende Klassen oder Interfaces um neue Methoden zu erweitern. Dabei muss der ursprüngliche Quellcode des Typs nicht verändert oder vererbt werden. Eine Extension Method ist immer eine statische Methode innerhalb einer statischen Klasse. Das wichtigste Merkmal ist das Schlüsselwort *this* vor dem allerersten Parameter der Methode. Dieser erste Parameter gibt an, welcher Datentyp genau erweitert werden soll. In unserem API-Beispiel erweitern wir das Interface *IEndpointRouteBuilder*. Für den Aufrufer sieht es später im Code so aus, als wäre die neue Methode ein nativer Bestandteil des erweiterten Objekts.

Mit der Einführung von C# 14 wurde dieses Konzept massiv erweitert. C# 14 fügt eine neue Syntax hinzu, um sogenannte Extension Members zu definieren. Diese neue Syntax ermöglicht es, neben den bekannten Methoden nun auch Extension Properties zu deklarieren. Darüber hinaus können nun auch statische Extension Members definiert werden, die den Typ selbst und nicht nur eine Instanz erweitern. In unserem Projekt benötigen wir dieses neue C# 14 Feature jedoch nicht. Da wir keine Extension Properties oder statischen Erweiterungen für unser Routing-Setup verwenden, reicht die klassische Syntax völlig aus.

Das Interface IEndpointRouteBuilder und Route Groups

Die Methode *MapGroup* hilft dabei, Gruppen von Endpunkten mit einem gemeinsamen Präfix zu organisieren. Indem wir Erweiterungsmethoden für das zugrundeliegende Interface schreiben, können wir unsere Endpunkte in logische Einheiten gruppieren. Dies reduziert wiederholenden Code und erlaubt die Anpassung ganzer Endpunktgruppen mit einem einzigen Aufruf. Dies ist nicht nur syntaktisch sauber, sondern auch technisch effizient, da das Präfix */minimalapi/contingents* nur ein einziges Mal definiert werden muss. Darüber hinaus lassen sich an einer Gruppe zentrale Einstellungen wie Autorisierungsregeln oder Metadaten gesammelt anhängen.

Vollständiger Code: → ContingentEndpoints.cs

```
public static class ContingentEndpoints
{
    public static RouteGroupBuilder MapContingentEndpoints(this IEndpointRouteBuilder
routes)
    {
        var group = routes.MapGroup("/minimalapi/contingents");

        group.MapGet("/{id}", async (int id, IEventService eventService) =>
        {
            // Die Logik zum Laden des Kontingents wird hier implementiert
        }).WithName("GetContingentById");

        return group;
    }
}
```

Dependency Injection mit Minimal API: Effizienz durch Method Injection

Minimal APIs nutzen ein hochoptimiertes Parameter-Binding, das die Dependency Injection vereinfacht. Anstatt Dienste im Konstruktor einer Klasse zu speichern, deklarieren wir sie einfach als Parameter im jeweiligen Lambda-Handler. Das Framework wandelt Anfragedaten in stark typisierte Parameter um und bindet Dienste automatisch aus dem DI-Container. Ein entscheidender Vorteil gegenüber Controllern ist das Ladeverhalten der Dienste. In Minimal APIs wird ein Dienst wie der *IEventService* nur dann instanziiert, wenn die spezifische Route auch tatsächlich aufgerufen wird. Bei Controllern hingegen müssen alle im Konstruktor definierten Abhängigkeiten sofort aufgelöst werden, sobald der Controller angefragt wird. Dies schont die Serverressourcen und beschleunigt die Antwortzeiten bei Routen mit wenigen Abhängigkeiten.

Konfiguration des Servers für die Minimal API

Die Einbindung der modularisierten Endpunkte erfolgt in der *Program.cs* während der Konfiguration der Request-Pipeline. Nachdem die Anwendung mit *builder.Build()* erstellt wurde, steht das Objekt *app* zur Verfügung. Da die *WebApplication* Instanz das entsprechende Routing-Interface implementiert, können wir unsere selbsterstellte Erweiterungsmethode dort einfach aufrufen. Es ist wichtig, diese Registrierung vor dem finalen Start des Servers mit *app.Run()* durchzuführen. Auf diese Weise behalten wir eine saubere Trennung zwischen der Konfiguration der Services und dem eigentlichen Mapping der Endpunkte bei.

```
var app = builder.Build();

app.MapControllers();
app.MapContingentEndpoints();

app.Run();
```

Unittests und Integration Tests: Stabilität durch Entkopplung

Integration Tests bewerten die Komponenten einer App auf einer breiteren Ebene als Unittests. Ein großer Vorteil von ASP.NET Core ist, dass diese Tests konsequent auf der HTTP-Ebene ansetzen können. Wir verwenden dafür die *WebApplicationFactory*, die den gesamten Server inklusive aller Dienste für das System under Test bereitstellt. Es spielt für den Test keine Rolle, ob die Anfrage intern von einer Controller-Klasse oder einer Minimal API verarbeitet wird. Der bestehende Integration Test für den *ContingentsController* kann daher fast ohne Code-Änderungen für die Minimal API übernommen werden. Wir müssen lediglich die Pfadangaben im Test-Client von */api/contingents* auf den neuen Pfad */minimalapi/contingents* anpassen. Diese Entkopplung macht unsere Testsuite extrem robust gegenüber internen Refactorings der Architektur.

Resümee: Wann sollte man welchen Ansatz wählen?

Minimal APIs sind die ideale Wahl für Microservices, kleine Prototypen oder Endpunkte, die eine extrem hohe Performance erfordern. Sie reduzieren das Rauschen im Code und erlauben es, Logik und Routing nah beieinander zu halten. Controller bieten hingegen eine klassische, sehr starre Struktur, die in riesigen Anwendungen mit komplexen Modell-Validierungen Vorteile bieten kann. In Controllern lässt sich eine sehr tief verschachtelte Logik manchmal übersichtlicher in verschiedene Methoden aufteilen. Schüler sollten jedoch lernen, dass es kein striktes "Entweder-oder" geben muss. In modernen Enterprise-Projekten ist es absolut üblich, beide Ansätze je nach spezifischem Anwendungsfall innerhalb derselben Applikation zu kombinieren.